



FUENLABRADA SCHOOL OF ENGINEERING

DEGREE IN ENGINEERING IN
TELECOMMUNICATION TECHNOLOGIES

BACHELOR'S DEGREE PROJECT

SECURE REPORTING SYSTEM AGAINST
USER-SPACE ROOTKITS

Author: Noel Rodríguez Pérez

Supervisor: Gorka Guardiola Múzquiz

Academic year 2025-2026



This work is distributed under the terms of the international CC BY-NC-ND International License (Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International). You are free to *share*: copy and redistribute the material in any medium or format. The licensor cannot revoke these freedoms as long as you follow the license terms:

- *Attribution*. You must give appropriate credit, provide a link to the license and indicate if changes were made. You may do so in any reasonable manner, but not in any way that suggests the licensor endorses you or your use.
- *NonCommercial*. You may not use the material for commercial purposes.
- *NoDerivatives*. If you remix, transform or build upon the material, you may not distribute the modified material.

No additional restrictions — You may not apply legal terms or technological measures that legally restrict others from doing anything the license permits.

You do not have to comply with the license for elements of the material in the public domain or where your use is permitted by an applicable exception or limitation. No warranties are given. The license may not give you all the permissions necessary for your intended use. For example, other rights such as publicity, privacy or moral rights may limit how you use the material.

Document by **Noel Rodríguez Pérez.**

Acknowledgements

First of all, I would like to thank my supervisor, **Gorka Guardiola Múzquiz**, for his constant support, patience and dedication throughout the development of this project. All the knowledge he has shared with me has been essential not only for this work, but also for helping me understand the genuine interest I have in the field of cybersecurity. Thank you for teaching me so much.

To all the lecturers of my degree, for providing me with the knowledge that now allows me to close this stage and begin my professional career in the best possible way.

Also, to all the people around me, for accompanying me throughout this process and showing me their support during all this time.

I would especially like to thank my family and my partner, for giving me the education, motivation and support that have brought me to where I am and made me the person I am today. And to my university group, for making this stage much more enjoyable and for helping me enjoy the process, even in the most difficult moments.

Thank you very much for everything.

*To my family, friends and partner,
my greatest support throughout this stage.*

Madrid, May 31, 2026

Noel Rodríguez Pérez

Abstract

Security in modern systems is a constantly evolving field in which data integrity and control over system operation play a fundamental role. This is due to the presence of threats and external agents whose malicious purpose is to compromise such information, making it necessary to develop increasingly robust protection mechanisms.

In this context, one of the most critical threats currently affecting information systems is represented by *rootkits*, malicious software designed to hide and manipulate system resources or activity, such as processes, files or network connections. These threats may operate both in *User Space* and in *Kernel Space*, compromising the information received by both the administrator and the user and preventing them from obtaining a truthful view of the real state of the system. This undermines the integrity of monitoring and control tasks, potentially leading the administrator to trust information sources that may have been compromised.

To address this problem, this project designs and implements a *Loadable Kernel Module* (*LKM*) on a *Linux* system with the aim of providing an authenticated source of critical system information against possible manipulation. The solution introduces a cryptographic authentication mechanism based on *HMAC* for the content provided by the *kernel*, using a secret symmetric key shared only between the administrator and the module. This approach relies on a chain of trust in which it is assumed that the *kernel* and the *LKM* have not been modified, for example through signing mechanisms for both the *kernel* and the module, thereby reinforcing the integrity of the information provided.

The experiments carried out approximate a real operating environment in which sensitive information is requested from *User Space* to *Kernel Space*. For this purpose, the sensitive system content to be authenticated is defined as a list of all running processes and their corresponding metadata, generated inside the *Loadable Kernel Module* itself and used as an example of critical information that may be manipulated

by a *rootkit*. An *HMAC* is computed over this content using the corresponding secret key and is attached to the information provided to the user, so that its integrity and authenticity can be verified.

To validate this proposal, user-level components were developed to test the functionality. A first component creates a file whose descriptor is transferred to the *kernel* so that it can reference it, thus establishing the secure communication channel with the core of the system. On the other hand, the integrity of the reported data is confirmed through a second verification component that analyzes the information written by the *kernel*, locally computes the *HMAC* of the content using the same secret key and then compares it with the received value to ensure that both match and that the content is authentic.

Acronyms

API *Application Programming Interface*

CIA *Confidentiality, Integrity and Availability*

CLI *Command Line Interface*

CPU *Central Processing Unit*

RAM *Random Access Memory*

DRAM *Dynamic Random Access Memory*

FD *File Descriptor*

GNU *GNU's Not Unix*

GPL *GNU General Public License*

OS *Operating System*

MAC *Message Authentication Code*

HMAC *Hash-based Message Authentication Code*

SHA *Secure Hash Algorithm*

ID *Identifier*

LKM *Loadable Kernel Module*

PCB *Process Control Block*

US *User Space*

KS *Kernel Space*

NS *Namespace*

UID *User Identifier*

PID *Process Identifier*

GID *Group Identifier*

PS *Process Status*

VM *Virtual Memory*

VFS *Virtual File System*

FS *File System*

FUSE *Filesystem in Userspace*

FHS *Filesystem Hierarchy Standard*

MMU *Memory Management Unit*

BSS *Block Started by Symbol*

PC *Program Counter*

SP *Stack Pointer*

SYS *System*

BUF *Buffer*

GFP *Get Free Pages*

UTS *UNIX Time-Sharing*

IPC *Inter-Process Communication*

EDR *Endpoint Detection and Response*

HIDS *Host-based Intrusion Detection System*

PoLP *Principle of Least Privilege*

RAT *Remote Administration Tool*

RCU *Read-Copy-Update*

Contents

1	Introduction	1
1.1	Cybersecurity	2
1.1.1	Vulnerabilities and Attacks	4
1.1.2	Defenses	5
1.1.3	Rootkits	7
2	Technological Framework	10
2.1	<i>Linux</i> Operating Systems	10
2.1.1	Architecture and Privilege Rings	12
2.1.2	Memory: <i>User Space</i> and <i>Kernel Space</i>	14
2.1.3	Files and File Systems (<i>FS</i>)	16
2.1.4	<i>Namespaces</i>	19
2.1.5	<i>Linux</i> Kernel Modules (<i>LKMs</i>)	23
2.1.6	Development Environment	25
3	State of the Art	29
3.1	User-Space Rootkits	30
3.2	Kernel-Space Rootkits	31
4	Problem Statement and Objectives	33
4.1	Problem Description	33
4.2	Threat Model	34
4.2.1	Operational Context and Security Assumptions	35
4.2.2	Attacker Capabilities and Limitations	36
4.2.3	Defensive Coverage	37
4.3	Objectives	37
4.3.1	Functional Objectives	38
4.3.2	Non-Functional Objectives	39
4.4	Methodology	40

4.5	Work Plan	41
5	Design and Development	44
5.1	Prototypes, Implementations and Validations	44
5.1.1	<i>Hello World</i>	45
5.1.2	Virtual File <code>/proc/fdev</code>	47
5.1.3	<i>File Handoff</i>	49
5.1.4	<i>HMAC-SHA256</i> Cryptographic Authentication Mechanism . . .	51
5.1.5	Secure Reporting System for Running System Processes	56
5.2	Problems and Errors Encountered	61
5.2.1	Writing to the Transfer File in <i>Append</i> Mode	62
5.2.2	Integration of the <i>HMAC-SHA256</i> Mechanism between <i>Kernel</i> <i>Space</i> and <i>User Space</i>	63
5.2.3	Safe Traversal of the Process List Using <i>RCU</i>	64
6	Conclusions	67
6.1	Achievement of Objectives	68
6.1.1	Achievement of Functional Objectives	68
6.1.2	Achievement of Non-Functional Objectives	70
6.2	Social, Economic, Temporal, Environmental and Ethical Impact	71
6.3	Skills Acquired and Used	73
6.4	Future Work	75
A	Code and Validation of the Prototypes	81
A.1	Prototypes, Implementations and Validations	81
A.1.1	<i>Hello World</i>	81
A.1.2	Virtual File <code>/proc/fdev</code>	85
A.1.3	<i>File Handoff</i>	90
A.1.4	<i>HMAC-SHA256</i> Cryptographic Authentication Mechanism . . .	97
A.1.5	Secure Reporting System for Running System Processes	112
	Bibliography	125

List of Figures

1.1	Main types of <i>malware</i>	3
1.2	Characteristic properties of a <i>rootkit</i>	7
2.1	Family tree of <i>Unix</i> operating systems.	11
2.2	Privilege rings in modern operating systems.	12
2.3	Structure of a modern operating system.	13
2.4	Logical memory division between <i>User Space</i> and <i>Kernel Space</i>	15
2.5	Standard hierarchical directory structure in <i>Linux</i> (<i>FHS</i>).	17
2.6	File-system management in <i>Linux</i>	18
2.7	<i>Mount point</i> : multiple file systems in a single logical tree.	19
2.8	Types of <i>Namespaces</i>	21
2.9	Example of a <i>Docker</i> architecture.	23
2.10	Physical system used in the project.	26
2.11	Armbian distribution logo for Raspberry Pi 4 Model B.	27
3.1	General classification of <i>rootkits</i> in <i>Linux</i> systems.	29
4.1	Authenticated reporting mechanism between <i>User Space</i> and <i>Kernel Space</i>	34
4.2	Diagram of the <i>Spiral</i> methodology used in the project.	41
5.1	Basic flow for loading, executing and unloading an <i>LKM</i>	45
5.2	Basic communication between <i>User Space</i> and <i>Kernel Space</i> through a virtual file in <i>/proc</i>	47
5.3	Transfer of a <i>File Descriptor</i> from a real file to the <i>LKM</i> to establish a communication path from <i>Kernel Space</i> to <i>User Space</i>	49
5.4	Incorporation of an <i>HMAC-SHA256</i> authentication mechanism into the <i>File Handoff</i> system.	53
5.5	Generation and authenticated transfer of a report of the active system processes from <i>Kernel Space</i>	57

List of Code Listings

2.1	Example output of the <code>ps aux</code> command in a Linux terminal.	20
2.2	Basic LKM management commands.	25
2.3	Installation of the development environment.	27
5.1	Example of the final structure of the authenticated transfer file.	60
A.1	Generic <i>Makefile</i> used to compile <i>LKMs</i>	82
A.2	Headers and metadata of the <i>Hello World</i> module.	83
A.3	Initialization/load and exit/unload functions of the <i>Hello World</i> module.	83
A.4	Association of the module entry/load and exit/unload points.	84
A.5	Compilation, loading, inspection and unloading of the <i>Hello World</i> prototype.	84
A.6	Additional headers for the prototype based on <code>/proc</code>	85
A.7	Reference to the virtual entry and associated operations table in <code>/proc</code>	86
A.8	<i>Handlers</i> with data transfer between <i>User Space</i> and <i>Kernel Space</i>	88
A.9	Basic validation of the prototype based on <code>/proc/fdev</code>	89
A.10	Additional headers for the <i>File Handoff</i> mechanism.	90
A.11	Read function for the <i>File Handoff</i> mechanism.	91
A.12	Auxiliary function for writing to a file from the <i>LKM</i>	91
A.13	Auxiliary function for validating file permissions from the <i>LKM</i>	92
A.14	Write function for the <i>File Handoff</i> mechanism.	93
A.15	<i>User-Space</i> program <code>mk_fhandoff.c</code> to activate <i>File Handoff</i>	95
A.16	Validation of the <i>File Handoff</i> mechanism.	97
A.17	Additional headers and embedded key for the <i>HMAC-SHA256</i> authentication mechanism.	97
A.18	<code>gen_k_embedded.sh</code> script to generate the <code>k_embedded.h</code> header with the embedded key.	98
A.19	Calculation of <i>HMAC-SHA256</i> for a data <i>byte</i> string.	99
A.20	Conversion of the binary <i>HMAC</i> to <i>Base64</i>	100

A.21 Joint writing of the content and its <i>HMAC</i> in <i>Base64</i>	101
A.22 Joint integration of the authentication mechanism.	103
A.23 Write handler adapted to the content-authentication mechanism.	104
A.24 Headers, constants and symmetric key required for verification.	105
A.25 Deserialization of the authenticated file content in <i>User Space</i>	106
A.26 Local <i>HMAC</i> calculation and <i>Base64</i> encoding.	108
A.27 Authenticator comparison and main verifier flow.	110
A.28 Validation of the authentication mechanism using <i>HMAC-SHA256</i>	111
A.29 Headers and constants for generating the process list.	113
A.30 Auxiliary functions for formatting and safely building serialized content.	114
A.31 Construction of the global process list in <i>kernel</i> memory.	115
A.32 Serialization of the metadata of one process.	117
A.33 User identifiers of each process.	119
A.34 Process identifiers of each task.	120
A.35 Group identifiers and command name associated with each process.	121
A.36 Validation of the secure metadata reporting system for active processes authenticated using <i>HMAC-SHA256</i>	122

Chapter 1

Introduction

“The worst enemy of security is complexity.”

Bruce Schneier (*A Plea for Simplicity*, 1999) [1].

Nowadays, the security of computer systems is a fundamental aspect to consider, becoming increasingly important as we store more sensitive information in them and as they become much more complex systems. Therefore, they may be exposed to a large number of threats if they are not properly protected. As the essay quoted at the beginning of this chapter warns, the inherent complexity of these infrastructures facilitates the proliferation of *malware* designed to operate and infiltrate from the strictest invisibility.

A modern operating system is not a simple structure or process, but rather an interwoven set of concentric layers or levels of abstraction that operate with respect to a hardware system. It can be described as a truly complex system, with a large number of components and functionalities. This complexity makes it difficult to protect and monitor, requiring deep knowledge of the system in order to know how to react to possible attacks.

At the same time, from the attacker’s point of view, significant technical knowledge is required to design an effective attack, but also creativity and skill to find vulnerabilities and ways to exploit them without being detected, depending on the attacker’s goal.

This makes cybersecurity a truly broad and complex discipline, in which attackers and defenders are engaged in a constant race to find new ways to compromise or protect systems.

This project focuses on the security of *Linux* operating systems, specifically on the design of a defensive mechanism that provides critical system information in an authenticated and verifiable way against possible manipulation.

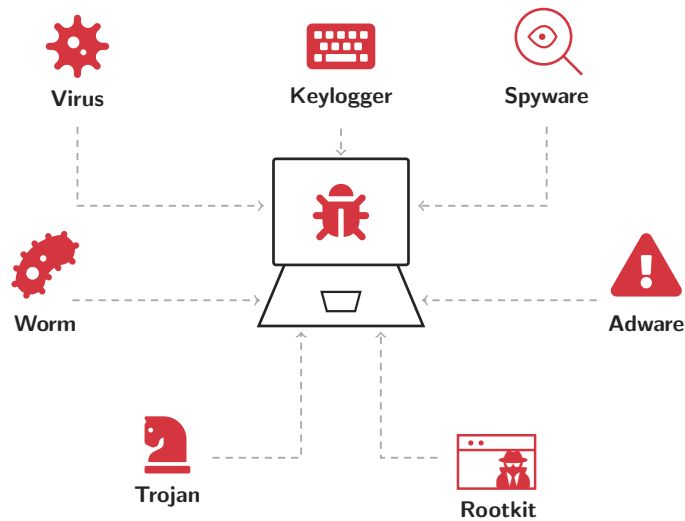
In order to understand the described scenario in detail and contextualize the topic of the project, the following sections introduce basic cybersecurity concepts and their main purposes, especially the main threat on which this project focuses: *rootkits*.

1.1 Cybersecurity

In the field of information systems, cybersecurity comprises the set of architectures, technologies and practices designed to protect systems, data and networks against attacks that may compromise their integrity, confidentiality or availability, whether intentional or not. However, from a technical point of view, it must be understood as a balance between the value of the assets to be protected, the risk associated with their compromise and the cost of the measures required to guarantee proportional protection.

From this perspective, protecting a system does not consist of indefinitely maximizing defenses, but of applying mechanisms that are proportional to the real impact of a possible attack.

The main flow of these threats is materialized through **malicious software** (*malware*), a concept that includes any code designed to infiltrate a system, damage it or steal data without the consent of its owner [2].



Fuente: Own elaboration based on Akamai Technologies, What is malware? [3], using vector iconography from Font Awesome Free [4].

Figure 1.1: Main types of *malware*.

There are currently several families of *malware*, mainly classified according to their behavior and objective. On the one hand, threats aimed at rapid infiltration and propagation stand out, such as **viruses**, **worms** and **Trojans**, which exploit technical vulnerabilities or psychological deception of the user to access the system. On the other hand, there are variants focused on espionage and/or financial gain, such as **ransomware**, which encrypts critical or personal data and then demands payment, and **spyware** together with **keyloggers**, designed to monitor and extract sensitive information from the system without being noticed.

Nevertheless, within this offensive ecosystem there is one category that stands out due to its extreme criticality and constitutes the main focus of this project: **rootkits**. Unlike other *malware* families focused on propagation, espionage or direct financial profit, *rootkits* are characterized by their ability to hide and manipulate system resources and activity, compromising the integrity of the information provided by the system. These threats may operate both in *User Space* and in *Kernel Space*, representing a major threat to the security of current information systems. Their operation will be discussed in greater detail later.

However, for this *malware* to infiltrate the system infrastructure, attackers need access methods and persistence mechanisms. They also require proper analysis, identification and exploitation of system vulnerabilities through which they can

infiltrate [5].

1.1.1 Vulnerabilities and Attacks

In the context of system cybersecurity, a **vulnerability** is defined as a weakness in the design, implementation or configuration of a system that can be exploited by a malicious agent to compromise its integrity, confidentiality or availability [6].

In this context, the **MITRE ATT&CK** framework becomes highly relevant, as it is a standardized source of information that classifies the techniques and tactics used by attackers during an intrusion. Its usefulness lies in the fact that it allows threats to be described using a common language and the detection, monitoring, robustness and response capabilities against these threats to be evaluated in a structured way [5].

These security gaps constitute the main entry point for intruders and can be broadly classified into three major groups according to their nature:

- **Software and hardware vulnerabilities:** Flaws at the level of the operating system source code, applications or libraries, as well as in physical components or in the interaction between them. In the case of *software*, they may arise from insufficient input validation, code injection or memory errors such as *buffer overflows*. In the case of *hardware*, attacks such as *Rowhammer* exploit physical effects in *DRAM* memory to modify bits in nearby memory locations. On the other hand, attacks such as *Spectre* exploit the interaction between *software* and internal processor optimizations, such as speculative execution, to infer sensitive information through side channels.
- **Human vulnerabilities:** Caused by lack of awareness, mistakes or negligence by system users, who may become the weakest link in the security chain.
- **Configuration vulnerabilities:** Insecure design or implementation of the system architecture itself, such as the use of default credentials, exposed network services or incorrect privilege assignment.

To exploit these vulnerabilities and carry out the *malware* intrusion, attackers use various **attack tools** and techniques specific to each scenario. Some examples¹:

- **Social engineering attacks:** Aimed at exploiting human vulnerabilities. *Phishing* and its variants (*spear phishing*) stand out, with the objective of psychologically manipulating the user through fraudulent communications that lead to revealing credentials or executing malicious *software* inadvertently.
- **Exploits:** To attack through configuration or *software* flaws, attackers use *exploits*. These are code fragments designed to take advantage of a technical weakness and force unexpected behavior in the machine. Their main objective is often to inject a triggering payload, such as **backdoors** or remote administration tools (**RATs**).
- **Privilege escalation attacks:** Once the attacker has minimally compromised the system, offensive tools are used to exploit secondary vulnerabilities in the same system and raise permissions, potentially obtaining administrator control (*root*).
- **Persistence and concealment:** After gaining access to the system, the attacker may use techniques to maintain presence and hinder detection. This is the context in which *rootkits* appear, as they can hide processes, files or connections by manipulating the information visible to the user.

In short, regardless of the vulnerability detected and the offensive technique used, there is a critical need to implement robust defense mechanisms and secure state monitoring. This, together with proper user education and awareness, is essential to mitigate risks and largely protect the system.

1.1.2 Defenses

To mitigate and protect against the main threats in this field, a strategic approach based on the principle of **Defense in Depth** was created. This concept consists of deploying multiple overlapping security layers, so that if one control fails or is bypassed, the attacker may encounter another one later, making access more difficult. Mainly

¹For a detailed breakdown of exploitation phases and techniques, MITRE ATT&CK may be consulted [5].

through this approach, defensive methods can be divided into two categories [7].

First, **preventive measures** seek to reduce the attack surface and prevent the intrusion from materializing. The most relevant practices include:

- ***Security Awareness***: Educating users to identify and report social engineering attacks, reducing the human error factor.
- ***Hardening***: Keeping applications and the operating system updated to mitigate possible *exploits*. It also involves secure system configuration, such as disabling unnecessary services or restricting network traffic through the use of ***firewalls***.
- ***Principle of Least Privilege (PoLP)***: Ensures that users, processes and applications operate only with the minimum and strictly necessary permissions required to perform their function. This drastically limits the impact of an initial access breach and makes privilege escalation more difficult.

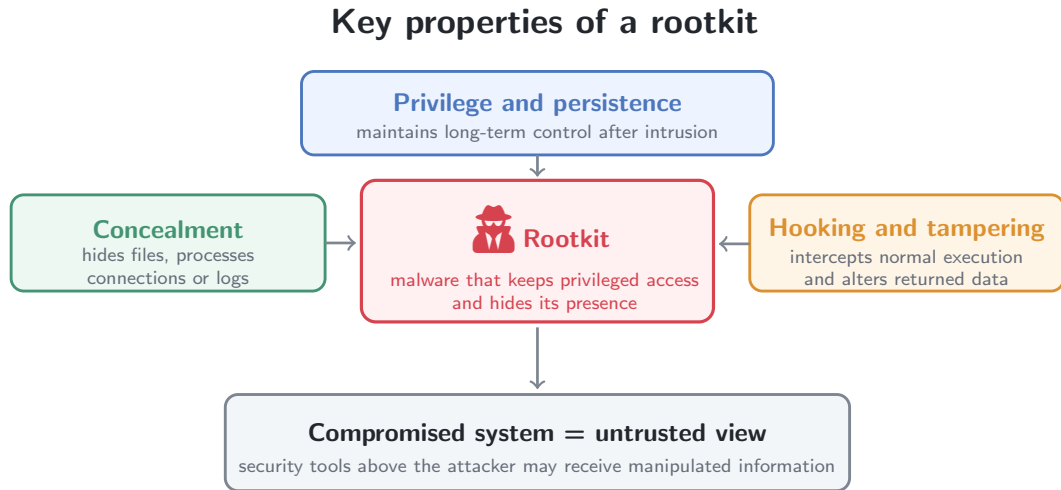
On the other hand, assuming that preventive barriers can be bypassed, it is essential to implement continuous **detection, response and monitoring** mechanisms [8]:

- ***Host-based Intrusion Detection Systems (HIDS)***: Tools that monitor machine activity, analyze event records (*logs*) and detect unauthorized alterations recorded in critical files (*File Integrity Monitoring*).
- ***Endpoint Detection and Response platforms (EDR)***: An evolution of the traditional antivirus concept. Instead of relying solely on signatures, they analyze process behavior in memory in real time in order to detect and block anomalous executions.

It should be noted that the field of cybersecurity is constantly evolving. The continuous sophistication and improvement of offensive techniques requires a major effort from the industry to research and design new protection methods. Thus, current defense paradigms, such as those explained in this section, must dynamically adapt to the temporal context of each environment in which they are used, since they may become obsolete and ineffective in the short term.

1.1.3 Rootkits

In this cybersecurity context, a **rootkit** is defined as a set of *malware* fragments specifically designed to maintain privileged access to a system and, at the same time, hide its presence and the activities selected by the attacker, such as processes, network connections or files, from administrators and security *software* [9].



Fuente: Own elaboration based on National Institute of Standards and Technology (NIST), Rootkit [10].

Figure 1.2: Characteristic properties of a *rootkit*.

Unlike other threats, their main objective is not to propagate or cause immediate destructive damage, but to achieve **undetectable persistence** in the long term after an initial intrusion into the target system. To reach this level of evasion, *rootkits* alter the normal execution flow of the operating system through interception and code modification techniques, typically known as **hooking**.

Depending on the layer in which they operate and the level of privileges they can escalate to, they can be divided into two broad categories in *Linux* systems:

- **User-Space Rootkits:** They operate in the least privileged region (*Ring 3*). The most common technique consists of capturing functions from the standard C library (`libc`) through mechanisms such as dynamic library loading using the `LD_PRELOAD` variable. In this way, they intercept requests to such functions (*hooks*), such as `open` or `read`, before they reach the *kernel*, hiding and filtering the output returned to the user. Recent research shows that classical detection

tools operating at the normal user privilege level are easily evaded, generating a false sense of security [9].

- ***Kernel-Space Rootkits:*** They represent the most critical threat, since they operate at the highest privilege level (*Ring 0*). They are usually implemented through *Loadable Kernel Modules (LKMs)*. Since they reside in the system *kernel*, they acquire administrator privileges over critical data structures and the System Call Table (*sys_call_table*). This allows them to manipulate the system from within, rendering monitoring tools operating above them with fewer privileges completely useless.

The fundamental challenge posed by these threats lies in a critical cybersecurity axiom: ***a compromised system cannot provide reliable information about itself***. Based on these arguments, attempting to detect or mitigate the action of a *rootkit* using security mechanisms that operate at a lower privilege level than the attacker is a completely ineffective strategy. For this reason, robust defensive and monitoring architectures must be designed in the deepest layers of the system in order to guarantee data integrity and the correct operation of the environment.

Taking as a reference the complex cybersecurity landscape described above and the major threat posed by *rootkits*, this project proposes the design and development of a secure reporting system for *Linux* environments, focused on providing critical system information in an authenticated and verifiable way against possible manipulation by this type of *malware*. Before that, the next chapter establishes the technological framework of current *Linux* operating systems, detailing their design, architecture and operation, thus facilitating the understanding of the project. Subsequently, the objectives and methodologies adopted for the design, integration and validation of this secure reporting system will be addressed.

Chapter 2

Technological Framework

In order to design and implement an effective security solution against the threats posed by *rootkits*, a detailed prior analysis of the technological environment in which they operate is required. For this reason, this chapter first analyzes the architecture and structure of *Linux* operating systems, detailing their internal mechanisms, such as file management, memory management and modularity.

2.1 *Linux* Operating Systems

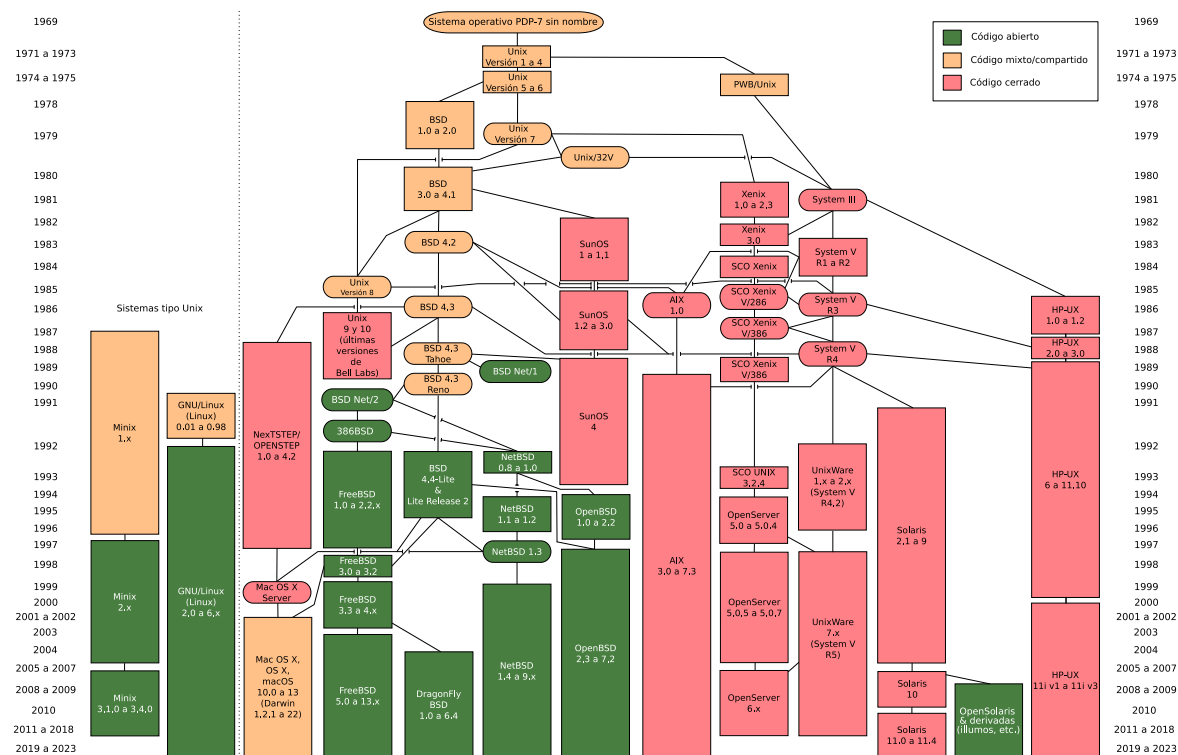
There is no canonical and universal definition of the concept of an operating system, since it is an idea that is, to some extent, relative and subjective depending on its purpose and context of use [11].

One possible definition is: the set of programs that makes it possible to use a machine as a system, doing what is necessary for multiple processes to run on the same *hardware*, multiplexing the machine in time by managing *CPU* usage and in space by managing *RAM*.

Another possible definition would be: the program, or set of programs, that acts as an intermediary between the user and the *hardware*, providing an interface as an abstraction of internal processes and instructions in order to interact with the machine and its resources efficiently and robustly.

In today's technology market there are different general-purpose operating systems, including closed-source commercial options such as *Microsoft Windows* or *macOS*, generally aimed at computers, and *Android* and *iOS*, mainly designed for mobile devices. However, within the field of infrastructure administration, cybersecurity and

software development, *GNU/Linux*, belonging to the *Unix* family, is considered the absolute reference standard.



Fuente: VARGUX, File:Unix history-simple es.svg (2023) [12].

Figure 2.1: Family tree of *Unix* operating systems.

As the previous figure illustrates, *Unix* is the pioneering system created in the 1970s that laid the foundations of modern computing, from which *Linux* was born as a clone developed entirely under an open-source model. *macOS* also belongs to this family; however, this operating system has evolved under a semi-proprietary commercial model with partially closed source code.

The hegemony of *Linux* (*GNU/Linux*) in these sectors is mainly due to its open-source philosophy¹. Having the source code publicly available, either through local installation of system headers and sources, typically located in development environments under `/usr/src`, or through public online repositories such as *Git*, allows a large global community of developers to audit, debug and continuously improve the code. This transparency guarantees a much higher level of adaptability and reliability

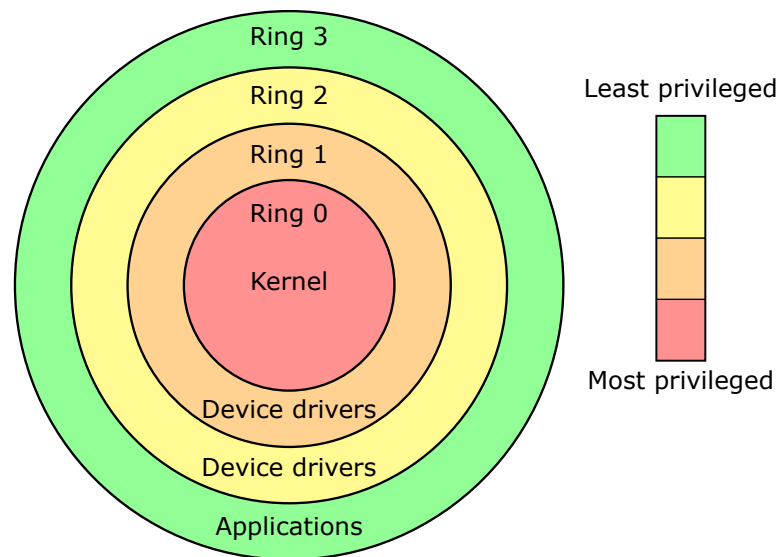
¹Open source focuses on source-code availability, review and collaborative development. Free software, on the other hand, emphasizes the user’s freedoms to run, study, modify and redistribute the code, although both approaches often overlap in many projects.

than that of commercial systems, allowing the environment to be configured according to the required needs.

For these reasons, *Linux* is considered the optimal operating system for developing this project. From this point onward, it will be assumed by default that the environment is *Linux*, although the fundamental principles can be extrapolated to any modern operating system.

2.1.1 Architecture and Privilege Rings

To guarantee their stability and security, modern operating systems do not rely solely on logical *software* restrictions, but are tightly supported by protection mechanisms implemented at the *hardware* level by the processor itself. In predominant system architectures such as the x86 family, this isolation mechanism is typically implemented through concentric hierarchical protection domains, commonly known as *Protection Rings* [11].



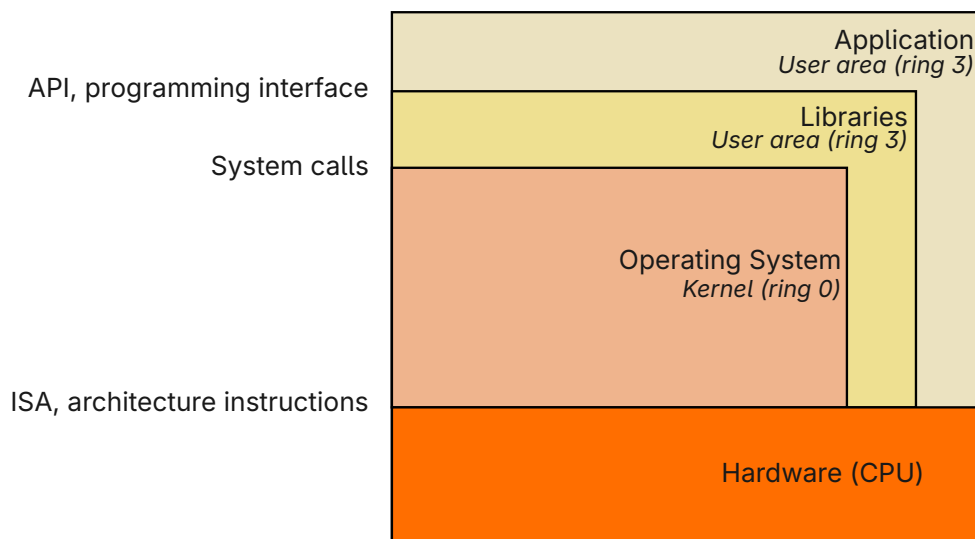
Fuente: Wikipedia contributors, *Protection ring* (2026) [13].

Figure 2.2: Privilege rings in modern operating systems.

This architecture mainly defines four concentric privilege levels, commonly called rings, numbered from 0 to 3. The privilege level determines which CPU instructions a process can execute and which memory regions it can access during execution, distinguishing the following four rings [11]:

- **Ring 0:** The highest privilege level, where the core of the system, the *kernel*, typically resides. Programs executed in this ring have almost direct and unrestricted access to all *hardware*, memory and processor instructions.
- **Ring 1 and 2:** Originally designed for system service and control with intermediate privileges, although in practice they have been largely discarded by most operating systems due to their complexity and low performance.
- **Ring 3:** The lowest privilege level. Processes launched in this ring cannot interact directly with the *hardware* and must delegate any critical operation to higher privilege levels.

For reasons of efficiency and portability to other processor architectures, such as *ARM*, which usually does not provide as many levels, *Linux* simplifies this model and uses only two of these rings. On the one hand, the *kernel* is isolated and executed entirely in *Ring 0*; on the other hand, the rest of the user programs are confined to the *Ring 3* region [14, 11].



Fuente: Guardiola Múzquiz, Gorka and Soriano Salvador, Enrique, *Fundamentos de Sistemas Operativos: Una aproximación práctica usando Linux (2025)* [11].

Figure 2.3: Structure of a modern operating system.

Basically, the structure of the operating system can be described as a physical division of privileges that directly translates into a logical architecture structured in layers built on top of each other [11].

At the base is the physical *hardware* of the machine, with which the *kernel* interacts directly through the available architecture instructions (ISA). Immediately above it, operating with absolute privileges over the system in *Ring 0*, resides the *kernel*. This is the heart of the operating system, responsible for managing the machine's resources and acting as the fundamental bridge between the *hardware* and user applications. Subsequently, in the outer layers, there are the system libraries, responsible for providing the ***Application Programming Interface (API)***, and the final applications, which are confined to the user area (*Ring 3*).

Due to this separation, if a process in this user region needs to perform a privileged action, such as reading a file from disk, sending a network packet or allocating memory, it cannot directly access the *hardware* by itself. Instead, it must request that privileged operation from the *kernel* by crossing the *Ring 0* boundary through a standardized and controlled communication mechanism, commonly known as ***System Calls*** or *syscalls*.

This architectural barrier not only defines which instructions can be executed and where, but also forces the operating system to divide fast-access main memory (*RAM*) into two large fully isolated virtual regions: ***User Space*** and ***Kernel Space***, thereby ensuring that no user process can corrupt sensitive internal system data [14, 11].

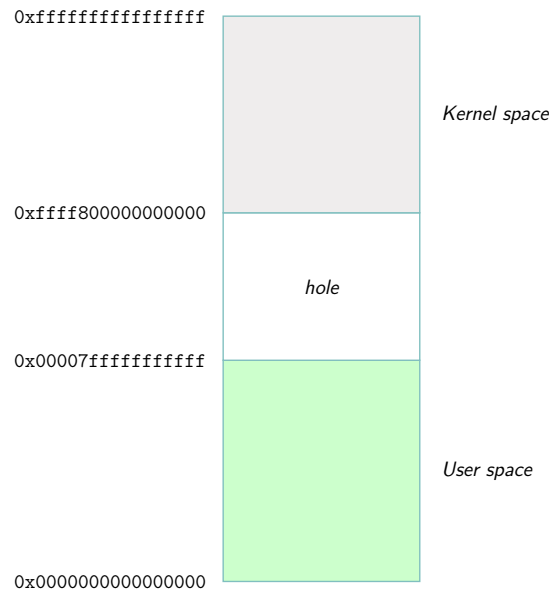
2.1.2 Memory: *User Space* and *Kernel Space*

To understand how the privilege division described in the previous section is actually materialized in the physical memory of the machine (*RAM*), it is necessary to introduce the concept of *virtual memory*.

In modern *Linux* systems, processes do not directly access physical *RAM* addresses. Instead, the *kernel* and the *CPU* memory management unit (*MMU*) provide each process with the illusion of having an exclusive and contiguous space through ***virtual memory***. To achieve this, the *MMU* dynamically translates virtual memory addresses into real physical locations and vice versa by consulting a ***page table***. In this way, multiple processes can run simultaneously in an isolated and secure manner, each with its own address map fully managed by the *kernel* [15].

At the same time, this *virtual memory* space is strictly divided into two large

regions, strongly related to the processor's protection-ring architecture:



Fuente: Guardiola Múzquiz, Gorka and Soriano Salvador, Enrique, *Fundamentos de Sistemas Operativos: Una aproximación práctica usando Linux (2025)* [11].

Figure 2.4: Logical memory division between *User Space* and *Kernel Space*.

- **User Space:** The lower portion of virtual memory, associated with *Ring 3*. User applications, their libraries and their corresponding data are loaded and executed in this region. If a process commits a critical error in this region, such as trying to access memory that does not belong to it, the operating system intervenes to terminate that process and raises an error, typically known as a *Segmentation Fault*, preventing the failure from affecting the rest of the system.
- **Kernel Space:** The upper portion of virtual memory, reserved exclusively for the *kernel*, its modules and device controllers (*drivers*). Since it is associated with *Ring 0*, the code residing here has full access to physical memory and, consequently, to the *hardware*. A programming error or malicious intrusion at this level compromises the entire system and may cause a complete system crash (*Kernel Panic*).

In the same way that *virtual memory* abstracts the complexity of physical memory organization (*RAM*) to guarantee process isolation and secure execution, the operating system also provides an equivalent abstraction for persistent storage management and interaction with the rest of the system components. This important task is handled by another major structural pillar of a *Linux* operating system: its *file system*.

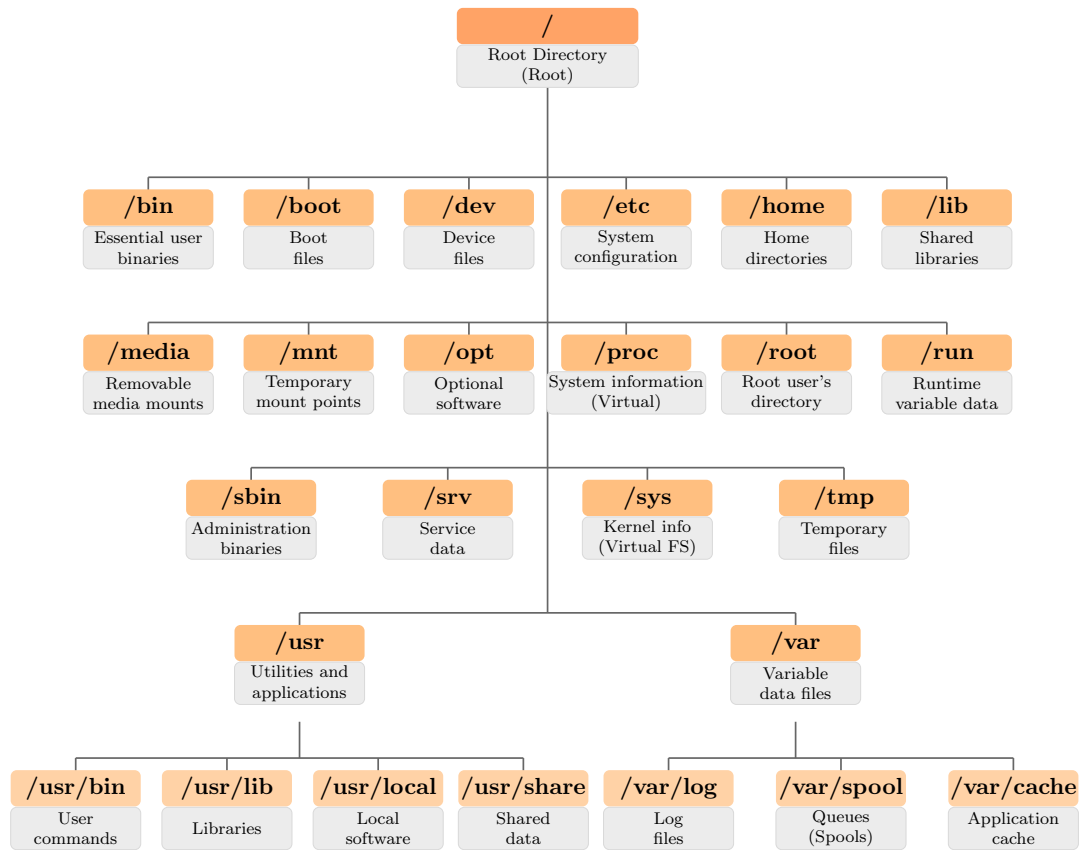
2.1.3 Files and File Systems (*FS*)

One of the most remarkable characteristics of the *Linux* architecture, inherited directly from the original *Unix* family, is the fundamental principle that “everything is a file”². In this type of operating system, not only text documents or binary executable programs are treated as files, but also directories, network connections (*sockets*), *hardware* devices such as terminals or hard disks, as well as dynamic information about running processes and the system itself, such as virtual files located in the `/proc` directory [14].

To structure, access and organize this enormous amount of information, the operating system uses a **File System (*FS*)**, which can be defined as the set of rules, methods and logical hierarchies responsible for managing how files are named, stored and retrieved from a logical medium.

Due to the large amount of both physical and logical data and resources handled by the machine, a universal organization must be maintained so that the operating system knows exactly where everything is located. As a solution to this organizational need, *Linux* adopts the ***Filesystem Hierarchy Standard (FHS)***, which defines a global hierarchy of root directories where the different types of system files are distributed, such as binaries, libraries, configuration files, persistent data and dynamic resources, according to their function, type and scope.

²A file is not limited to a document stored on disk, but represents a uniform interface for accessing system resources. In *Plan 9*, the conceptual successor to *Unix*, this philosophy was taken even further, making practically all resources visible as files and directories. The `/proc` system follows this same philosophy, mainly from *Plan 9*.



Fuente: Own elaboration based on Albritton, William McDaniel, "Module 2, Session 7 - Linux File System Structure" (2016) [16].

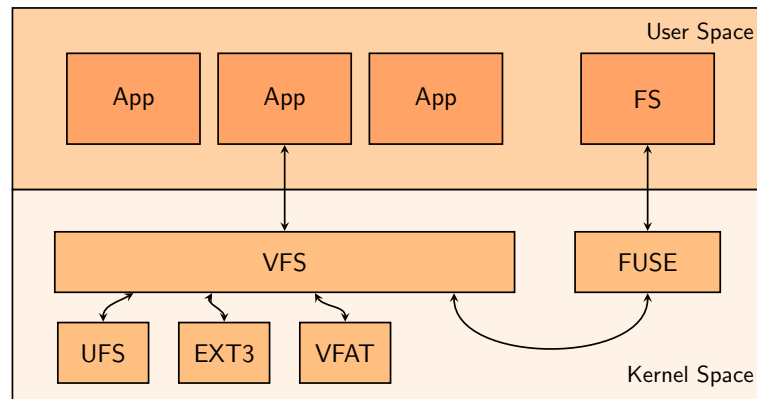
Figure 2.5: Standard hierarchical directory structure in *Linux* (FHS).

Within this structure, one of the most distinctive directories is `/proc`, since it does not store persistent data on disk, but constitutes a pseudo-file system dynamically generated by the *kernel* in memory (*RAM*) through **virtual files**. Its main purpose is to expose, through a file-based interface, information about system parameters, running processes and certain internal *kernel* states, such as `/proc/meminfo` or `/proc/cpuinfo`. Access to these entries is performed through standard file operations, such as `read()` or `write()`, which the *kernel* redirects to the corresponding handlers according to the nature and purpose of each virtual file.

Thus, it is possible to create communication interfaces between *User Space* and *Kernel Space* through *kernel modules* that create new virtual file entries inside this directory, allowing read and write operations to be associated with specific handlers in order to request or report sensitive system information in a controlled manner.

To manage this wide heterogeneity of resources and allow multiple physical file systems, such as *ext4* or *NTFS*, to coexist on the same host system, the *kernel*

implements an abstraction layer called the **Virtual File System (VFS)**, located between the system-call layer (*syscalls*) and the low-level *drivers*. Its function is to provide a unified *API* that allows applications in *User Space* to use standard calls, such as `read()` or `write()`, on any file or device, completely abstracting the machine details [11].



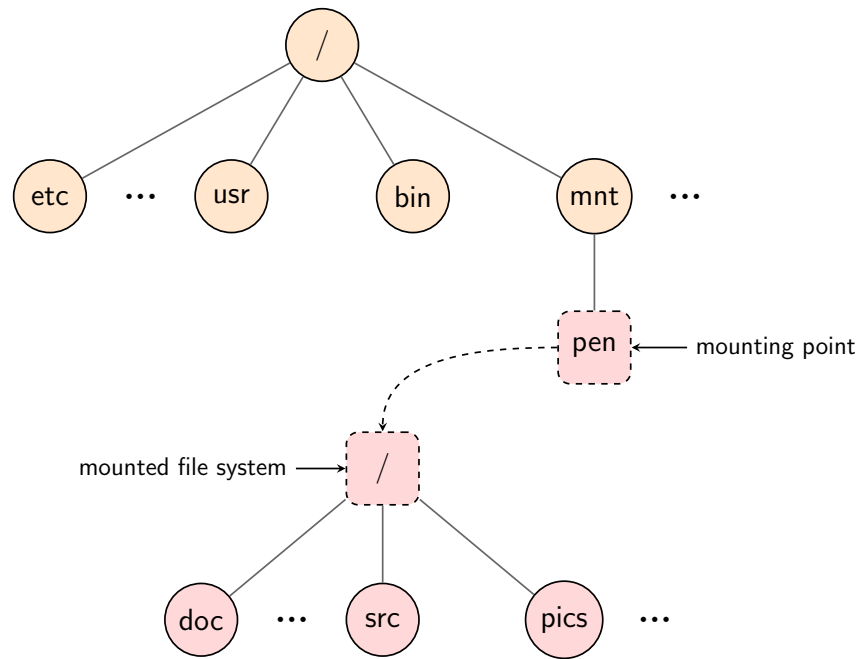
Fuente: Guardiola Múzquiz, Gorka and Soriano Salvador, Enrique, *Fundamentos de Sistemas Operativos: Una aproximación práctica usando Linux* (2025) [11].

Figure 2.6: File-system management in *Linux*.

Mainly, the *VFS* acts as a central router between *User Space* and *Kernel Space*, receiving application requests and transparently redirecting them to the corresponding *driver*. In addition, modern systems support advanced mechanisms such as *FUSE* (*Filesystem in Userspace*), which allows the *VFS* to delegate the management of certain file systems to processes running safely in *User Space*.

To integrate different file systems within the host system, *Linux* dispenses with isolated drive units and uses **mounting**. This unifying mechanism logically attaches any external file system to an existing directory, for example `/mnt` or `/media`, considered a **mount point**. In this way, a continuous and transparent directory structure is created, abstracting the user from the nature of the underlying external medium [14].

As explained in the previous sections, the *Linux* architecture is based on strongly centralized resources: a single *kernel*, shared physical memory, a global directory hierarchy, among others. To break with this centralization and provide the system with isolation capabilities, the *kernel* implements the technology of *Namespaces*, which enables the confinement of processes and resources, both physical and logical, within an isolated and controlled environment. This concept is explained in detail in the



Fuente: Guardiola Múzquiz, Gorka and Soriano Salvador, Enrique, *Fundamentos de Sistemas Operativos: Una aproximación práctica usando Linux (2025)* [11].

Figure 2.7: *Mount point*: multiple file systems in a single logical tree.

following section [11, 14].

2.1.4 Namespaces

In order to explain in depth how *Namespaces* work, it is first necessary to define how the operating system manages and classifies the programs running on the machine.

Any program that is running and loaded into virtual memory is called a **process**. To efficiently manage the large number of processes that may coexist simultaneously in the system, the *kernel* assigns each of them a set of attributes and critical identifiers that are highly relevant for recording the system state:

- **PID (Process ID)**: A unique integer dynamically assigned to each active process, identifying it unambiguously within the system.
- **UID (User ID)**: The numeric identifier that associates the process with a specific user of the machine. This attribute determines the main permissions and privileges that the process has to interact with the file system, send signals or execute certain system calls.

- **GID (Group ID):** The numeric identifier of the group to which the process belongs. It is used as a complement to the *UID* to determine and refine access permissions to shared system resources.
- **Command:** The name or direct reference to the binary executable file that originated the process, for example `/bin/ls` or `/usr/bin/python3`.

```
$localhost:$ ps aux
```

USER	PID	%CPU	%MEM	VSZ	RSS	TTY	STAT	START	TIME	COMMAND
root	1	0.7	0.3	22208	12444	?	Ss	12:37	0:04	/sbin/init
root	2	0.0	0.0	0	0	?	S	12:37	0:00	[kthreadd]
root	3	0.0	0.0	0	0	?	S	12:37	0:00	[pool_workque
root	4	0.0	0.0	0	0	?	I<	12:37	0:00	[kworker/R-kv
root	5	0.0	0.0	0	0	?	I<	12:37	0:00	[kworker/R-rc
root	7	0.0	0.0	0	0	?	I<	12:37	0:00	[kworker/R-sl
root	8	0.0	0.0	0	0	?	I<	12:37	0:00	[kworker/R-ne
root	11	0.1	0.0	0	0	?	I	12:37	0:00	[kworker/0:1-
...										
noel	3039	53.0	14.7	1217632252	573236	?	Sl	12:39	3:52	/usr/share/co
noel	3145	0.7	2.0	1215834136	78112	?	Sl	12:40	0:02	/usr/share/co
noel	3206	7.9	0.8	82304	31816	?	Sl	12:41	0:23	/home/noel/.v
root	3327	0.0	0.0	0	0	?	I<	12:42	0:00	[kworker/0:0H
root	3330	0.2	0.0	9760	3676	pts/0	S	12:42	0:00	su
root	3332	0.8	0.1	14356	7184	pts/0	S	12:42	0:01	sh
root	3619	0.0	0.0	0	0	?	I	12:44	0:00	[kworker/u20:
noel	3668	3.1	0.5	4260844	21540	?	Sl	12:45	0:02	/home/noel/.v
root	3683	0.7	0.0	0	0	?	I	12:45	0:00	[kworker/u16:
root	3726	700	0.1	10852	4264	pts/0	R+	12:46	0:00	ps aux

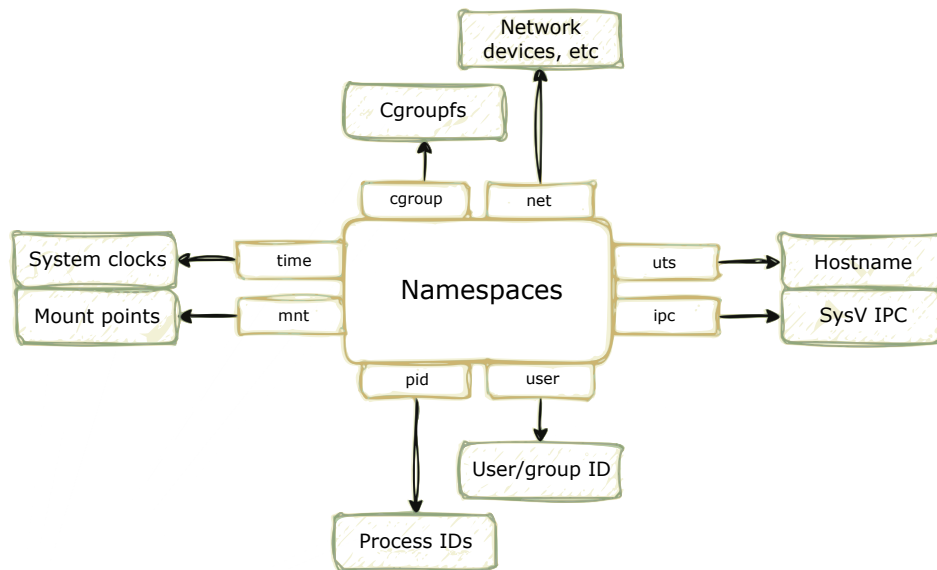
Code 2.1: Example output of the `ps aux` command in a *Linux* terminal showing the list of currently running system processes.

If the concept of *Namespace* is not considered for the moment, all these resources and attributes can be assumed to be **global**. This means that if, for example, a process has been assigned *PID* 3039, that identifier is unique, absolute and visible to any other monitoring tool on the machine, such as the virtual files in `/proc`. Similarly, the *root* user, whose *UID* is 0, has unrestricted absolute privileges over all system processes, unlike a standard user with a different *UID*, such as 1000, who is mostly restricted and can only manage the processes that they have started.

This is where *Namespaces* come into play, completely altering this paradigm. A *Namespace* works as a logical abstraction layer that wraps a global system resource and exposes it to a group of processes as if it were an isolated and privileged resource,

with the *kernel* being responsible for translating these identities, attributes, permissions and resources in a fully transparent way.

In practice, this means that a process running inside an isolated *Namespace* may be assigned *PID* 1, typically associated with the first process executed on the machine, *systemd* or *init*. However, from the external perspective of the *kernel*, that same process is simply another regular process with, for example, *PID* 4500.



Fuente: Gmkziz, *Understanding Linux Namespaces* (2024) [17].

Figure 2.8: Types of *Namespaces*.

To achieve this level of flexibility, the *kernel* does not apply monolithic isolation to the entire operating system at once. Instead, it provides a modular architecture based on different **types of *Namespaces***. Each type is responsible for isolating a specific global resource, allowing administrators, or attackers, to create combinations of them independently depending on their purpose [18].

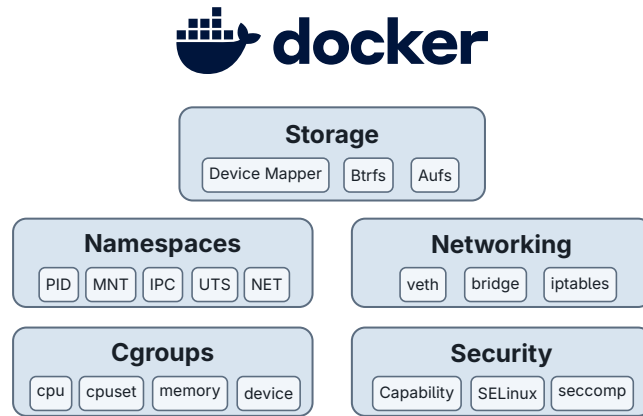
The main types of *Namespaces* implemented in a modern *Linux* system are:

- **Mount (mnt):** Isolates the file-system mount points, a concept directly related to Figure 2.7. Processes inside this *Namespace* have their own view of the directory hierarchy (*FHS*). This means that mounting or unmounting a removable drive, such as a *USB* device, within this isolated space will not affect or be visible to the rest of the host system.

- ***PID*** (*pid*): Provides an independent process hierarchy. As mentioned earlier, it allows the first process executed in that space to receive *PID* 1, completely isolating visibility outside the *Namespace*. This prevents processes inside this space from interacting with processes outside it.
- ***User*** (*user*): Isolates user identifiers (*UID*) and group identifiers (*GID*). In this space, a process may have absolute *root* privileges (*UID* 0) within it, while externally it continues to be treated and restricted as a standard unprivileged user.
- ***Network*** (*net*): Virtualizes and isolates the entire network stack of the system. Each *Namespace* of this type has its own network interfaces, including its own *loopback*, ports, *IP* addresses, *firewall* rules and routing tables, such as *iptables* or *nftables*.
- ***UTS*** (*uts*): Allows a group of processes to have its own machine name (*hostname*) and *NIS* domain name, independently from the real name of the physical server.
- ***IPC*** (*ipc*): Isolates inter-process communication resources, such as message queues (*POSIX*), shared memory segments and semaphores. It prevents a process in this space from interfering with or intercepting messages from external processes.
- ***Cgroup*** (*cgroup*): Encapsulates the view of the root directory of control groups. This hides the hierarchy of policies and resource limits, such as *CPU* or memory, established by the *kernel*, preventing an isolated process from knowing or attempting to modify them.
- ***Time*** (*time*): Isolates the internal system clocks, especially the boot clock and the monotonic clock. This allows the isolated space to have its own timeline, which is particularly useful for migrating containers between different physical servers without altering their perception of uptime.

The strategic combination of these blocks is precisely the technological engine that makes modern virtualization technologies such as containers (*Docker*) possible, whose typical architecture is illustrated in Figure 2.9.

All these isolation and abstraction technologies, from the memory management of the machine itself to *Namespace*s, reside and are managed in the core of the operating



Fuente: Own elaboration based on Docker, Inc., *What is Docker?* (2026) [19], using Docker's official logo [20].

Figure 2.9: Example of a *Docker* architecture.

system. However, despite its robustness, the *kernel* is not a monolithic and unalterable entity. To provide high flexibility and allow its functionalities to be extended at runtime, or maliciously altered, it has a dynamic architecture that allows code with maximum privileges to be injected and loaded, as explained in the following section.

2.1.5 *Linux* Kernel Modules (*LKMs*)

A **Loadable Kernel Module** (*LKM*) is compiled object code, whose source code is written in *C*, that can be loaded into and unloaded from the memory space of the *kernel* at runtime. These modules allow the *kernel* to extend its functionality, such as incorporating new file systems, without needing to reboot the system or recompile the entire *kernel* [21].

Developing *LKMs* presents drastic differences compared with conventional programming in *User Space*. Since they operate directly at the highest privilege level, modules cannot access the standard *C* library (`libc`) and, consequently, typical functions such as `printf()` or `malloc()`. In addition, since they are part of the *kernel* itself, they cannot use system calls through the standard interface either, but instead invoke internal *kernel* functions directly.

On the other hand, there is no standard console output as in *User Space*; to replace it, the *kernel* provides its own log-printing macro: `printk()`. The traces generated through this function are stored in an internal circular *buffer* and are registered in

files, typically in `/var/log/kern.log`. Moreover, `printk()` requires each trace to be classified with a priority level (*log level*), ranging from critical or error events (`KERN_EMERG` and `KERN_ERR`) to informational or debugging events (`KERN_INFO` and `KERN_DEBUG`).

Regarding compilation, the *kernel* headers (`linux-headers` packages) matching the exact system version are required. Through a `Makefile`, the code is compiled into a special packaged object file with the `.ko` extension (*kernel object*). Once compiled, its life cycle management can be summarized in four basic phases:

- **insmod**: Insertion of the code into *Kernel Space*.
- **modinfo**: Reporting of the module metadata and dependencies.
- **dmesg**: Inspection of the *kernel* execution traces.
- **rmmod**: Removal of the module and release of memory resources.

At source-code level, the logical structure of any *LKM* is based on two main functions. First, an **initialization or load function** (`module_init()`), which is executed automatically when the module is inserted into the *kernel* through `insmod` and is responsible for reserving the required resources or creating the involved virtual files. Second, a **cleanup or unload function** (`module_exit()`), invoked when the loaded module is removed through `rmmod`, in order to release the used memory and avoid leaks or inconsistent states in the system.

Finally, in order to establish a bidirectional communication channel or interface between *User Space* and *Kernel Space*, an efficient technique is to create virtual files through the `/proc` pseudo-file system. In this way, the module instantiates a virtual file, for example `/proc/mydev`, and defines custom handlers that intercept the system calls acting on it, such as `read` or `write`, thus declaring specifically how to interact with *User Space*. To implement this methodology in the *LKM*, specific functions from the *kernel API* are required to ensure safe operation, such as `copy_to_user()` and `copy_from_user()` for transferring information between different memory spaces [22].

In conclusion, the set of structural and functional characteristics presented above constitutes the technological basis of a *Linux* operating system. As previously stated, this has been the working environment chosen to develop this project, deployed on

```
# 0. Root privileges
$ su

# 1. Compile the module using Makefile
$ make
make -C /lib/modules/6.12.41-current-bcm2711/build M=/home/lkms modules
make[1]: Entering directory '/usr/src/linux-headers-6.12.41-current-bcm2711'
  CC [M] /home/lkms/hw.o
  MODPOST /home/lkms/Module.symvers
  CC [M] /home/lkms/hw.mod.o
  LD [M] /home/lkms/hw.ko
make[1]: Leaving directory '/usr/src/linux-headers-6.12.41-current-bcm2711'

# 2. Load the module (.ko) into the kernel
$ insmod hw.ko

# 3. Inspect the module metadata
$ modinfo hw.ko
filename:      /home/lkms/hw.ko
version:       0.1
description:    A simple Hello world LKM
author:        Noel Rodriguez Perez
license:        GPL
srcversion:     2F2B1B95DA1F08AC18B09BC
depends:
vermagic:      6.12.41-current-bcm2711 SMP mod_unload modversions

# 4. Check the generated logs by printk
$ dmesg

# 5. Unload the module freeing its resources
$ rmmod hw
```

Code 2.2: Basic commands for managing an *LKM*.

an independent and controlled physical machine. This guarantees direct access to the deepest layers of the operating system, such as the *kernel* and its modules, while mitigating any risk to production or real-use machines.

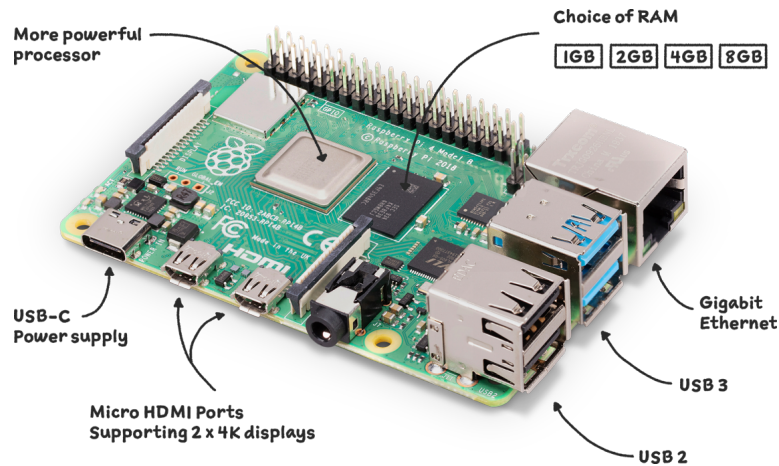
2.1.6 Development Environment

Taking into account all the functionalities described in *Linux* environments, the development of this project was carried out on a dedicated physical machine: a **Raspberry Pi 4 Model B** with **4 GB of RAM**, belonging to the *ARM* architecture

family.

The choice of this platform and a *Linux*-based system is mainly due to its versatility and the high degree of control it offers over the operating system, the *kernel* and the low-level tools required to compile, load and debug *kernel modules*. Furthermore, since it is an isolated physical machine, the risks associated with testing code in *Kernel Space* are reduced, avoiding corruption of an everyday-use computer and enabling safer experimentation on a real environment [23].

Nevertheless, the choice of this board does not conceptually constrain the designed mechanism. The *Raspberry Pi* is used as a controlled, portable and suitable experimental platform for testing. However, the ideas developed rely on interfaces and mechanisms native to *Linux*, so they could be transferred to other compatible systems with few adaptations. It is a fairly flexible and portable operating system, so almost no changes to the code would be required.



Fuente: Official image of Raspberry Pi 4 Model B [24].

Figure 2.10: Physical system used in the project.

The **Armbian** distribution was installed on this board, specifically an image for Raspberry Pi 4 based on *Ubuntu Noble*, aimed at *single-board computer* boards and optimized for *ARM* architectures. It was chosen because it provides a configurable, lightweight environment that is particularly suitable for this type of device, while also preserving the convenience offered by *Ubuntu/Debian* environments and the `apt` package manager [25, 26].

On the other hand, preparing the environment began with the external installation



Fuente: Official Armbian image [27].

Figure 2.11: Armbian distribution logo for Raspberry Pi 4 Model B.

of that *Armbian* image on a 32 GB *microSD* card and the initial boot of the board after inserting it. Once the system had started, it was updated and the necessary tools were installed for compilation in *C* (`gcc` and `make`), *kernel module* development (`kmod`) and the *kernel* headers required for compilation (`linux-headers`), the use of *OpenSSL* (`openssl` and `libssl-dev`) and the deployment of containers to create new namespaces with *Docker* (`docker.io`).

```
$ uname -r
6.12.41-current-bcm2711

$ sudo apt update
$ sudo apt install gcc make linux-headers-$(uname -r) \
> kmod openssl libssl-dev docker.io
```

Code 2.3: Installation of the development environment on *Armbian* based on *Ubuntu Noble*.

Once the technological basis of *Linux* systems has been established and the development environment defined, the security problem exploited by threats such as *rootkits* will be formally introduced next, together with the requirements and implementations for the proposed solution.

Chapter 3

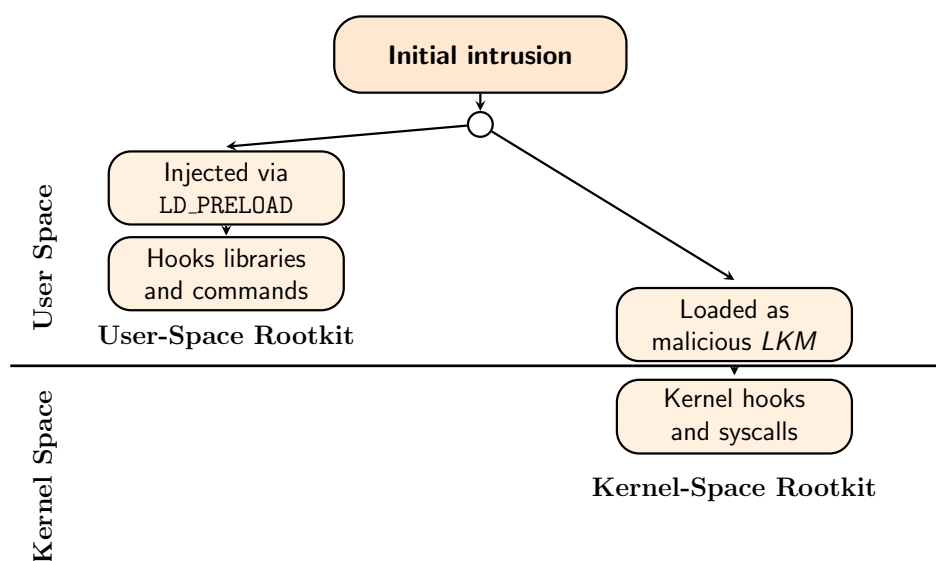
State of the Art

“A collection of files that is installed on a host to alter the standard functionality of the host in a malicious and stealthy way.”

NIST CSRC, *Rootkit - Glossary* [28].

Based on the definition above and on what was introduced earlier in this report, a *rootkit* is a type of *malware* designed to hide and manipulate system resources, such as processes, files, network connections or persistence mechanisms. Its danger does not lie only in the privilege escalation it may achieve, but also in its ability to falsify the view of the real state of the machine, compromising the reliability of its monitoring.

In *Linux* systems, *rootkits* can mainly be classified into two broad categories according to the layer in which they operate:



Fuente: Own elaboration.

Figure 3.1: General classification of *rootkits* in *Linux* systems.

3.1 User-Space Rootkits

User-Space rootkits operate on programs, dynamic libraries and user tools of the operating system. Their most common technique consists of exploiting the LD_PRELOAD dynamic-linking mechanism to load malicious libraries and intercept commonly used functions (*hooking*), such as those provided by `libc`, thereby altering the behavior of commands such as `ps`, `ls`, `top` or `netstat` [9]. On the other hand, these threats are not limited exclusively to this mechanism: they may also replace legitimate commands with manipulated versions or rely on isolation and virtualization techniques to present an altered view of the system.

If *User Space* is already compromised, attempting to detect a *rootkit* in this region using tools that also run in *User Space* presents significant limitations and may even become completely ineffective. Depending on the design of the *rootkit*, it may evade detection mechanisms specialized in hidden processes and even manipulate the *shell* that launches them or the information exposed through the `/proc` file system by means of *namespaces* and *bind mounts*.

Among the most prominent examples of this category are several families of *rootkits* based on LD_PRELOAD, such as **Jynx** and **Jynx2**, which popularized this concealment approach in *User Space* (*userland*). Later, **Azazel** extended these techniques with anti-debugging mechanisms, process hiding, remote connections and evasion against detection mechanisms such as `unhide` or monitoring commands such as `ps` and `netstat`. In parallel, **BEURK** is presented as a *preload rootkit* focused on anti-detection and hiding of processes, files and accesses; while **vlany** also adds persistence, dynamic linker modification and *backdoor* capabilities.

In this context, it is worth highlighting common detection tools, mainly **unhide**, a forensic tool used to detect hidden processes by searching for discrepancies between different views of the system, for example by comparing the information provided in `/proc` with that reported by tools such as `ps`. However, as previously mentioned, once this region is compromised, its usefulness becomes severely limited, since these tools operate at the same privilege level (*User Space*) and under equal conditions, giving attackers an advantage [29].

3.2 Kernel-Space Rootkits

Kernel-Space rootkits infiltrate the system's highest privilege level and are usually materialized through malicious *kernel modules* (*LKMs*) or through direct modifications of the *kernel* itself. Their privileged position allows them to alter internal structures, hide processes, files or network traffic, intercept sensitive system calls and even falsify the information supplied to user tools.

For this reason, detecting them from higher layers is especially complex, and any defensive mechanism operating with lower privileges starts from a clearly disadvantageous position. This motivates the need to design authenticated monitoring architectures that operate in regions close to the *kernel* or, at least, that do not depend exclusively on information already provided by the user environment itself.

Tools such as `chkrootkit` still include detection for families of this type of threat, such as **Adore LKM**, **knark**, **SucKIT**, **Fu** or **Syslogk**, reflecting the *LKM* model as a concealment and persistence vector. The *MITRE ATT&CK* framework documents cases such as **REPTILE**, implemented as an open-source *Linux rootkit* based on *LKMs*, and **Drovorub**, a *Linux malware* suite that includes *LKMs* for persistence and remote control [30].

In conclusion, *rootkits* are not a homogeneous threat that always acts in the same way, but rather a family of concealment techniques distributed across several layers of the system.

In light of the above, this project focuses mainly on the problem of *User-Space rootkits*, due to their ability to falsify the information supplied by tools that run in that same region. The proposed solution to this limitation focuses on moving the collection of critical system information to a more trusted layer, implementing a *kernel module* that generates and authenticates such information, so that its integrity can later be verified from *User Space*.

Chapter 4

Problem Statement and Objectives

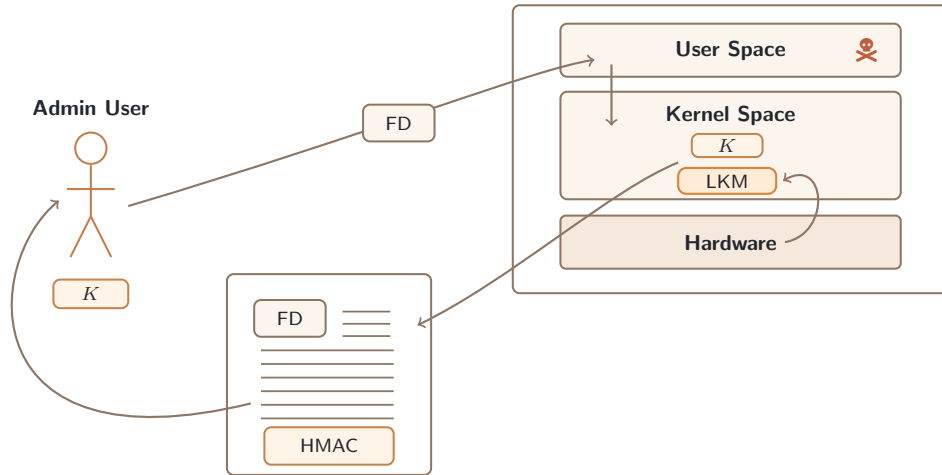
After establishing the necessary context on *Linux* systems and the threat posed by *rootkits*, this chapter describes the specific problem that motivates this project and the objectives pursued.

4.1 Problem Description

One of the fundamental problems in operating-system security is determining to what extent the critical information offered to the administrator about the state of a machine can be considered reliable once the environment has been compromised, particularly *User Space*. Based on the state of the art described above, *rootkits* operating in this region can intercept functions from fundamental libraries and manipulate the output of tools such as `ps`, `ls` or `netstat`, falsifying the operator's view of the system and making it more difficult to detect malicious activity [9].

This scenario is especially problematic in *Linux* systems, because many monitoring and administration tasks rely on *User-Space* utilities, where information is exposed through high-level interfaces. If that region is compromised, relying on it to verify the state of the system is no longer a solid strategy. Consequently, there is a need to obtain sensitive information from a more privileged layer, making its manipulation more difficult, and to provide that information with a mechanism that allows its integrity and authenticity to be checked afterwards.

This project addresses a specific and representative case, taking as critical information a list of all system processes together with their most relevant metadata, including global and local process identifiers, the users that execute them and references to their *namespaces*.



Fuente: Own elaboration.

Figure 4.1: Authenticated reporting mechanism between *User Space* and *Kernel Space*.

This choice is not arbitrary, since this information provides a global view of the state of the system and can therefore be compared with data obtained from other tools in order to detect possible signs of manipulation and concealment in that region. However, if an attacker manages to manipulate the information shown to the administrator, malicious processes could be hidden or the perceived state of the system could be altered, making detection more difficult.

In conclusion, the problem addressed by this project can be formulated as follows: **how to provide the administrator of a *Linux* system with a reliable view of critical system information when *User Space* may have been compromised by *malware* with persistence and concealment capabilities, such as *rootkits*.**

4.2 Threat Model

This project specifically addresses the threat of ***User-Space rootkits*** in *Linux* systems. These are *malware* tools designed to maintain control over this region of the system, allowing them to alter the output of critical monitoring tools through code interception and modification techniques [9].

This manipulation capability allows an attacker to hide:

- **Malicious processes:** Hiding the execution of remote-control tools or *backdoors*.
- **Files and network connections:** Masking suspicious connections to remote servers or persistence files.
- **Resource accesses:** Falsifying file-access records or permission information.
- **User activity:** Falsifying information about active sessions or executed commands.

In this context, the threat model focuses on a potentially compromised *User Space*, capable of hiding or falsifying information visible to the administrator. The security assumptions considered are detailed below.

4.2.1 Operational Context and Security Assumptions

The project assumes a scenario in which *User Space* has been compromised, allowing the attacker to execute *malware* with persistence and concealment capabilities. In this context, the following assumptions are made:

- ***User Space* is under the attacker's control.** The administrator cannot trust the output of monitoring tools such as `ps`, `ls` or `netstat`, since they may have been intercepted and manipulated by a *rootkit*.
- ***Kernel Space* remains intact.** The project assumes that there is no compromise by *Kernel-Space rootkits*, preserving the integrity of the code executed in *Ring 0*.
- **The administrator is trusted.** It is assumed that the shared symmetric key used for HMAC authentication is known only by the trusted administrator of the system.
- **Communication is unidirectional from the *kernel* to *User Space*.** The *kernel* does not trust data received from *User Space* except for the identifier of the output file, which is the main input vector.

- **Hardware and Firmware are intact.** It is assumed that neither the system *hardware* nor its *firmware* have been compromised.
- **Boot chain of trust.** A chain of trust is conceived to preserve the integrity of the *kernel* and the *LKM*, for example through signing mechanisms, preventing the execution of unauthorized components within the trusted computing base.

4.2.2 Attacker Capabilities and Limitations

Attacker capabilities within the modeled scenario:

- Intercepting and modifying the output of user tools through `LD_PRELOAD` and *hooking* of library functions such as `libc`.
- Hiding processes, files, network connections and malicious activity from *User-Space* monitoring tools.
- Executing malicious code with arbitrary privileges in *User Space*.
- Manipulating information in `/proc` through *bind mounts* and *namespaces*.

Attacker limitations within the modeled scenario:

- The attacker cannot modify the *kernel* or load malicious *LKMs* (no access to *Ring 0*).
- The attacker cannot intercept system calls or modify the *System Call Table* (*sys-call_table*).
- The attacker cannot forge HMAC-SHA256 without knowing the shared symmetric key between the *kernel* and the system administrator.
- The symmetric key is loaded into the *kernel* during the boot process without the attacker having access to it.

4.2.3 Defensive Coverage

What the system protects:

- **Process hiding:** It provides visibility from the *kernel*, which cannot be manipulated from *User Space*.
- **Information integrity:** Through HMAC-SHA256 cryptographic authentication, mathematically verifiable using the same key with which it is encoded.
- **Manipulation detection:** It allows authenticated information from the *kernel* to be compared with compromised *User-Space* tools.

What the system does NOT protect:

- Threats from *Kernel-Space rootkits* operating in *Ring 0*.
- Post-load compromise of the *LKM* through modification of *kernel* memory.
- Exposure or interception of the shared symmetric key between the *kernel* and the administrator.
- Processes or malicious activity prior to loading the *LKM*.

4.3 Objectives

The main objective of this project is to **design a secure reporting mechanism for *Linux* systems capable of providing the administrator with a more reliable view of critical system information, specifically the state of active processes, especially in environments compromised by *User-Space rootkits*.** To this end, the proposed solution moves the collection of critical information to *Kernel Space* through an *LKM*, also incorporating a cryptographic mechanism that allows its integrity and authenticity to be verified afterwards from *User Space* by the administrator.

Based on this general objective, the project is specified through the following objectives:

4.3.1 Functional Objectives

The functional objectives, which describe **what** the proposed solution must do, are:

- **Obtain critical system information from *Kernel Space*.** A secure reporting mechanism must be designed from *Kernel Space*, using an *LKM*, capable of collecting information about active processes without relying on *User-Space* tools.
- **Build a global and structured view of system processes.** This reporting mechanism must generate a structured list of active system processes including their most relevant metadata: global *UID*, local *UID*, user namespace identifier, global *PID*, local *PID*, process namespace identifier, *GID* and associated command.
- **Transfer the information obtained from *Kernel Space* to a channel accessible from *User Space*, when requested by the administrator.** A controlled communication mechanism between both regions must be enabled so that the *kernel* can detect the information request and transfer it. In this case, the information transfer is articulated by sending the reference to a file set by the administrator, through its file descriptor, from *User Space* to *Kernel Space*.
- **Cryptographically authenticate the content generated in *Kernel Space*.** The produced report must be accompanied by a mechanism that proves its authenticity and integrity, implemented in this project through an *HMAC* based on SHA-256, calculated with a symmetric key *K* from the process-list information.
- **Allow subsequent verification of the information requested from the *kernel* from *User Space*.** The administrator must be able to verify the authenticity and integrity of the report generated by the *LKM*, being able to perform this check afterwards on another non-compromised machine after copying the obtained data. To do so, a *trust framework* is created in which both the administrator and the *kernel* must share a symmetric key *K*; in this same context, a *User-Space* mechanism is implemented to recalculate the corresponding *HMAC* and check whether the result matches the authenticated value emitted by the *kernel*.
- **Facilitate the detection of concealment and manipulation threats.** The developed reporting system must allow authenticated information obtained from

Kernel Space to be compared with that offered by *User-Space* monitoring tools, enabling the identification of signs of concealment or manipulation.

- **Validate the operation of the prototype.** The designed mechanism must be tested in a real *Linux* environment, including processes and users belonging to different *namespaces*, in order to demonstrate that the administrator correctly obtains the critical information and that its authentication can be successfully verified.

4.3.2 Non-Functional Objectives

The non-functional objectives, which describe **how** the designed system must operate, are:

- **Verifiable integrity and authenticity of critical information.** The reporting system must not merely expose data from the *kernel*; it must guarantee that such data can be checked afterwards through a cryptographic authentication mechanism, in this case an *HMAC* based on *SHA-256*.
- **Robustness in communication between spaces.** The interface between *Kernel Space* and *User Space* must validate all information entering the *kernel*, including permissions, file descriptors, opening modes and memory limits, reducing the risk of incorrect use or unsafe behavior that could compromise system stability.
- **Security.** The implemented system must minimize trust in *User Space*, so that critical information is generated primarily in a more privileged region, such as *Kernel Space*, and is accompanied by cryptographic and authenticatable mechanisms that allow unauthorized modifications to be detected.
- **Modularity and low impact on the system *kernel*.** The secure reporting mechanism must be integrated as a *kernel module* (*LKM*), avoiding permanent modification of the core and facilitating both controlled deployment and unloading from the system, while also fulfilling the previous objective of generating critical information from a highly privileged region to favor its integrity.

- **Portability across *Linux* environments.** The design must rely on standard interfaces and mechanisms of the *Linux* ecosystem, such as `/proc`, the *kernel Crypto API*, conventional C compilation utilities and the headers required to compile *LKMs*.
- **Prototype reproducibility.** The prototype must be compilable, loadable, unloadable and verifiable through a clear and precise procedure, so that the results obtained during development can be repeated and analyzed afterwards.
- **Output format.** The information provided by the reporting mechanism must be presented in a readable and ordered structure, facilitating inspection, analysis and comparison with the output of common system-monitoring tools.

4.4 Methodology

The methodology followed in this project has mainly been of the *Spiral* type. This choice is due to the fact that, although the problem to be addressed was clearly defined from the beginning, the specific technical solution was not. During the early phases of the project, several aspects required to design an appropriate solution were still not precisely known. For this reason, a spiral methodology was chosen, since it allows the required knowledge to be researched, designed, implemented and validated iteratively and progressively through a series of functional prototypes that serve as the solution to the stated problem.

First, the problem and the technological framework in which the prototype is developed were studied in depth, as well as the technical foundations required to begin designing the *LKM*. Subsequently, several incremental prototypes were built, adding new elements and functionalities in each iteration on top of the validated basis of the previous prototype. This made it possible to progressively build the component until the defined functionalities were achieved. Finally, in each iteration the necessary validations were carried out in the *Linux* development environment itself, also relying on two *User-Space* components that made it possible to verify the correct operation of the created secure reporting system.

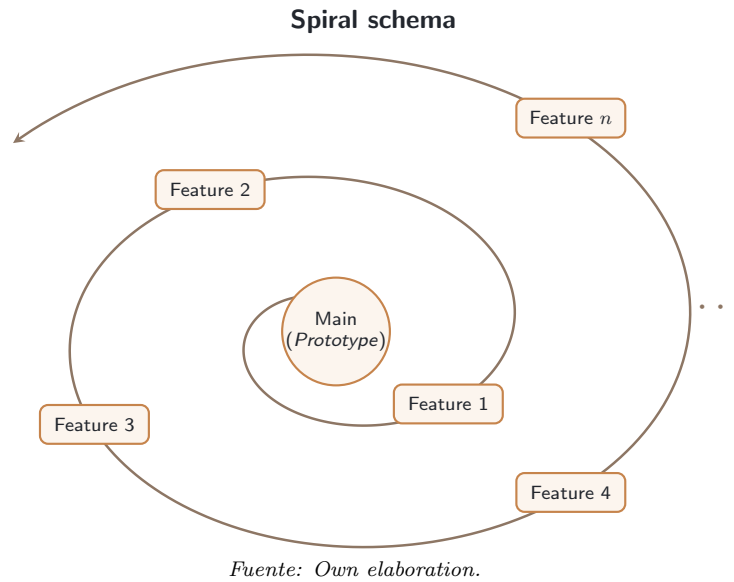


Figure 4.2: Diagram of the *Spiral* methodology used in the project.

In conclusion, the project can be considered to have followed a spiral methodology with progressive refinement over what had been developed in previous iterations, since the final design of the prototype was consolidated as the technical study, implementation and validation of each new prototype progressed.

4.5 Work Plan

The planning followed in the project has been directly aligned with the spiral methodology described in the previous section, progressing through successive iterations that depended on the knowledge and results obtained in the previous ones. Throughout the development of the project, weekly meetings were held with the supervisor, with an average duration of approximately 30 to 60 minutes. In these meetings, the work carried out in the current iteration was reviewed, doubts were resolved and the relevant corrections were proposed in order to refine each prototype. Once an iteration was completed, the partial objectives to be addressed in the next one were established.

The first phase began in July 2025 and focused on studying the development environment (*Linux*), the problem to be addressed (*User-Space rootkits*) and the technical basis required to start proposing a viable practical solution (*LKM* programming and cryptographic mechanisms). After this basis had been consolidated, implementation of the main component of the project began in September 2025.

From then until January 2026, the final prototype of the secure reporting system was progressively developed as the required technical knowledge was further deepened, gradually incorporating the different defined functionalities, carrying out the necessary validations and refining the code through adjustments, comments and corrections until a functional version was achieved.

Once the prototype was finished, the writing of the report began. This required continued research into several of the topics developed in it, both to properly substantiate the problem and the proposed solution and to document the work carried out with technical rigor. During the writing of the report, a generative AI tool, specifically *Gemini 3 Pro*, was also used as an auxiliary tool to generate some schematic and illustrative vector images from concepts previously explained through a well-structured *prompt*, as well as to detect and correct wording and grammar errors in the report. The report was completed at the end of May 2026.

In conclusion, the project followed a progressive and organized plan, with weekly monitoring and the achievement of partial objectives until completing the final result of the Bachelor's Degree Project, including the necessary validations.

Once the security problem to be addressed has been defined, the project objectives have been established and the methodology followed to achieve them has been described, the next chapter focuses on the development and technical explanation of the proposed solution. It details the design of the different prototypes developed until obtaining the final functional version of the secure reporting system, the implementation decisions adopted and the process followed to build and validate the final prototype.

Chapter 5

Design and Development

Once the addressed problem, the project objectives and the technological framework in which it is developed have been defined, this chapter describes the process followed to build the proposed solution. The prototypes developed until reaching the final functional version are presented, describing their implementation and validation, as well as the errors encountered and the solutions adopted.

It should be noted that the official source code of the *Linux kernel* is available at <https://www.kernel.org>, and that the *kernel* version used in the project is 6.12.41-current-bcm2711. This precision is important, since the development and the references consulted are based on the source code of this version, and both functionality and the location of some resources may vary with respect to other *kernel* versions.

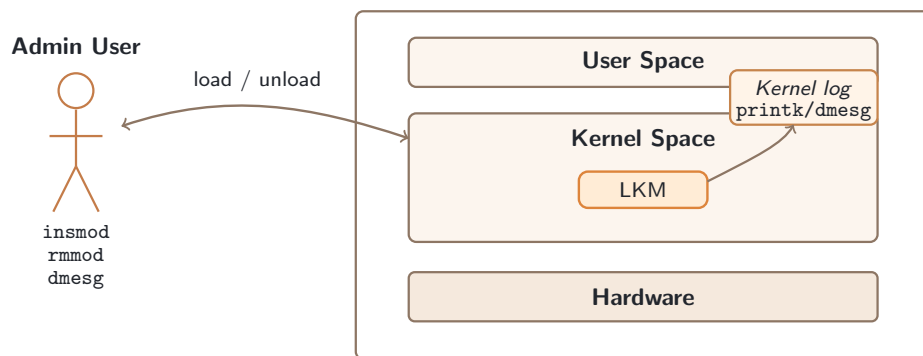
5.1 Prototypes, Implementations and Validations

This section describes the prototypes developed during the project from a conceptual perspective, explaining the theoretical knowledge applied at each stage and how these concepts are progressively integrated into the final solution. The aim is not to reproduce the source code here, but to justify the design followed, detail the operation of each mechanism and relate each prototype to the technical need it addresses within the proposed secure reporting system.

The detailed implementation, code and scripts used, together with the generated validation outputs, are collected in Appendix A. In addition, the complete source code of the project is also available in the public *GitHub* repository <https://github.com/nowelito28/TFG>.

5.1.1 *Hello World*

The first prototype developed consisted of a minimal *Hello World LKM*. It was the first contact with programming inside the *kernel*, validating the basic workflow required to develop code executed in this region.



Fuente: Own elaboration.

Figure 5.1: Basic flow for loading, executing and unloading an *LKM*.

The *LKM* component is the main tool used in the project, since all prototypes are built as external modules and keep the same general cycle of compilation, loading, testing and unloading without needing to reboot the system each time they are executed.

Module compilation relies on *kbuild*, the build system used by the *Linux kernel* itself. Instead of compiling the source file as a conventional user program, the *Makefile* delegates the build to the header tree of the active *kernel* through the path `/lib/modules/$(uname -r)/build`. In this way, the module is compiled with the same *kernel* version that will load it, in this case `6.12.41-current-bcm2711`. The details of the compilation file used are collected in Code A.1 of Appendix A.

From a structural point of view, the module needs to include the headers that expose the basic development primitives in *Kernel Space*. These include the ones required to declare modules (*LKMs*), use *kernel* functions and register initialization and exit

routines. In addition, the prototype defines metadata through macros such as the license, author, description and version. These metadata are not merely informative, since the declared license affects how the *kernel* treats the loaded module. Specifically, declaring a *GPL*-compatible license prevents the *kernel* from marking the module as *tainted* due to license incompatibility, which warns that the *kernel* has been modified or is running components that may make error diagnosis more difficult. The specific declaration of headers and metadata is shown in Code A.2 of the appendix.

The behavior of the module is mainly organized through two entry points. The first is the initialization function, associated with the `module_init()` macro, executed automatically when the module is loaded using `insmod`. The second is the exit function, associated with the `module_exit()` macro, executed when the module is unloaded from the *kernel* using `rmmod`. These points represent the minimum life cycle of an *LKM*: reserving or initializing resources when entering the *kernel* (*load*), and releasing or finishing the work when leaving it (*unload*). In this first prototype no memory is reserved and no communication interfaces are created; the objective is only to demonstrate how both phases of the module and its execution flow work. This logic is detailed in A.3 and A.4.

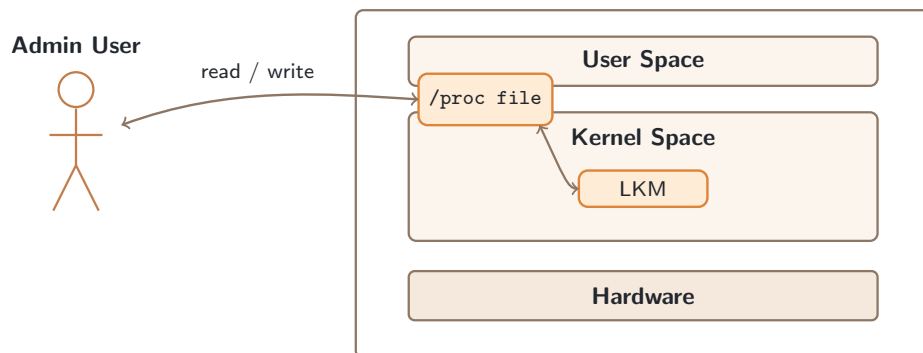
To validate that the code is actually executed inside the *kernel*, `printk` is used. This function is the usual mechanism for registering messages or traces from *Kernel Space*, similarly to how a user program could write to standard output, but integrated into the *kernel* logging system. Each message may include a priority level depending on the trace and its purpose, such as `KERN_INFO`, used for merely informative traces, or `KERN_ERR`, used to register errors. These traces can later be consulted from *User Space* using tools such as `dmesg` or the *kernel* log file `/var/log/kern.log`, accessible from *User Space*. These validation methods are important because they make it possible to confirm simultaneously that the module loads and unloads, that the *kernel* executes its initialization and exit routines correctly, and, if necessary, to inspect debugging traces of the logic implemented in the module.

Finally, once the *LKM* has been programmed, its operation is validated by compiling it, loading it into the *kernel*, checking the generated messages (*logs*), consulting its metadata with `modinfo` and unloading it while verifying the execution of the exit routine. The complete command and output procedure is collected in Code A.5.

With this, the technical basis on which the following prototypes are built was developed and validated.

5.1.2 Virtual File `/proc/fdev`

After validating the basic life cycle of an *LKM*, the next prototype incorporated a communication interface between *User Space* and *Kernel Space*. The objective of this stage was to expose a controlled interaction point that would allow data to be sent to the module and enable customized handling inside the *kernel*. To do this, the `/proc` virtual file system was used as the input interface to the *LKM*, through the file `/proc/fdev`.



Fuente: Own elaboration.

Figure 5.2: Basic communication between *User Space* and *Kernel Space* through a virtual file in `/proc`.

First, in order to create and manage this virtual file, it was necessary to extend the module headers with support for two main aspects: the creation of entries in the `/proc` virtual file system and the controlled transfer of data between *User Space* and *Kernel Space*. The first group allows the virtual file to be registered and associated with custom operations, while the second provides specific primitives for copying information between both memory regions without directly dereferencing pointers coming from *User Space*. This separation is important because the *kernel* and user processes work

in different memory contexts, so direct access to user addresses from the *kernel* must be performed correctly, avoiding unsafe or corrupted accesses. The specific set of headers incorporated in this prototype is collected in Code A.6 of Appendix A.

In this prototype, a specific entry is created inside `/proc` through the file `/proc/fdev`, which is registered during module loading through the initialization function. Conceptually, the module asks the `/proc` subsystem to create a new node visible from *User Space* and to associate that node with a table of operations. This table acts as the contract between file operations invoked from *User Space* and the internal functions of the module. Thus, a read operation on `/proc/fdev` is redirected to the read handler, while a write operation on that same path is redirected to the write handler. When the module is unloaded, the exit routine removes the entry so as not to leave invalid references inside `/proc`. The reference to the virtual entry, its operations table and its registration through `proc_create` are detailed in Code A.7.

In relation to the above, the core of this prototype can be considered to be the read and write handlers. The write handler receives the bytes provided from *User Space*, copies them into a temporary *buffer* in *kernel* memory and interprets them as test values. In the opposite direction, the read handler builds a response inside the *kernel* and returns it to the user *buffer*. To perform these transfers, specific functions such as `copy_from_user` and `copy_to_user` are used, which also make it possible to detect access failures and return errors instead of causing unsafe accesses. The specific implementation of these handlers is found in Code A.8.

Another relevant aspect of this implementation is the use of the file-position pointer. Although `/proc/fdev` does not represent a persistent file, read and write operations still receive a position associated with the open instance. The prototype uses that position to impose a simple single effective operation per opening (*single-shot*) semantics, preventing several consecutive reads or the same write from being processed several times within the same flow. This decision simplifies validation and makes the behavior observed from tools such as `cat` controlled and predictable.

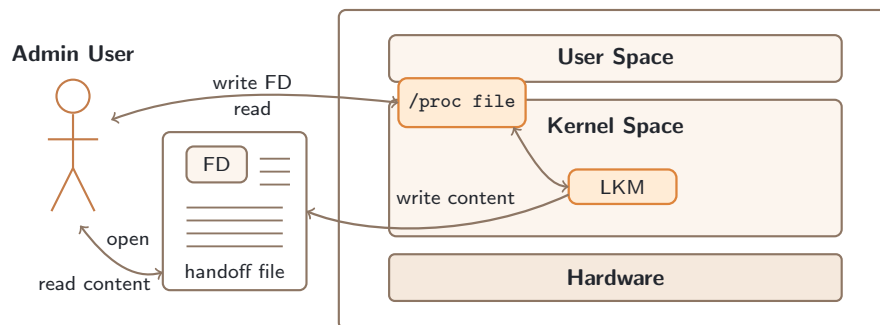
To complete this prototype, the validation process consisted of compiling the module, loading it, checking the creation of `/proc/fdev`, writing values from *User Space*, reading them back, reviewing the traces generated in the *kernel* log and unloading the *LKM*. This verified that the virtual entry was correctly created and

removed, that read and write operations were redirected to the corresponding module handlers, and that information transfer between both regions worked in a controlled way. The complete procedure of commands and outputs is collected in Code A.9.

In conclusion, this prototype establishes the first functional interface between the administrator and the *kernel module*. From this point on, the following prototypes reuse `/proc/fdev` as the input channel to transfer information from *User Space* to the *LKM*, progressively extending the logic executed inside the module.

5.1.3 File Handoff

Once basic communication through the virtual file `/proc/fdev` had been established, the next prototype incorporated a *File Handoff* mechanism. The objective of this stage was to allow the *LKM* not only to receive simple data from *User Space*, but also to resolve the reference of a file descriptor (*FD*) opened by the administrator and use it as a communication path towards *User Space*.



Fuente: Own elaboration.

Figure 5.3: Transfer of a *File Descriptor* from a real file to the *LKM* to establish a communication path from *Kernel Space* to *User Space*.

An *FD* is an integer identifier of a file associated with the open-file table of the process that owns it, and it may reference a regular file, a network connection, etc. Therefore, when a process in *User Space* opens a file, the system returns an integer

that serves as a local reference to an internal *kernel* structure representing that file. The *File Handoff* mechanism consists of sending that identifier to the module through `/proc/fdev`, so that the *kernel* can resolve it and operate on the referenced file.

To support this mechanism, it was necessary to extend the module headers with functionality related to the file subsystem, descriptor resolution, safe conversion of strings to integers and permission validation. This set of dependencies makes it possible to move from text-based communication to interaction with real *VFS* objects in the *kernel*. The specific headers incorporated in this prototype are collected in Code A.10 of Appendix A.

In this prototype, the read function keeps an auxiliary role. Instead of returning test values as in the previous prototype, it informs the administrator that the module is ready to receive an *FD* and execute the *File Handoff* mechanism. In this way, reading `/proc/fdev` works as a simple status check for the *LKM*, while the main logic is concentrated in the write operation. The specific implementation of this function is shown in Code A.11.

On the other hand, writing to `/proc/fdev` is the main entry point of the prototype. The administrator opens or creates the destination file in write and append mode from *User Space*, obtains its *FD*, converts it to text and writes it to `/proc/fdev`. From the *LKM* point of view, that written text string is received as a sequence of *bytes*, copied to *kernel* memory, interpreted as an integer and checked to determine whether the extracted *FD* is valid. The *User-Space* auxiliary program responsible for opening the destination file and sending the *FD* to the module is collected in Code A.15.

Once the descriptor is received, the module uses a *kernel* primitive to transform the integer into an internal reference of type `struct file`. This step is important because the *FD* alone does not allow the file to be written directly; it must first be resolved through the open-file table of the process invoking the operation. If the resolution succeeds, the *LKM* obtains a valid reference to the `struct file` object of the file and must release it later when it no longer uses it. This main logic of descriptor reception, conversion, resolution and release is detailed in Code A.14.

Before writing from *Kernel Space*, the prototype checks that the file satisfies the defined policy. Specifically, the file is required to have been opened with write capability

and in *append* mode. This decision avoids accidental overwrites and problems with the write-position pointer, ensuring that the information generated by the module is always added at the end of the destination file. In addition, the effective permissions on the file are checked to verify that write and append operations are authorized. The auxiliary function responsible for these checks is referenced in Code A.13.

The effective write to the file is not performed using the usual *User-Space* functions, but through the writing functionality available inside the *kernel*, using data stored in this same region (`kernel_write`). For this purpose, an auxiliary function (`write_full`) is implemented to encapsulate a single call to `kernel_write` and centralize error checking, verifying that the operation does not fail and that the number of written *bytes* matches the requested size. The auxiliary write function is found in Code A.12.

Finally, validation of the prototype consisted of compiling and loading the module, checking that `/proc/fdev` was available, creating the destination file through the user utility, sending its *FD* to the module and verifying that the test content was correctly written from *Kernel Space*. The *kernel* traces were also reviewed and the module was unloaded at the end of the test. The complete procedure is collected in Code A.16.

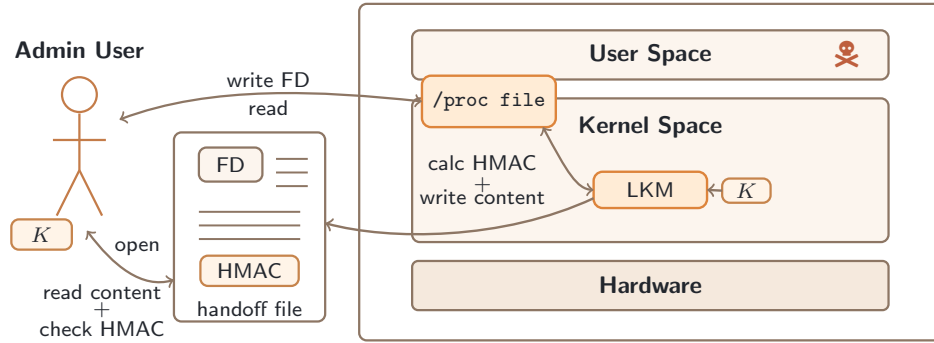
In conclusion, this prototype extends the interface created in the previous stage by allowing the administrator to dynamically indicate the reference to the file where the information generated by the *LKM* will be deposited. From this point on, the following prototypes reuse this mechanism as the main output channel for writing content generated in *Kernel Space*, subsequently adding cryptographic authentication and the final report of system processes.

5.1.4 *HMAC-SHA256* Cryptographic Authentication Mechanism

After validating the *File Handoff* mechanism, the next prototype incorporated cryptographic authentication over the content generated in *Kernel Space* and written to the file referenced in *User Space* through an *HMAC-SHA256*.

The objective of this stage was for the file written by the *LKM* to contain not

only data, but also an authentication tag that would later allow checking whether that content had been authenticated with the expected secret key and whether it had not been modified after being written.



Fuente: Own elaboration.

Figure 5.4: Incorporation of an *HMAC-SHA256* authentication mechanism into the *File Handoff* system.

An *HMAC-SHA256* cryptographic tag was used as a message-authentication mechanism, based on a secret symmetric key (K) and on the *SHA-256* hash function, whose block size is 512 *bits*. It is important to emphasize that this mechanism does not encrypt the content, so the data written to the file remain readable. Its purpose is to generate an authentication value that depends simultaneously on the content and on the secret key K , so that, for a given content, only one valid *HMAC-SHA256* tag can be computed for a given symmetric key K . Thus, any modification of the content, or any attempt to generate a tag without knowing that key, would cause the subsequent verification not to match, thereby ensuring the integrity of the content to be transferred.

For this mechanism to be meaningful, the key K must be secret and shared only between the *LKM* in *Kernel Space* and the content-verification utility used by the system administrator in *User Space*. As explained in the threat model in Section 4.2, in the environment surrounding the proposed secure reporting system it is assumed that the attacker does not have access to this secret symmetric key (K) according to the described chain of trust, so valid tags cannot be generated for forged content and the content cannot be modified without invalidating the tag. Otherwise, this mechanism would not provide any integrity to the content to be transferred.

In this project, a symmetric key K with an optimal size of 64 *bytes* is generated, matching the block size used by *SHA-256*. If the key were longer than that block size,

HMAC would first reduce it by computing $SHA-256(K)$, obtaining a 32-byte output, and would then pad it with zeros (0x00) on the right until reaching the block size. If the key were shorter, it would also be padded with zeros on the right up to 64 *bytes*.

Once the binary *HMAC-SHA256* tag has been computed using the 64-byte block processing of the content and the optimal-size symmetric key (K), the prototype encodes it in *Base64* before writing it to the transfer file. This encoding groups the input binary data into 3-byte blocks (24 *bits*), which are then divided into four 6-bit groups, representing each 6-bit block with one character from the limited 64-character *ASCII* alphabet (A-Z, a-z, 0-9, +, /). For this reason, the 32 *bytes* of the *HMAC-SHA256* tag become 44 *Base64* characters, including the final padding character (=) when padding is applied to incomplete groups, as in this case for a 32-byte size, since it does not form an integer number of 3-byte groups. This conversion does not modify the cryptographic security of the mechanism, but only its representation so that it can be stored and processed as text.

To integrate this mechanism into the *LKM*, support for the *kernel Crypto API*, *Base64* encoding and the embedded symmetric key had to be incorporated. The key was not written manually in the code; instead, it was generated through an auxiliary script that creates a random 64-byte value and transforms it into a C header importable by the module (.h). In this way, both the *kernel* and the administrator can operate with the same secret key during the test. The added headers and the key-generation process are collected in Codes A.17 and A.18 of Appendix A.

The calculation of the *HMAC* tag is delegated to the *kernel Crypto API* through a synchronous *hash* instance (**shash**) associated with the algorithm **hmac(sha256)**. Conceptually, the module requests this *hash* instance, associates the previously loaded secret symmetric key K with it within the considered chain of trust, reserves memory for the result and executes the calculation on the received data block. The high-level call abstracts the internal *HMAC* operations. The specific implementation of the calculation is shown in Code A.19.

Once the 32-byte binary *HMAC* has been computed, the prototype converts it to *Base64* following the process explained above. The binary 32-byte tag becomes a 44-character *Base64* string: ten complete groups generate 40 characters and the remaining two bytes generate four additional characters with padding. Since this

is a file generated to be reviewed and validated from *User Space*, encoding the tag in *Base64* avoids writing raw binary bytes and facilitates its later extraction by the checking program. The auxiliary function responsible for this conversion is collected in Code A.20.

The final format written to the *handoff* file is organized using textual separators. First, a marker identifying the beginning of the content generated by the *kernel* is written, then the protected content is added, then a specific separator for the tag is written, and finally the *HMAC* encoded in *Base64* is incorporated. This structure makes the file deterministic and allows the verification utility to separate the transferred useful information from its authenticator without ambiguity. The logic for jointly writing the content and its *HMAC* tag is detailed in Code A.21.

In this prototype, the main integration function still generates fixed test content, computes its *HMAC-SHA256*, encodes it in *Base64* and writes the complete set to the real file received through the *File Handoff* mechanism. Therefore, the logic for receiving the *FD*, resolving it through the `struct file` object and validating the file remains the same as in the previous prototype, but writing to that file from the *kernel* is replaced by authenticated writing. This integration is shown in A.22 and A.23.

To close the validation cycle, it was also necessary to implement an additional utility in *User Space*, with which the administrator could validate the integrity and authenticity of the transferred content, even on another machine. This tool opens the transfer file, locates the separators, extracts the content or *payload* on the one hand and the *HMAC* tag in *Base64* on the other, locally recalculates that authentication tag with *OpenSSL* using the same key *K* over the extracted content, encodes the result again and compares both tags. The comparison is performed with a specific cryptographic primitive, avoiding an ordinary string comparison and reducing exposure to timing side-channel attacks. The main parts of this utility are collected in A.24, A.25, A.26 and A.27.

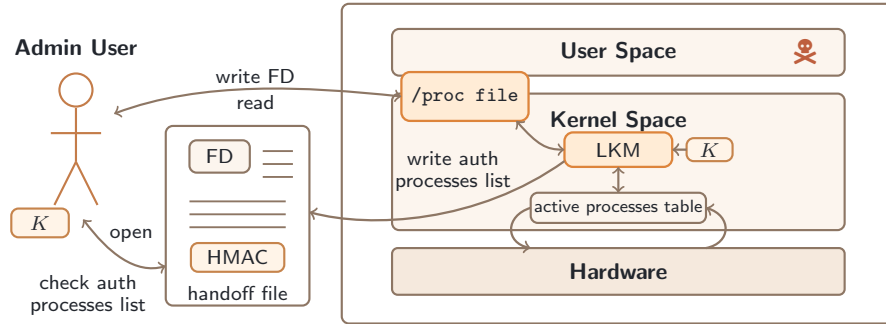
Finally, the practical validation confirmed that the module could generate authenticated content in *Kernel Space*, write it to the file received from *User Space* and later verify that the received tag matched the one recalculated by the user utility. The complete compilation, execution and verification procedure is collected in Code A.28.

This implementation is used as the authenticity and integrity mechanism in the final project solution, providing a robust tool against *User-Space rootkit* attacks under the defined *Threat Model*. From this point, the next and final step consisted of replacing the fixed test content with real critical system information, dynamically generated in *Kernel Space* and protected using the same *HMAC-SHA256* scheme.

5.1.5 Secure Reporting System for Running System Processes

Once the *File Handoff* channel and the *HMAC-SHA256* authentication mechanism had been designed, the final prototype replaced the fixed test content with real critical system information dynamically generated in *Kernel Space*.

The objective of this stage was to build a report of all active system processes and transfer it in an authenticated way using the mechanisms designed in the previous prototypes.



Fuente: Own elaboration.

Figure 5.5: Generation and authenticated transfer of a report of the active system processes from *Kernel Space*.

This prototype unifies all the capabilities developed in the previous stages, adding functionality for generating reports of active processes as content to be transferred in an authenticated way. Communication between the administrator and the module continues to be performed through the virtual file `/proc/fdev`. The administrator still provides an *FD* of a real file, the *LKM* resolves that descriptor as a `struct file` object, and the final write preserves the structure formed by content, textual separator and *HMAC-SHA256* tag encoded in *Base64*. The main difference with respect to the previous prototype is that the protected content is no longer a fixed string, but a table generated from the internal structures of the *kernel* with the main information of all active processes in the system.

To build this report, it was necessary to extend the module dependencies with headers related to credentials, processes, identifiers, namespaces and synchronization. These headers make it possible to access the credentials of each running task, traverse the global process list, obtain global and local identifiers with respect to active namespaces and protect the traversal through the *RCU* mechanism, avoiding race conditions on shared *kernel* structures. In addition, constants were defined to limit the maximum size of the report and simplify the implementation; fixed widths were also reserved for each field and a well-spaced header was built with the metadata to be shown for each process, improving readability. The headers, constants and initial table structure are collected in Code A.29 of Appendix A.

The format chosen for the report is inspired by the output of tools such as `ps`, but with fields adapted to the objective of the project. Each row represents a running process and includes the global user identifier *UID* visible from the *kernel* and the local one relative to the process namespace, the user namespace identifier, the global *PID* and the *PID* relative to its namespace, together with the process namespace identifier, the group identifier *GID* and the associated command name. This combination allows the global view of the system seen from the *kernel* itself to be compared with the local view that processes running inside containers or differentiated namespaces may have.

Before traversing the processes, the module incorporates auxiliary functions to build the content in an ordered and controlled way, also improving its later interpretability. On the one hand, one of them fills each field with spaces until reaching a fixed width, so that the columns of the report are fully aligned. On the other hand, another auxiliary function copies each fragment into the global *buffer* with additional padding if required, but only if enough space is available, preventing overflows while concatenating rows. These formatting and safe data-copying operations are detailed in Code A.30.

The central function of this final prototype is responsible for generating the complete report content. First, memory is allocated in *Kernel Space* for a fixed maximum-size *buffer*; the table header is added; and a read-side section protected by *RCU*, delimited by `rcu_read_lock` and `rcu_read_unlock`, is entered. Within this section, the global task list is traversed using `for_each_process`, checking before each insertion that the *buffer* still has enough space for the new row plus the separators and the *HMAC-SHA256* tag. For each process, the extraction and serialization of its metadata is delegated to an auxiliary function. The general construction of the in-memory list is shown in A.31.

The use of *RCU* is highly relevant because the process list and the `task_struct` structures may be consulted while the system continues executing and modifying its state. The read-side protection does not stop the system or globally block the creation or termination of processes, but it allows references to be traversed coherently during report generation.

The serialization of each row is encapsulated in a specific function that receives the current task and adds its fields to the global report *buffer*. This function does not directly calculate all values; rather, it coordinates specialized auxiliary calls to obtain

each metadata item, format it as a portable textual string and add it sequentially. If any extraction fails or if there is not enough space in the *buffer*, generation is aborted to avoid producing an inconsistent report. The serialization logic of a complete row is referenced in Code A.32.

User identifiers (*UIDs*) are handled by distinguishing between the global value and the local value with respect to a specific namespace. The global *UID* is obtained using the initial system namespace as reference, so it represents the *kernel* view. The local *UID*, on the other hand, is obtained from the namespace associated with the process credentials, making it possible to reflect the identifier value that the user would have inside its own context. The internal identifier of the user namespace is also included, allowing processes that may share local values but belong to different namespaces to be distinguished. These functions are shown in A.33.

Similarly, process identifiers (*PIDs*) are separated into global *PID* and local *PID* with respect to the namespace associated with each task. The global *PID* corresponds to the identifier visible from the *kernel*, while the local *PID* is obtained with respect to the namespace in which the task operates. This distinction is especially useful in container environments, where a process may appear with local *PID* 1 inside the container even though it has a different *PID* from the global perspective of the system. The extraction of these fields is collected in Code A.34.

The report is completed with the group identifier (*GID*) and the command associated with each process. For the *GID*, the process credentials are used and its global value is obtained with respect to the initial namespace of the system. As the final type of metadata, the command name associated with each process is obtained, distinguishing between user processes with an associated executable, *kernel* threads, shown between brackets [], and cases where only the name stored in `task->comm` is available. When an associated executable exists, its path is resolved through the internal structures of the executable file; if this is not possible, the short process name registered in the task itself (`task->comm`) is used. These functions are referenced in A.35.

Once the real critical report content has been built, the flow again matches the previous prototype. The module calculates the *HMAC-SHA256* over the complete generated process table, encodes the tag in *Base64* and writes to the transfer file

the start marker, the process table as critical information to be transferred, the tag separator and the final *HMAC*. In this way, authentication does not protect only a test string, but the complete list of running system processes generated with their most relevant metadata from *Kernel Space*. The joint writing of the content and its authenticator relies on the logic previously described in Code A.21.

The result written to the transfer file keeps a clear textual structure, where the process block and the authentication tag are delimited by explicit markers. Thus, the final content of the file would have a structure like the one shown below if that file were empty, since writing is performed in *append* mode:

```
--KERNEL-PS--
UID_KERNEL UID_LOCAL UID_NS      PID_KERNEL PID_LOCAL PID_NS      GID      COMMAND
0           0         4026531837 1           1         4026531836 0        /usr/lib/systemd/systemd
0           0         4026531837 2           2         4026531836 0        [kthreadd]
0           0         4026531837 3           3         4026531836 0        [pool_workqueue_]
...
113         113        4026531837 1440        1440       4026531836 119      /usr/sbin/chronyd
113         113        4026531837 1448        1448       4026531836 119      /usr/sbin/chronyd
0           0         4026531837 1484        1484       4026531836 0        /usr/sbin/gdm3
0           0         4026531837 1489        1489       4026531836 1000     /usr/libexec/gdm-session-worker
1000        1000       4026531837 1496        1496       4026531836 1000     /usr/lib/systemd/systemd
1000        1000       4026531837 1497        1497       4026531836 1000     /usr/lib/systemd/systemd-exe
...
0           0         4026531837 8211        8211       4026531836 0        /usr/bin/dockerd
0           0         4026531837 8212        8212       4026531836 0        [kworker/1:1]
0           0         4026531837 8239        8239       4026531836 0        [kworker/1:2H]
0           0         4026531837 8500        8500       4026531836 0        /usr/bin/containerd-shim-runc-v2
100000      0         4026532674 8520        1          4026532678 100000   /usr/bin/sleep
0           0         4026531837 8532        8532       4026531836 0        [kworker/2:0]
0           0         4026531837 8596        8596       4026531836 0        /usr/bin/containerd-shim-runc-v2
100000      0         4026532754 8617        1          4026532758 100000   /usr/bin/sleep
0           0         4026531837 8667        8667       4026531836 0        [kworker/2:2H]
0           0         4026531837 8678        8678       4026531836 0        /usr/bin/containerd-shim-runc-v2
100000      0         4026532834 8698        1          4026532838 100000   /usr/bin/sleep
0           0         4026531837 8741        8741       4026531836 0        [kworker/u20:2]
0           0         4026531837 8935        8935       4026531836 0        [kworker/0:0]
0           0         4026531837 8947        8947       4026531836 0        [kworker/u16:3]
0           0         4026531837 8989        8989       4026531836 0        [kworker/3:2]
0           0         4026531837 9033        9033       4026531836 0        /home/noel/Project/mk_fhandoff

--HMAC--
ckvLUckcgmt6RFfdGzuvgnAEgt9rLPJ5q9FYJoFKAW0=
```

Code 5.1: Example of the final structure of the authenticated transfer file.

Validation of this final prototype was carried out by compiling and loading the module, generating the embedded key again, enabling the output file through the *File Handoff* utility and verifying the result with the *User-Space HMAC* checking program. In addition, several *Docker* containers were executed to force the presence of processes in different namespaces and verify that the report reflected differences in fields such as *UID_LOCAL*, *PID_LOCAL*, *UID_NS* and *PID_NS*, also with respect to their global identifiers. The complete validation procedure, including file generation, the presence of container

processes and cryptographic verification, is collected in Code A.36.

Finally, based on the results obtained, it was concluded that the *LKM* was capable of generating from *Kernel Space* a global table of active system processes, including relevant metadata of users, processes, groups and namespaces, transferring that information to a transfer file indicated by the administrator and attaching an *HMAC-SHA256* tag that can later be verified from *User Space*. This completes the final prototype of the secure reporting system, integrating into a single solution the communication between *User Space* and *Kernel Space*, the generation of critical information from a privileged region and the cryptographic authentication of the transferred content to ensure its integrity against *User-Space rootkits*.

5.2 Problems and Errors Encountered

During the development of the project, it was not only necessary to progressively implement the prototypes described above, but also to study truly complex details and correct initial decisions that, although they could work in simple tests, were not appropriate for a solution executed inside the *kernel*.

The main difficulty was that each mechanism had to respect the rules of the system core: file access could not be treated as a conventional *User-Space* write, the cryptographic calculation had to rely on the internal *kernel* infrastructure, and process traversal had to take system concurrency issues into account. For this reason, the final decisions were not limited to producing correct output; they also sought to avoid unsafe accesses, follow a methodology compatible with *kernel* functionality and maintain a structure that could be verified from *User Space* in a simple and practical way.

The three most relevant problems encountered during development and the solution adopted in each case are described below.

5.2.1 Writing to the Transfer File in *Append* Mode

One of the first important problems appeared when designing the *File Handoff* mechanism. The objective was for the administrator to open a real file in *User Space*, send its *FD* to the *LKM* and have the module write to that file from *Kernel Space*. Initially, this operation might seem equivalent to an ordinary write on an open file, but when analyzing the actual *kernel* write flow, it became clear that handling the file-position pointer (`f_pos`) was a truly delicate point to consider.

In a normal write, the position of the file write pointer determines where the new bytes are inserted. If the module wrote directly using the pointer associated with the `struct file` object, it would have to assume responsibility for correctly controlling that position, especially if the file could be used by more than one execution path. This would have added additional complexity to the prototype, since it would be necessary to reason about pointer updates, locking mechanisms over the pointer, possible intermediate writes and accidental overwrites of previous content.

Moreover, since the final content is written in several parts (initial marker, content, separator and *HMAC* tag), incorrect position management could produce a poorly formatted file that would be difficult to verify afterwards.

The adopted solution was to require that the received file be opened with write capability and in *append* mode. In this way, the prototype policy is restricted to adding the report at the end of the transfer file, avoiding dependence on an arbitrary position provided by the previous state of the descriptor and on its correct management and modification.

This decision was implemented in the file metadata validation, checking both `FMODE_WRITE` and the `O_APPEND` flag, and also verifying effective permissions through `file_permission` with `MAY_WRITE` and `MAY_APPEND` inside the *LKM*. The code associated with this validation logic is found in A.13.

In addition, the auxiliary function `write_full` was implemented to keep the write operation from the *LKM* encapsulated. This function performs a single call to `kernel_write` and checks both the possible error code returned by the *kernel* and whether the number of written *bytes* matches the requested size, thereby ensuring that

a complete write is performed.

With this solution, the module maintains a more controlled write semantics, accepting only files compatible with the defined append-mode write policy and always writing at the end of the file. The final design of this part is collected in Code A.12.

5.2.2 Integration of the *HMAC-SHA256* Mechanism between *Kernel Space* and *User Space*

Another especially delicate point was the incorporation of the cryptographic authentication mechanism into the *LKM*. The first tests made it possible to validate the concept from *User Space*, generating a symmetric key and calculating an *HMAC-SHA256* using *OpenSSL*.

However, transferring that idea to the *kernel module* did not consist of directly reusing the same user-level functionality, since the *kernel* does not provide the same libraries or the same memory model. It was necessary to rely on the *kernel Crypto API*, specifically on the synchronous *hash* interface (*shash*) associated with the *hmac*(sha256) algorithm.

The calculation required first requesting the *hash* instance (*shash*) with *crypto_alloc_shash*, associating the key through *crypto_shash_setkey*, obtaining the actual digest size with *crypto_shash_digestsize* and allocating memory for the resulting tag using *kmalloc*. Then, the temporary descriptor required by the *API* had to be built with *SHASH_DESC_ON_STACK*, the digest executed with *crypto_shash_digest* and errors and resource release controlled at every step, knowing that the output of this whole process is binary. This made the mechanism much more complex than calculating it with *OpenSSL* in *User Space*.

Another important obstacle in this part of the project was correctly coordinating three aspects: the shared key, the authenticator calculation in *Kernel Space* and the subsequent verification from *User Space*.

During development, the project moved from initial tests with manually created keys

to an embedded key generated beforehand through an auxiliary script in a random way. This change allowed the *LKM* and the administrator's *User-Space* verification utility to work with the same key K in a practical way. The generation of the header with the embedded key is collected in Code A.18.

On the other hand, the representation of the *HMAC* had to be handled correctly. The result of *HMAC-SHA256* is a 32-byte binary tag, not a directly readable text string. Writing those bytes without encoding would complicate manual inspection of the file and its verification. Therefore, the module converts the binary tag to *Base64* before writing it to the transfer file, generating a stable textual representation that can be read, separated and recalculated by the user utility in a simple and practical way. This decision required careful handling of the file structure, explicitly separating the protected content from the tag by means of textual markers, so that verification is performed exactly over the same bytes that had been authenticated by the *kernel*.

Finally, the checking utility in *User Space* had to reproduce the same process coherently: extracting the content between separators, recalculating the *HMAC-SHA256* with the same key, encoding it in *Base64* and comparing it with the value written by the module. The comparison is also performed through a specific cryptographic primitive, avoiding an ordinary string comparison and thereby reducing possible information leakage due to timing differences in the comparison. This closed the complete authentication cycle: generation inside the *kernel*, structured textual transfer and external verification of the content. The implementation of this mechanism is detailed in A.19, A.20, A.21 and A.27.

5.2.3 Safe Traversal of the Process List Using *RCU*

The last relevant problem during the development of the project appeared when replacing the fixed test content with real system information. The final prototype had to traverse the global *kernel* task list and extract the most relevant metadata of each process, such as user, process and group identifiers, namespaces and the associated command name.

This operation cannot be treated as a static traversal of an immutable structure, since

the process list changes continuously while the system is running: new tasks may be created, processes may terminate or internal structures related to them may be modified.

An initial approach without specific protection would have exposed the module to race conditions during traversal. For example, while an entry in the list is being consulted, the corresponding process could terminate or change state, leaving inconsistent or potentially invalid references.

In code executed inside the *kernel*, because this is a privileged region of the system, this type of error is especially critical, since it is not limited to producing incorrect output, but may cause inconsistent reads, invalid memory accesses or failures in the module itself. For this reason, traversing internal structures such as the process list requires respecting the synchronization mechanisms provided by the *kernel*.

The adopted solution was to protect the traversal through an *RCU* (*Read-Copy-Update*) read-side section, delimited by `rcu_read_lock` and `rcu_read_unlock`. *RCU* allows the module to traverse shared references safely without globally blocking process creation or termination. This fits the objective of the project, since the system does not need to be completely frozen, but rather to obtain a practical and coherent snapshot of the active processes from a privileged region. The `for_each_process` macro is executed inside this protected section, and each task is immediately serialized into the report *buffer* through auxiliary functions.

This solution does not turn the report into a perfect transactional snapshot of the system, since the operating system continues executing and its state may change while the list is being generated. Nevertheless, it does avoid traversing the process list without protection and reduces the risk of accessing unstable references while building the report.

Moreover, at the points where task credentials are consulted, the module uses controlled references through the corresponding primitives, such as `get_task_cred` and `put_cred`, so as not to depend on structures whose lifetime is not protected. The construction of the list under *RCU* protection is shown in Code A.31.

The final solution developed does not completely eliminate the risk associated with a compromised system, but it does provide a practical mechanism to increase trust in a specific source of critical information, reducing exclusive dependence on *User-Space* tools when they may have been altered by a *rootkit*. The following chapter further explores the conclusions obtained after completing the project from different points of view.

Chapter 6

Conclusions

“You can’t trust code that you did not totally create yourself.”

Ken Thompson (*Reflections on Trusting Trust*, 1984) [31].

After presenting the addressed problem, the required technological framework, the threat model, the defined objectives and the progressive development of the solution, this final chapter provides an overall assessment of the project.

Its main purpose is to interpret the obtained result from a broader perspective, considering its impact, the skills applied during the work, the remaining limitations and the possible lines of evolution.

The developed prototype demonstrates that it is possible to build a secure reporting mechanism for *Linux* systems capable of generating critical information from *Kernel Space*, specifically a list of the active processes in the system, transferring it in an accessible way for the administrator through a transfer file and authenticating it using an *HMAC-SHA256* tag that can later be verified in *User Space*. This solution does not completely eliminate the risks associated with a compromised system, but it does provide a practical way to reduce exclusive dependence on *User-Space* tools when these may be manipulated by a *rootkit* operating in that region. Therefore, the conclusions of the project must be understood both from the technical result achieved and from the security, responsibility and sustainability implications associated with this type of mechanism.

6.1 Achievement of Objectives

Once the progressive design of the adopted solution, its validations and the main problems encountered during development have been described, it is necessary to compare the final result with the objectives defined in Chapter 4. This closing assessment makes it possible to verify whether the implemented prototype truly responds to the initial problem and whether it provides the administrator with a reliable source of critical information when *User Space* may be compromised by a *rootkit*.

The final result of the project consists of a *kernel module* capable of generating from *Kernel Space* a structured list of the processes present in the system, including relevant metadata about users, processes, groups, namespaces and the associated command name. This content is transferred to a transfer file indicated by the administrator through a *File Handoff* mechanism, and is accompanied by an *HMAC-SHA256* tag calculated with a shared symmetric key, allowing subsequent verification from *User Space* to check the authenticity of the content.

Therefore, the final prototype fulfills the general objective, since it moves the generation of critical information to a more privileged region and adds a cryptographic mechanism to detect later modifications of the report.

6.1.1 Achievement of Functional Objectives

Regarding the functional objectives defined in Section 4.3.1, the developed solution can be considered to fulfill the main capabilities initially planned.

- **Obtaining critical information from *Kernel Space*.** This objective is fulfilled through the final *LKM*, which traverses the *kernel* task list and builds the report directly from a privileged region, without relying on *User-Space* monitoring tools. This decision responds to the main premise of the threat model, where such user tools could be manipulated.
- **Building a global and structured process view.** The generated report includes the fields defined in the objectives: `UID_KERNEL`, `UID_LOCAL`, `UID_NS`,

PID_KERNEL, PID_LOCAL, PID_NS, GID and COMMAND. This provides a more complete view than a simple process list, since it also reflects differences between global and local identifiers, especially relevant in environments with *namespaces*.

- **Controlled transfer to a channel accessible from *User Space*.** This objective is fulfilled through the virtual entry `/proc/fdev` and the user utility responsible for creating the transfer file and sending its *File Descriptor* to the *kernel*. From that reference, the module resolves it and writes the report directly to the file selected by the administrator, keeping validations over the descriptor, permissions and opening mode under control.
- **Cryptographic authentication of the generated content.** The report content is authenticated through *HMAC-SHA256*, calculated inside the *kernel* using the *Crypto API* and a previously embedded symmetric key *K*. In this way, the generated tag is associated with the exact content produced by the module.
- **Subsequent verification from *User Space*.** This objective is fulfilled through the checking utility developed in C, which extracts the protected content, recalculates the *HMAC-SHA256* with the same key, encodes the result in *Base64* and compares it with the tag written by the *kernel*. The validation confirms that the administrator can later verify the integrity and authenticity of the received report from another non-compromised machine.
- **Facilitating the detection of concealment or manipulation.** The system does not implement an automatic *rootkit* detection engine, but it does fulfill the objective of providing an authenticated source of information generated from the *kernel* that can be compared with the output of *User-Space* tools. Therefore, it facilitates the detection of inconsistencies when a *rootkit* manipulates the view offered by conventional commands such as `ps`, leaving the final analysis decision to the administrator.
- **Prototype validation.** The prototype was validated in a real *Linux* environment using the *kernel* version employed during the project, compiling and loading the module, generating the embedded key, creating the transfer file, executing processes in *Docker* containers and cryptographically verifying the resulting report. The tests show that the system generates the expected list and that the *HMAC* tag can be correctly checked from *User Space*.

6.1.2 Achievement of Non-Functional Objectives

The non-functional objectives in Section 4.3.2 are also covered by the design decisions adopted during development.

- **Verifiable integrity and authenticity.** This is achieved through the use of *HMAC-SHA256* and later verification with the same symmetric key, under the premise that both the *kernel* and the administrator have the same symmetric key K and that no one else has access to it. Encoding the tag in *Base64* also facilitates its textual storage and checking in the transfer file.
- **Robustness in communication between spaces.** The module validates the input received from *User Space*, limits the write size over the */proc* entry, transforms the received descriptor in a controlled way using `fget` and checks that the destination file is writable and opened in *append* mode. In addition, the use of auxiliary functions such as `write_full` and `safe_chunk` centralizes error checking during writing and reduces the risk of overflows in the report *buffer*.
- **Security with respect to the defined threat model.** The designed solution minimizes trust in user tools, since the sensitive content is generated inside the *kernel*. However, this guarantee holds within the assumptions of the threat model: the *kernel*, the *hardware*, the *firmware* and the shared key must remain intact. Therefore, the system does not aim to protect against *Kernel-Space rootkits* or against scenarios where the symmetric key has been exposed.
- **Modularity and low impact on the *kernel*.** The mechanism is implemented as an *LKM*, so it can be loaded and unloaded without permanently modifying the source code of the operating-system core. This decision facilitates controlled deployment of the prototype and fits its experimental nature, although it does not yet turn the mechanism into a complete production tool with advanced key management, alert integration or hardening against more complex concurrent loads.
- **Compatibility with standard *Linux* mechanisms.** The implementation relies on interfaces belonging to the *Linux* ecosystem, such as */proc*, `proc_create`, `kernel_write`, `fget`, *RCU*, the *kernel Crypto API* and conventional C compilation tools.

- **Prototype reproducibility.** The project provides the source code of the *LKM*, the user utilities, the script for generating the embedded key and the validation sequences collected in the appendix and in the *GitHub* repository: <https://github.com/nowelito28/TFG>. This makes it possible to repeat and use, under license, the flow of compilation, loading, execution, verification and unloading of the designed module, provided that compatible *Linux* environments are used.
- **Managed portability.** Although the validation was performed on a Raspberry Pi 4 with *ARM* architecture, the design is not conceptually tied to that specific board. The solution relies on native *Linux* mechanisms and standard compilation tools, so it could be transferred to other compatible *Linux* environments by making the necessary adaptations to the *kernel*, the available headers and the target architecture.
- **Readable and ordered output format.** The transfer file contains explicit separators, an aligned process table and a final *HMAC* tag in *Base64*. This structure facilitates manual inspection, automatic extraction of the protected content and comparison with other system information sources.

Therefore, it can be concluded that the project achieves the defined objectives and materializes a functional, verifiable solution consistent with the stated problem. Nevertheless, the result should not be understood as a final or complete solution, but as a prototype that demonstrates the feasibility of the proposed approach and opens the door to future improvements, extensions and validations in more complex or realistic scenarios.

6.2 Social, Economic, Temporal, Environmental and Ethical Impact

The main social impact of the project lies in the field of cybersecurity and trust in critical system information. In scenarios where an attacker has managed to compromise *User Space*, the usual administration tools may no longer provide a reliable view of the real state of the machine. The developed mechanism helps mitigate this

problem by providing the administrator with an alternative and authenticated source of information about active processes, generated from a more privileged region and verifiable through a shared symmetric key.

From this perspective, the project may have a positive impact on analysis, incident response and technical auditing tasks. Having an authenticated report makes it possible to contrast the information offered by conventional tools such as *ps* with a detailed view obtained from the *kernel*, facilitating the identification of signs of concealment or manipulation. This type of mechanism is especially relevant in systems where service continuity, activity traceability and trust in sensitive monitoring data are critical aspects.

Nevertheless, it must be taken into account that the proposed mechanism is aimed at a technical profile. Its use requires understanding concepts of *Linux* system administration, C programming, basic *kernel* operation and cryptographic verification using *HMAC*, as well as general cybersecurity knowledge. Therefore, it is not proposed as a tool intended for end users without technical knowledge, but as support for administrators, analysts or profiles with training in systems, programming and cybersecurity.

It should also be noted that, in environments subject to internal security policies or legal requirements for traceability and information protection, having verifiable technical evidence can facilitate the justification of decisions during an audit or security investigation.

Regarding the economic impact, the value of the project is mainly related to the potential cost associated with attacks with concealment and persistence capabilities. A compromise of this type may cause service interruptions, theft or leakage of information, costs derived from forensic analysis and system recovery, reputational damage and possible regulatory breaches. In this context, having a mechanism capable of providing an authenticated source of critical information allows earlier detection of signs of malicious presence or persistence, reducing the time required to diagnose the incident and consequently limiting part of the economic impact derived from a delayed response.

From a temporal point of view, the prototype may reduce the time required to obtain a first reliable technical evidence about the active processes of a compromised system,

since it generates an authenticated report on demand without depending only on *User-Space* tools. However, its configuration, loading, verification and later analysis still require the intervention of a technical profile, so the time saving mainly occurs in diagnostic or incident-response scenarios, not in automated use by non-specialized users.

The direct environmental impact of the project is limited, since the development has been restricted to a *software* prototype executed in a local testing environment (*on-premise*). Validation was performed on a real *Linux* system, using only the resources required to compile, load and unload the module, generate test processes and verify the resulting report. In addition, the design of the prototype avoids continuous or intensive monitoring of the system. The report is generated on demand when the administrator requests the transfer through the *File Handoff* mechanism, so the system does not need to remain continuously active to operate, consuming little energy. This would change, for example, if it were modified to monitor the system continuously or if it were deployed in the cloud, since the system's energy consumption and the need for operational resources, such as servers or support infrastructure, would increase.

Finally, from the point of view of ethical and professional responsibility, the project requires careful treatment of the balance between defensive security and potential misuse. Developing mechanisms close to the *kernel* implies operating in a critical area of the system, where implementation errors may affect machine stability and where poor permission or key management may reduce the expected protection, as well as increase the risk of vulnerabilities. For this reason, the design has been framed under an explicit threat model, with validations over input received from *User Space*, checks over the destination file, use of standard *kernel* mechanisms and a clear delimitation of what the prototype protects and what remains outside its scope.

6.3 Skills Acquired and Used

The completion of this *Bachelor's Degree Project* made it possible to integrate different skills from the *Degree in Engineering in Telecommunication Technologies*, especially those related to operating systems, low-level programming, cybersecurity, technical

analysis and documentation of engineering solutions. Although these skills were already presented in Chapter 4 as part of the project statement, this section provides a final assessment of how they were applied and reinforced during the development of the project.

Specifically, the most relevant skills applied and acquired during the project are the following:

- **Analysis of *Linux* systems and operating-system fundamentals.** The project required understanding the separation between *User Space* and *Kernel Space*, the role of processes, user, process and group identifiers, *namespaces*, the */proc* virtual file system and the mechanisms for loading and unloading modules in a privileged system region.
- **Systems programming in C.** The development of the *LKM* required working with *kernel* structures and functions, carefully managing memory, validating data coming from *User Space*, handling file descriptors through internal *kernel* mechanisms and writing in a controlled way to a file selected by the administrator.
- **Development in *Kernel Space*.** Specific skills were acquired in *kernel module* programming, creation of */proc* entries, communication between *User Space* and *Kernel Space*, writing from the *kernel* to user files and using both synchronization and safe-reading mechanisms (*RCU*) and specific *APIs* from the *kernel* source code, such as the *Crypto API* for handling and calculating *HMAC*. It should be noted that correctly using these functionalities required a prior technical analysis of the *kernel* source code to determine which mechanisms had to be used and how to apply them in each case.
- **Cybersecurity and threat analysis.** The work mainly allowed the application of knowledge about *User-Space rootkits*, manipulation of monitoring tools, definition of a threat model and design of a defensive solution adjusted to specific security assumptions.
- **Applied cryptography and information integrity.** The solution incorporates cryptographic authentication through *HMAC-SHA256*, calculated in the *kernel* and later verified from *User Space*. This reinforced the practical understanding of concepts such as integrity, authenticity, symmetric key, *Base64* encoding and the limits of protection based on a shared symmetric key.

- **Incremental design, validation and error resolution.** The construction by prototypes made it possible to first validate basic concepts, such as loading a module or communicating through a virtual file in `/proc`, before incorporating more delicate mechanisms such as *File Descriptor* handoff, *HMAC* calculation and generation of the running-process report.
- **Technical communication and documentation.** The report collects the context of the problem, the justification of the threat model, the progressive design of the solution, the problems encountered, the validations carried out and the limitations of each prototype up to the final solution, demonstrating the ability to document and present a technical solution in a coherent and verifiable way.

These skills were not applied in isolation, but in an integrated way throughout the development of the project.

At the same time, previous knowledge from the following subjects of the *Degree in Engineering in Telecommunication Technologies* was applied: *Operating Systems*, *Computer Network Security*, *Telecommunication Systems Programming*, *Telematic Systems*, *Telematic Systems Engineering* and *Information Systems Engineering*.

The final result not only materializes a functional implementation, but also reflects a process of autonomous technical learning, engineering reasoning, experimental validation and documentation adjusted to the scope of a final engineering project.

6.4 Future Work

Although the developed prototype fulfills the defined objectives and demonstrates the feasibility of the proposed mechanism, there are still several lines of work that would allow it to evolve towards a more complete, robust solution closer to a real operational environment. These improvements would be aimed both at expanding the protected critical information and at enabling the selection of the sensitive information to request from the system, as well as strengthening deployment, key management and integration

with monitoring infrastructures.

A first line of evolution would consist of improving the management of the shared symmetric key K , one of the most critical aspects. In the current prototype, the key is generated beforehand and embedded in the module and in the user-level utility in order to validate the operation of the *HMAC-SHA256* mechanism and allow its use by the administrator during verification. However, a more realistic solution should prevent the key from being statically incorporated into the code or binary of the *LKM*.

To this end, secure key-provisioning mechanisms, integration with protected stores, periodic rotation, use of hardware security modules (*HSM*) or support from technologies such as *TPM* could be studied, so that authentication would no longer depend on a fixed key without access protection.

Another important line would be to adapt the system to broader deployments, both *on-premise* and in distributed or cloud environments. The mechanism could evolve so that several nodes independently generate authenticated reports and send them to a central verification component, making it possible to build an aggregated view of the state of an operational infrastructure.

In this scenario, it would be necessary to define sending protocols, node identification, trust policies, secure report storage and temporal-correlation mechanisms, especially if the system is to be applied in architectures with containers, virtual machines or multiple *Linux* servers on a network.

It would also be possible to expand the type of critical information obtained from *Kernel Space* and allow it to be requested dynamically and selectively. In this work, the list of active processes has been used as the main case, but the same approach could be extended so that the administrator requests different types of information in each execution, such as network connections, loaded modules, open files, user information, changes in internal tables or process creation and termination events.

These extensions would make it possible to build a more complete and flexible reporting mechanism, although they would require defining a more expressive request interface and carefully studying collection cost, system stability and the amount of transferred information.

For practical application, it would also be advisable to improve verification automation. Currently, the administrator executes the checking utility on the transfer file generated

from *User Space*. A natural evolution would be to integrate this verification into an analysis tool that automatically compares the authenticated *kernel* report with the output of conventional *User-Space* tools, highlighting relevant discrepancies and generating alerts when possible signs of concealment or manipulation are detected.

Finally, the prototype would need to be hardened from an operational point of view. This would include expanding tests across different *kernel* versions, reviewing behavior under concurrent loads, improving error handling, incorporating stricter access controls over the `/proc` interface, auditing the code with specific tools and documenting secure procedures for installing, loading and unloading the module.

These lines would make it possible to move from a functional prototype towards a more mature tool, while always preserving the defined threat model and the limitations inherent in trusting the integrity of the *kernel*.

In conclusion, these ideas do not exhaust all possible evolutions of the project, but represent some future implementations that are especially relevant to increase its security, flexibility and applicability in real scenarios.

Starting from the developed prototype, new technical and operational improvements not included in this section could be explored, thereby extending the capabilities and features of the designed system towards more robust operation and a possible production deployment.

To conclude this project, from a personal perspective, I would like to finish by emphasizing that this *Bachelor's Degree Project* has been an intense journey of learning, deep research and demanding design, implementation and validation, starting from a truly complex problem and leading to a functional solution consistent with the objectives set.

Beyond the technical result obtained, this development period has allowed me to consolidate knowledge and acquire new skills while enjoying the process, working on a topic that genuinely interests me. Therefore, I would once again like to express my gratitude to my supervisor for his guidance and support throughout the entire process, to all the people close to me who have supported me not only during this project but throughout the degree, and finally to those who have devoted part of their time to reading this report.

I hope it has been of interest and may provide some value, either as knowledge or as a starting point or support for future projects.

Thank you very much for everything.

Appendix A

Code and Validation of the Prototypes

This appendix explains in detail the code used in the implementation and validation of the prototypes developed throughout the project. To this end, it includes the most relevant code fragments, the auxiliary scripts used during development and the outputs obtained in the validation tests performed.

The complete source code of the project is also available in the public repository <https://github.com/nowelito28/TFG>.

A.1 Prototypes, Implementations and Validations

The main development of the project was structured through several *LKM* prototypes built progressively and based on the functionality validated in the previous ones. This strategy made it possible to move from a first contact with programming in *Kernel Space* to the construction of the final prototype, capable of generating an authenticated list of the active system processes from that privileged region.

A.1.1 *Hello World*

The first development step consisted of building a basic *LKM* whose objective was to verify how the basic workflow associated with *kernel modules* operates.

For this purpose, the external compilation file *Makefile*, based on *kbuild*, was initially designed. It delegates the compilation process to the build system of the *kernel* itself and is reused to compile later prototypes. This file defines the variable `obj-m`, which tells *kbuild* which object file (`.o`) must be compiled as a loadable module

(.ko). On the other hand, two main rules are declared to structure the compilation flow. The first one, `all`, invokes the build system of the active *kernel* through the path `/lib/modules/$(shell uname -r)/build`, and specifies the path of the module source code to compile in the current directory with `M=$(shell pwd)`. Therefore, in order to compile, the *Makefile* must be located in the same directory as the module source code and compilation must be executed from that directory. The second rule, `clean`, removes the intermediate files generated during compilation. Finally, these rules are declared as *phony targets* to avoid conflicts with files of the same name. In this way, the module can be compiled externally without needing to modify or recompile the system core.

```
obj-m := module_name.o

all: aux.sh
    make -C /lib/modules/$(shell uname -r)/build M=$(shell pwd) modules

clean:
    make -C /lib/modules/$(shell uname -r)/build M=$(shell pwd) clean

.PHONY: all clean
```

Code A.1: Generic *Makefile* used to compile *LKMs*.

If it is necessary to provide a dependency before compilation, such as an auxiliary file, it can be declared in the `all` rule, as shown in the previous code with the file `aux.sh`. In this initial prototype it was not necessary to use this feature, so the dependency shown should only be understood as a general example of how to declare it if needed.

Once the compilation tool based on *Makefile* and *kbuild* had been defined, a simple module, `hello_world.c`, was implemented in order to validate the basic life cycle of an *LKM*: loading the module into the *kernel*, executing the module load and unload control functions, registering messages in the *kernel* log and later unloading the module.

First, the headers required to compile the module were included and the basic metadata of the component were declared:

```
#include <linux/module.h>
#include <linux/kernel.h>
#include <linux/init.h>
```

```
MODULE_LICENSE("Dual BSD/GPL");
MODULE_AUTHOR("Noel Rodriguez Perez");
MODULE_DESCRIPTION("A simple Hello world LKM!");
MODULE_VERSION("0.1");
```

Code A.2: Headers and metadata of the *Hello World* module.

This fragment includes the headers required to program a *Linux LKM* and, on the other hand, defines several descriptive metadata fields for the component, later visible through the `modinfo` tool. Among them, the license is especially important because it conditions the behavior of the module inside the *kernel* and determines whether it is marked as *tainted* when loaded, meaning that it is not considered completely clean for debugging and support purposes. In this case, a dual *BSD/GPL* license is chosen. On the one hand, compatibility with *GPL*, used by the *Linux kernel* itself, prevents the module from being marked as *tainted*. On the other hand, the permissive *BSD* license allows the module to be used in proprietary environments without having to share the source code.

Next, the module needs to implement its initialization and exit functions:

```
static int __init hello_start(void)
{
    printk(KERN_INFO "Loading hello world module...\n");
    printk(KERN_INFO "Hello world!!\n");
    return 0;
}

static void __exit hello_end(void)
{
    printk(KERN_INFO "Goodbye Mr.\n");
}
```

Code A.3: Initialization/load and exit/unload functions of the *Hello World* module.

The function `hello_start` is executed when the module is loaded into the *kernel* with `insmod <module_name>.ko`. In this prototype, its only purpose is to write messages to the *kernel log buffer* using the function `printk`. These messages can be consulted directly through `dmesg` and, depending on the configuration of the logging system, may also appear in *kernel log files* such as `/var/log/kern.log`. Similarly, the function `hello_end` is executed when the module is unloaded with `rmmod <module_name>`.

The use of `printk` was verified in the source code of the *kernel* message log, exposed

in `include/linux/printk.h`, where the function itself and the different priority levels that can be associated with each trace are defined, such as `KERN_INFO`, `KERN_DEBUG` or `KERN_ERR`.

Finally, both handlers must be explicitly associated with the module control flow:

```
module_init(hello_start);
module_exit(hello_end);
```

Code A.4: Association of the module entry/load and exit/unload points.

These macros indicate to the *kernel* which function must be executed during module loading and which one must be invoked during unloading. With this, the minimum functional structure of an *LKM* is defined and its correct operation can be validated.

With administrator privileges (*root*), the module was compiled, loaded into the *kernel*, the traces registered in the *kernel* log were inspected, the module metadata were consulted and, finally, it was unloaded while also verifying execution of the exit routine:

```
$ make
make -C /lib/modules/6.12.41-current-bcm2711/build M=/home/LKM modules
make[1]: Entering directory '/usr/src/linux-headers-6.12.41-current-bcm2711'
  CC [M] /home/LKM/hello_world.o
  MODPOST /home/LKM/Module.symvers
  CC [M] /home/LKM/hello_world.mod.o
  LD [M] /home/LKM/hello_world.ko
make[1]: Leaving directory '/usr/src/linux-headers-6.12.41-current-bcm2711'
$ insmod hello_world.ko
$ dmesg -T | tail
...
[Fri Sep 5 19:45:58 2025] Loading hello world module...
[Fri Sep 5 19:45:58 2025] Hello world!!
$ modinfo hello_world.ko
filename:      /home/LKM/hello_world.ko
version:       0.1
description:    A simple Hello world LKM!
author:        Noel Rodriguez Perez
license:       Dual BSD/GPL
$ rmmod hello_world
$ dmesg -T | tail
...
[Fri Sep 5 19:46:30 2025] Goodbye Mr.
```

Code A.5: Compilation, loading, inspection and unloading of the *Hello World* prototype.

Through this workflow, it was verified that the module could be correctly compiled as an external *LKM*, loaded without errors into the *kernel* while executing its initialization routine, with its metadata correctly associated, execute its exit routine when unloaded and properly register messages through `printk` during both phases.

A.1.2 Virtual File `/proc/fdev`

The next step consisted of providing the module with a communication interface between *User Space* and *Kernel Space*, implemented in `proc_fdev.c`. For this purpose, the `/proc` virtual file system was used to expose *kernel* information and allow controlled interaction with the *LKM* by creating a custom virtual file in this system, `/proc/fdev`.

The main objective of this phase was to verify that the module could create its own entry in `/proc`, associate read and write handler functions with it and respond in a controlled way to these operations invoked from *User Space*.

First, it was necessary to extend the set of headers included in the module in order to work with `/proc` and with data transfers between *User Space* and *Kernel Space*:

```
#include <linux/proc_fs.h>
#include <asm/uaccess.h>
```

Code A.6: Additional headers for the prototype based on `/proc`.

The header `<linux/proc_fs.h>` is incorporated, required to create and manage entries inside the `/proc` virtual file system, and, on the other hand, `<asm/uaccess.h>`, which imports functionality such as `copy_to_user` and `copy_from_user`, used to transfer information in a controlled and safe way between *kernel* memory and *User-Space* memory.

Next, it was necessary to define a reference to the virtual entry and the operations table associated with that entry in `/proc`:

```
static struct proc_dir_entry *ent;
```

```

static struct file_operations myops =
{
    .owner = THIS_MODULE,
    .read = myread,
    .write = mywrite,
};

static int simple_init(void)
{
    ent = proc_create("fdev", 0660, NULL, &myops);
    if (!ent)
        return -ENOMEM;
    printk(KERN_INFO "/proc/fdev created\n");
    return 0;
}

static void simple_cleanup(void)
{
    proc_remove(ent);
    printk(KERN_INFO "/proc/fdev removed\n");
}

module_init(simple_init);
module_exit(simple_cleanup);

```

Code A.7: Reference to the virtual entry and associated operations table in `/proc`.

In this fragment, the pointer `ent` stores the reference to the entry created in `/proc` when the module is loaded, later allowing it to be managed and removed when the module is unloaded. On the other hand, the structure `myops` defines the operations table of the virtual file to be created, associating read and write actions on that file with the handler functions `myread` and `mywrite`, respectively. In addition, the field `.owner = THIS_MODULE` links these operations to the loaded module itself. Finally, `simple_init` creates the virtual entry `/proc/fdev` through `proc_create` during module loading, returning `-ENOMEM` (*no memory error*) if the entry cannot be created, while `simple_cleanup` removes it with `proc_remove` during unloading.

The use of `struct proc_dir_entry`, `proc_create` and `proc_remove` was checked against the source code of the `/proc` file system of the *kernel*, defined in `include/linux/proc_fs.h` and developed in the `fs/proc` subsystem, while `struct file_operations` is implemented in the general file-operations interface exposed in `include/linux/fs.h`.

One aspect worth highlighting in the development of *kernel module* prototypes is the convenience of using *static* to declare variables and functions that are not going to be used outside the module itself, restricting their visibility to the module source code and favoring better code encapsulation. This criterion is maintained throughout the rest of the project.

Once the virtual file entry `/proc/fdev` had been created and its operations table associated, the write and read handlers of that file were implemented:

```
enum { BUFSIZE = 100 };

static int irq = 20;
static int mode = 1;

static ssize_t mywrite(struct file *file, const char __user *ubuf,
                      size_t count, loff_t *ppos)
{
    int num, c, i, m;
    char buf[BUFSIZE];

    if (*ppos > 0 || count >= BUFSIZE)
        return -EFAULT;

    printk(KERN_DEBUG "/proc/fdev: write handler\n");

    if (copy_from_user(buf, ubuf, count))
        return -EFAULT;

    num = sscanf(buf, "%d %d", &i, &m);
    if (num != 2)
        return -EFAULT;

    irq = i;
    mode = m;

    c = strlen(buf);
    *ppos = c;

    printk(KERN_DEBUG "/proc/fdev: %d bytes received from userspace\n", c);
    return c;
}

static ssize_t myread(struct file *file, char __user *ubuf,
                     size_t count, loff_t *ppos)
{
    char buf[BUFSIZE];
    int len = 0;
```

```

    if (*ppos > 0 || count < BUFSIZE)
        return 0;

    printk(KERN_DEBUG "/proc/fdev: read handler\n");

    len += sprintf(buf, "irq = %d\n", irq);
    len += sprintf(buf + len, "mode = %d\n", mode);

    if (copy_to_user(ubuf, buf, len))
        return -EFAULT;

    *ppos = len;

    printk(KERN_DEBUG "/proc/fdev: %d bytes sent to userspace\n", len);
    return len;
}

```

Code A.8: *Handlers* with data transfer between *User Space* and *Kernel Space*.

The write handler, `mywrite`, receives data from *User Space* through `copy_from_user`, moving `count` bytes from the user-memory *buffer* (`ubuf`) to a *buffer* in *kernel* memory (`buf`). Similarly, `myread` returns data to *User Space* through `copy_to_user`, moving `len` bytes from the *buffer* in *kernel* memory (`buf`) to the user-memory *buffer* (`ubuf`). In addition, a *single-shot* access semantics was imposed through the file-position pointer (`*ppos`).

The use of `copy_from_user` and `copy_to_user` was checked in the `uaccess` infrastructure of the *kernel* source code, specifically in `include/linux/uaccess.h`, where both functions rely on the internal primitives `raw_copy_from_user` and `raw_copy_to_user`.

In this example, the values `irq` and `mode` are used as integer test variables to validate the exchange of information between *User Space* and *Kernel Space*. This makes it possible to check their correct interpretation from the data received through `sscanf`, as well as their output formatting through `sprintf`, returning errors such as `-EFAULT` (*bad address*) when the restrictions are not satisfied or when the input format is not the expected one. On the other hand, an enumerated constant `BUFSIZE` is defined to simply set the maximum size of the temporary *buffer* used, in this case 100 bytes, both for reading and writing.

Once the virtual file `/proc/fdev` and its read and write handlers had been created, the

correct operation of the prototype was checked, assuming administrator permissions (root) on the system:

```
$ make
make -C /lib/modules/6.12.41-current-bcm2711/build M=/home/LKM modules
make[1]: Entering directory '/usr/src/linux-headers-6.12.41-current-bcm2711'
  CC [M] /home/LKM/proc_fdev.o
  MODPOST /home/LKM/Module.symvers
  CC [M] /home/LKM/proc_fdev.mod.o
  LD [M] /home/LKM/proc_fdev.ko
make[1]: Leaving directory '/usr/src/linux-headers-6.12.41-current-bcm2711'
$ insmod proc_fdev.ko
$ dmesg -T | tail
...
[Tue Oct 14 17:25:58 2025] /proc/fdev created
$ ls -l /proc/fdev
-rw-rw---- 1 root root 0 Oct 14 17:25 /proc/fdev
$ echo "7 3" > /proc/fdev
$ dmesg -T | tail
...
[Tue Oct 14 17:26:10 2025] /proc/fdev: write handler
[Tue Oct 14 17:26:10 2025] /proc/fdev: 4 bytes received from userspace
$ cat /proc/fdev
irq = 7
mode = 3
$ dmesg -T | tail
...
[Tue Oct 14 17:26:18 2025] /proc/fdev: read handler
[Tue Oct 14 17:26:18 2025] /proc/fdev: 17 bytes sent to userspace

$ rmmod proc_fdev
$ dmesg -T | tail
...
[Tue Oct 14 17:26:30 2025] /proc/fdev removed
```

Code A.9: Basic validation of the prototype based on `/proc/fdev`.

This validation confirmed that all module functionality executed correctly, both the creation of the virtual entry `/proc/fdev` and its custom read and write operations, ensuring a basic communication path between *User Space* and *Kernel Space*.

A.1.3 *File Handoff*

Once a valid entry in `/proc` had been created and the basic exchange of information between *User Space* and *Kernel Space* had been achieved in a controlled way, the next step consisted of implementing a mechanism that would allow the module to operate on a real file in the file system, materialized in `fd_handoff.c`. To do this, the transfer of a file reference (*File Descriptor*) from *User Space* to the *LKM* was proposed, using as input interface the same virtual file `/proc/fdev` developed in the previous prototype.

The general objective of this prototype was to verify that the module could receive from *User Space* the *FD* of a file encoded as a string of *bytes* and convert it to an *int* value, internally resolve that reference to locate the corresponding file in the *kernel* and use it to transfer information from that privileged region to a system file. With this, the administrator can decide in a controlled way which file receives the requested information.

It was necessary to extend the module headers again for this development:

```
#include <linux/file.h>
#include <linux/fs.h>
#include <linux/kstrtox.h>
#include <linux/security.h>
```

Code A.10: Additional headers for the *File Handoff* mechanism.

The header `<linux/file.h>` allows working with internal *kernel* file references, such as the data structure `struct file`, as well as resolving an *FD* through `fget`. On the other hand, `<linux/fs.h>` provides general definitions from the file subsystem, including opening modes and *flags*. The header `<linux/kstrtox.h>` is also added, providing safe conversion functions, such as `kstrtoint`, to transform a string of *bytes* into an integer. Finally, `<linux/security.h>` allows permissions on referenced files to be validated through *kernel* access-control mechanisms, such as `file_permission`.

Regarding the read handler, it was simplified with respect to the previous prototype, limiting itself to returning a deterministic information string indicating to the administrator that the module is ready to receive a *File Descriptor* (*FD*) and perform the *File Handoff*:

```

static ssize_t myread(struct file *file, char __user *ubuf,
                      size_t count, loff_t *ppos)
{
    char buf[] = "Waiting for a FD to perform file handoff\n";
    int len = strlen(buf);

    if (*ppos > 0 || count < len)
        return 0;
    printk(KERN_DEBUG "/proc/fdev: read handler\n");

    if (copy_to_user(ubuf, buf, len))
        return -EFAULT;

    *ppos = len;
    printk(KERN_DEBUG "/proc/fdev: %d bytes sent to userspace\n", len);
    return len;
}

```

Code A.11: Read function for the *File Handoff* mechanism.

However, the main functionality of this prototype is concentrated in the write handler, which requires auxiliary functions to organize the write logic in a clearer and more modular way:

```

static int write_full(struct file *f, const char *buf, int len)
{
    int w = 0;
    loff_t *ppos = &f->f_pos;

    w = kernel_write(f, buf, len, ppos);
    if (w < 0)
        return w;

    if (w != len)
        return -EIO;

    return w;
}

```

Code A.12: Auxiliary function for writing to a file from the *LKM*.

The auxiliary function `write_full` encapsulates writing to an already open file from *Kernel Space* through a single call to `kernel_write`. For this purpose, it uses the pointer associated with the file (`f_pos`) as the write position, checks whether the call returns an error and verifies that the number of written *bytes* (`w`) matches the requested

length (`len`). Otherwise, an error code (`-EIO`) is returned, keeping the write check centralized inside the *LKM*.

The use of `kernel_write` was supported by verifying its implementation inside the *kernel VFS* source code, specifically in the file `fs/read.write.c`.

```
static int val_metadata(struct file *f)
{
    int rv = 0;

    if (!(f->f_mode & FMODE_WRITE) || !(f->f_flags & O_APPEND))
        return -EBADF;

    rv = file_permission(f, MAY_WRITE | MAY_APPEND);
    if (rv < 0)
        return rv;

    return 0;
}
```

Code A.13: Auxiliary function for validating file permissions from the *LKM*.

In this case, the auxiliary function `val_metadata` encapsulates the prior validation logic for the file referenced from *User Space*, checking that it can be used safely in *append* mode from *Kernel Space*. First, it verifies that it was opened with write capability (`FMODE_WRITE`) and with the `O_APPEND` flag, avoiding accesses incompatible with the policy defined for this prototype. Then, its effective permissions are checked through `file_permission`, ensuring that the *kernel* can perform write and append operations on it through the permissions `MAY_WRITE` and `MAY_APPEND`. If any of these restrictions is not satisfied, the function returns the corresponding error code, returning `-EBADF` (*bad file error*) when the opening metadata are not valid.

In this part, the *kernel* source code also had to be consulted. The use of `file_permission` was supported by its definition in `include/linux/fs.h`, while the relationship between `O_APPEND` and `MAY_APPEND` was checked in `fs/open.c`. In addition, in `fs/namei.c`, inside the function `inode_permission`, it is explicitly stated that every check with `MAY_APPEND` must also include `MAY_WRITE`.

```
static ssize_t mywrite(struct file *file, const char __user *ubuf,
                      size_t count, loff_t *ppos)
```

```

{
    int fd = 0;
    int rv = 0;
    char buf[BUFSIZE];
    static const char msg[] = "Kernel-Space: Writing test for file
        handoff\n";
    struct file *f = NULL;

    if (*ppos > 0 || count > BUFSIZE)
        return -EFAULT;

    printk(KERN_DEBUG "/proc/fdev: write handler\n");

    if (copy_from_user(buf, ubuf, count))
        return -EFAULT;

    rv = strlen(buf);
    *ppos = rv;

    if (kstrtoint(buf, 10, &fd))
        return -EINVAL;

    if (fd < 0)
        return -EBADF;

    f = fget(fd);
    if (!f)
        return -EBADF;

    rv = val_metadata(f);
    if (rv < 0)
        goto out_put;

    rv = write_full(f, msg, strlen(msg));
    if (rv < 0)
        goto out_put;

    printk(KERN_DEBUG "/proc/fdev: %d bytes written to handoff file\n", rv);

out_put:
    fput(f);
    return rv;
}

```

Code A.14: Write function for the *File Handoff* mechanism.

Finally, the handler function `mywrite` concentrates the main logic of the *File Handoff* mechanism, receiving from *User Space* the file descriptor encoded as a string of *bytes* through `/proc/fdev`. First, as in the previous prototype, it checks that the *single-shot*

access semantics is satisfied and that the maximum size set by `BUFSIZE` is not exceeded, then transferring the information to *kernel* memory through `copy_from_user`.

Next, the received string of *bytes* is converted to a decimal integer value with `kstrtoint`, obtaining the corresponding *FD*. In this way, using `fget` and the obtained *FD*, the reference to the real file inside the *kernel* is recovered; its permissions are then verified with `val_metadata` to check that it complies with the established policy, and a static test string is written to it through `write_full`.

Finally, the file reference is released with `fput`, returning the corresponding error code in each case if any check fails, with the help of the `goto` label (`out_put`).

The use of `kstrtoint`, `fget` and `fput` was checked against the *kernel* source code, locating `kstrtoint` in the file `lib/kstrttox.c`, while `fget` and `fput` are found in `fs/file.c` and `fs/file_table.c`, with the exact lines varying depending on the *kernel* version considered.

To complete this prototype, it was also necessary to implement an additional utility in *User Space*, `mk_fhandoff.c`:

```
#include <stdlib.h>
#include <stdio.h>
#include <string.h>
#include <err.h>
#include <errno.h>
#include <fcntl.h>
#include <unistd.h>

static int write_full(int fd, const char *buf, int len)
{
    int w = 0;

    w = write(fd, buf, len);
    if (w < 0)
        return -1;

    if (w != len) {
        errno = EIO;
        return -errno;
    }

    return w;
}
```

```

int main(int argc, char *argv[])
{
    if (argc != 3) {
        errx(EXIT_FAILURE, "Usage: %s <common_path> <proc_path>\n"
            "e.g.: %s ./file_handoff /proc/fdev", argv[0], argv[0]);
    }

    const char *f_handoff = argv[1];
    const char *f_proc = argv[2];

    int fd_handoff = open(f_handoff, O_CREAT | O_RDWR | O_APPEND, 0666);

    if (fd_handoff < 0) {
        err(EXIT_FAILURE, "Error opening/creating the file %s",
            f_handoff);
    }

    int fd_proc = open(f_proc, O_RDWR);
    if (fd_proc < 0) {
        err(EXIT_FAILURE, "Error opening %s", f_proc);
    }

    char fd_str[12];
    snprintf(fd_str, sizeof(fd_str), "%d", fd_handoff);

    if (write_full(fd_proc, fd_str, strlen(fd_str)) <= 0) {
        close(fd_proc);
        close(fd_handoff);

        err(EXIT_FAILURE, "Error writing in %s", f_proc);
    }

    printf("File descriptor written in %s:\n%d\n", f_proc, fd_handoff);

    close(fd_proc);
    close(fd_handoff);

    exit(EXIT_SUCCESS);
}

```

Code A.15: *User-Space* program `mk_fhandoff.c` to activate *File Handoff*.

This program acts as an intermediary from *User Space*, creating or opening the real file that will act as the destination from the name indicated as the first argument, obtaining its *FD* and sending it to the module through `/proc/fdev`, whose path is

indicated as the second argument when executing the program, thus completing the *File Handoff* mechanism. It should be noted that this user utility also implements an auxiliary write function (`write_full`), responsible for encapsulating the call to `write` and checking that the operation does not fail and that the number of written *bytes* matches the requested size.

Once the complete basic *File Handoff* mechanism had been finished, its operation was validated by checking that the module correctly received the *FD* of a real file from *User Space*, resolved it inside the *kernel* and wrote to that file the string defined in `mywrite`:

```
$ uname -r
6.12.41-current-bcm2711
$ cd LKM
$ make
make -C /lib/modules/6.12.41-current-bcm2711/build M=/home/LKM modules
make[1]: Entering directory '/usr/src/linux-headers-6.12.41-current-bcm2711'
  CC [M] /home/LKM/fd_handoff.o
  MODPOST /home/LKM/Module.symvers
  CC [M] /home/LKM/fd_handoff.mod.o
  LD [M] /home/LKM/fd_handoff.ko
make[1]: Leaving directory '/usr/src/linux-headers-6.12.41-current-bcm2711'
$ insmod fd_handoff.ko
$ dmesg -T | tail
...
[Wed Oct 29 16:25:15 2025] /proc/fdev created
$ cd ..
$ gcc -c -Wshadow -Wvla -Wall mk_fhandoff.c
$ gcc -o mk_fhandoff mk_fhandoff.o
$ cat /proc/fdev
Waiting for a FD to perform file handoff
$ dmesg -T | tail
...
[Wed Oct 29 16:28:27 2025] /proc/fdev: read handler
[Wed Oct 29 16:28:27 2025] /proc/fdev: 41 bytes sent to userspace
$ ./mk_fhandoff ./file_handoff /proc/fdev
File descriptor written in /proc/fdev:
3
$ dmesg -T | tail
...
[Wed Oct 29 16:30:11 2025] /proc/fdev: write handler
[Wed Oct 29 16:30:11 2025] /proc/fdev: 1 bytes written to handoff file
$ cat ./file_handoff
Kernel-Space: Writing test for file handoff

$ cd LKM
$ rmmod fd_handoff
```

```
$ dmesg -T | tail
...
[Wed Oct 29 16:32:20 2025] /proc/fdev removed
```

Code A.16: Validation of the *File Handoff* mechanism.

Through this validation, it was confirmed that the module could correctly receive the *FD* of a real file from *User Space*, resolve its reference in the *kernel* and use it as the effective write destination through the *File Handoff* mechanism.

As a consequence, a functional channel was implemented to transfer information generated in *Kernel Space* in a controlled way to the file indicated by *User Space*, serving as the basis for the later incorporation of authenticated critical content in the secure report.

A.1.4 *HMAC-SHA256* Cryptographic Authentication Mechanism

Once the *File Handoff* process had been validated, the next step consisted of incorporating into the module `crypto_fd.c` a cryptographic authentication mechanism for the content generated in *Kernel Space* (*HMAC-SHA256*) and including that *HMAC* tag in the final transfer file, in order to guarantee its integrity and authenticity by relying on the previously validated infrastructure. In this way, the content can later be verified from *User Space* using the same shared secret key.

For this purpose, it was necessary to extend the headers required to compile the module and incorporate the embedded symmetric key with which the *HMAC* is calculated, which must also be available to the administrator during later verification.

```
#include <crypto/hash.h>
#include <linux/base64.h>
#include <linux/crypto.h>

#include "k_embedded.h"
```

Code A.17: Additional headers and embedded key for the *HMAC-SHA256* authentication mechanism.

At this point, `<crypto/hash.h>` and `<linux/crypto.h>` allow the use of *kernel Crypto API* functionality to request a synchronous `shash` instance, while `<linux/base64.h>` allows the binary result of the *HMAC* to be encoded into a portable textual representation in *Base64* format. On the other hand, including `"k_embedded.h"` introduces an externally generated binary symmetric secret key embedded in the module as a constant vector of *bytes*, so that it can be used directly during the *HMAC* calculation.

Consequently, the module compilation process no longer depends only on the main file, but first requires the header containing the embedded key. For this purpose, an auxiliary script `gen_k_embedded.sh` was designed. It generates a random 64-byte key, stores it in a binary file `key.bin` and then transforms it into the header `k_embedded.h` as a constant vector of *bytes*, making it importable by the *LKM*. At the same time, the *Makefile* used to compile the *LKM* declares `k_embedded.h` as a prerequisite of the `all` rule.

```
#!/bin/sh
set -e

KEY_LEN=64
openssl rand -out key.bin ${KEY_LEN}

xxd -i -n K key.bin | \
    sed -e 's/^unsigned char/const unsigned char/' \
        -e 's/^unsigned int/const unsigned int/' > k_embedded.h

echo "OK: generated k_embedded.h with key K (${KEY_LEN} bytes)."
```

Code A.18: `gen_k_embedded.sh` script to generate the `k_embedded.h` header with the embedded key.

It should be noted that, in order to use this script once created, it was necessary to assign it execution permissions externally with `chmod +x gen_k_embedded.sh`, allowing it to be executed afterwards with `./gen_k_embedded.sh` in the same working directory.

Regarding the programming of the content-encoding logic to obtain the *HMAC* in the *LKM*, this functionality was implemented to calculate *HMAC-SHA256* from a data block treated as an *array* of *bytes*:

```

static int get_hmac_sha256(const u8 *buf, int buf_len, u8 **hmac, int
    *hmac_len)
{
    int rv = 0;
    struct crypto_shash *tfm;

    tfm = crypto_alloc_shash("hmac(sha256)", 0, 0);

    if (IS_ERR(tfm))
        return PTR_ERR(tfm);

    rv = crypto_shash_setkey(tfm, K, K_len);
    if (rv)
        goto out_free_tfm;

    *hmac_len = crypto_shash_digestsize(tfm);

    *hmac = kmalloc(*hmac_len, GFP_KERNEL);
    if (!*hmac) {
        rv = -ENOMEM;
        goto out_free_tfm;
    }

    SHASH_DESC_ON_STACK(desc, tfm);
    desc->tfm = tfm;

    rv = crypto_shash_digest(desc, buf, buf_len, *hmac);

out_free_tfm:
    crypto_free_shash(tfm);
    return rv;
}

```

Code A.19: Calculation of *HMAC-SHA256* for a data *byte* string.

This function encapsulates the main logic for calculating the *HMAC-SHA256* cryptographic authenticator over an arbitrary data block represented as an *array* of *bytes* in *kernel* memory.

To do this, it requests from the *Crypto API* a synchronous *hash* instance (*shash*) associated with the algorithm `hmac(sha256)` through `crypto_alloc_shash`. Then, the embedded secret key is associated with it through `crypto_shash_setkey` and the *digest* size is obtained with `crypto_shash_digestsize`. Afterwards, dynamic memory is allocated with `kmalloc` to store the *HMAC* to be calculated with the corresponding length (`hmac_len`). In addition, the macro `SHASH_DESC_ON_STACK` builds on the

execution stack the temporary descriptor required by the cryptographic subsystem (`desc`). After that, `crypto_shash_digest` calculates the complete *HMAC* in a single operation over the received content.

Once the raw *HMAC* of the content had been calculated, an auxiliary function was implemented to convert it to *Base64* format so that it could be included in the transfer file in a readable and portable way:

```
static int get_hmac_b64(const u8 *hmac, int hmac_len, u8 **hmac_b64,
                       int *hmac_b64len)
{
    int hmac_b64cap = BASE64_CHARS(hmac_len);

    *hmac_b64 = kmalloc(hmac_b64cap, GFP_KERNEL);

    if (!*hmac_b64)
        return -ENOMEM;

    *hmac_b64len = base64_encode(hmac, hmac_len, *hmac_b64);
    if (*hmac_b64len < 0) {
        kfree(*hmac_b64);
        return *hmac_b64len;
    }

    return 0;
}
```

Code A.20: Conversion of the binary *HMAC* to *Base64*.

This auxiliary function facilitates serialization of the cryptographic authenticator in the transfer file, transforming the previously calculated binary *HMAC* into a textual *Base64* representation.

First, it determines the maximum capacity required for the encoded string through the macro `BASE64_CHARS` and dynamically allocates memory to store it. Then, the function `base64_encode` performs the effective conversion of that raw *HMAC*, returning the resulting string in `hmac_b64` and its final length in `hmac_b64len`. In this way, the authenticator can be incorporated into the output file in a readable, portable format that can later be easily verified from *User Space*.

The operation and implementation of `crypto_alloc_shash`, `crypto_shash_setkey`, `crypto_shash_digestsize`, `crypto_shash_digest` and the macro

SHASH_DESC_ON_STACK were checked in the synchronous *hash* infrastructure of the *kernel Crypto API*, exposed in `include/crypto/hash.h` and developed in the `crypto` subsystem within the source code of the core itself. On the other hand, encoding the authenticator in *Base64* relied on `include/linux/base64.h` and on the corresponding implementation in `lib/base64.c`.

At output-format level, the content is no longer written in isolation to the transfer file, but accompanied by an explicit separation between the data generated by the *kernel* and its *HMAC* authentication tag in *Base64* format:

```
static const char sep[] = "\n--KERNEL-PS--\n";
static const int sep_len = sizeof(sep) - 1;
static const char sep_hmac[] = "\n--HMAC--\n";
static const int sep_hmac_len = sizeof(sep_hmac) - 1;

static int write_cont_hmac(struct file *f, const char *cont, int cont_len,
                          const char *hmac_b64, int hmac_b64len)
{
    int w = 0;
    int total = 0;

    w = write_full(f, sep, sep_len);
    if (w < 0)
        return w;
    total += w;

    w = write_full(f, cont, cont_len);
    if (w < 0)
        return w;
    total += w;

    w = write_full(f, sep_hmac, sep_hmac_len);
    if (w < 0)
        return w;
    total += w;

    w = write_full(f, hmac_b64, hmac_b64len);
    if (w < 0)
        return w;
    total += w;

    return total;
}
```

Code A.21: Joint writing of the content and its *HMAC* in *Base64*.

This function organizes the final serialization of the authenticated content inside the

transfer file. To do this, it sequentially writes distinguishable textual separators, starting with an initial separator before the block generated by the *kernel* (`sep`), the content to be protected, the separator `sep_hmac` and, finally, the *HMAC* already encoded in *Base64*, reusing the `write_full` function validated in the previous prototype.

In this way, the resulting file is structured deterministically, facilitating the separation between the main content to be transferred and the *HMAC* required for later validation from *User Space*.

Once the development of the authentication mechanism had been explained, the write handler was modified to integrate all unified components:

```
static int printh(struct file *f)
{
    int rv = 0;

    static const char cont[] =
        "Kernel-Space: Authentic content protected by HMAC-SHA256\n";
    const int cont_len = sizeof(cont) - 1;

    u8 *hmac = NULL;
    int hmac_len = 0;

    u8 *hmac_b64 = NULL;
    int hmac_b64len = 0;

    rv = get_hmac_sha256(cont, cont_len, &hmac, &hmac_len);
    if (rv < 0)
        goto out;

    rv = get_hmac_b64(hmac, hmac_len, &hmac_b64, &hmac_b64len);
    if (rv < 0)
        goto out_free_hmac;

    rv = write_cont_hmac(f, cont, cont_len, hmac_b64, hmac_b64len);
    if (rv < 0)
        goto out_free_hmacs;

    printk(KERN_INFO "printh: file has been written with HMAC\n");

out_free_hmacs:
    kfree(hmac_b64);
```

```

out_free_hmac:
    kfree(hmac);

out:
    return rv;
}

```

Code A.22: Integration of the authentication mechanism.

Now, the function `printh` concentrates the main integration logic of the mechanism designed in this prototype. In this test case, a constant content string `cont` is used; its *HMAC-SHA256* is calculated, encoded in *Base64* and everything is written in a well-structured way to the transfer file.

```

static ssize_t mywrite(struct file *file, const char __user *ubuf,
                      size_t count, loff_t *ppos)
{
    int fd = 0;
    int rv = 0;
    char buf[BUFSIZE];

    if (*ppos > 0 || count > BUFSIZE)
        return -EFAULT;
    printk(KERN_DEBUG "/proc/fdev: write handler\n");

    if (copy_from_user(buf, ubuf, count))
        return -EFAULT;

    rv = strlen(buf);
    *ppos = rv;

    if (kstrtoint(buf, 10, &fd))
        return -EINVAL;
    if (fd < 0)
        return -EBADF;

    struct file *f = fget(fd);
    if (!f)
        return -EBADF;

    rv = val_metadata(f);
    if (rv < 0)
        goto out_put;
}

```

```

    rv = printh(f);
    if (rv < 0)
        goto out_put;
    printk(KERN_DEBUG "mywrite: printh OK for fd %d (%d bytes written)\n",
           fd, rv);

out_put:
    fput(f);

    return rv;
}

```

Code A.23: Write handler adapted to the content-authentication mechanism.

For its part, `mywrite` keeps the general logic of the previous prototype for receiving and resolving the *FD*, but replaces the simple static write with a call to `printh`.

Once the authentication mechanism had been incorporated into the *File Handoff* flow in the *LKM*, it was necessary to develop an additional utility in *User Space* to verify the validity of the generated authenticated content, intended to be used with administrator permissions. For this purpose, the program `check_hmac.c` was implemented. It deserializes the content of the transfer file and separates the main content from the *HMAC* tag in *Base64* format, making it possible to recalculate the authenticator with the same shared secret key and compare both results to ensure content integrity.

First, the headers and constants required to compile this program were added, including the embedded key shared between the *kernel* and the administrator for calculating the *HMAC*:

```

#include <stdlib.h>
#include <stdio.h>
#include <string.h>
#include <err.h>
#include <errno.h>
#include <fcntl.h>
#include <unistd.h>
#include <openssl/hmac.h>
#include <openssl/evp.h>
#include <openssl/crypto.h>

#include "../LKM/k_embedded.h"

enum { LEN = 1024 * 40 };

```

```
const char sep[] = "--KERNEL-PS--\n";
const char sep_hmac[] = "--HMAC--\n";
```

Code A.24: Headers, constants and symmetric key required for verification.

First, the functions `read_until_separator`, `read_hmac_line` and `extract_data` are responsible for deserializing the content of the transfer file generated through the *File Handoff* mechanism, separating the main content from the *HMAC* tag stored in *Base64* format by detecting the defined separators:

```
int read_until_separator(FILE *f, char **content, int *content_len)
{
    char *line = NULL;
    size_t cap = 0;
    int len = 0;

    int sep_found = 0;
    int sep_hmac_found = 0;

    *content = (char *)malloc(LEN);
    if (!(*content)) {
        warnx("malloc failed for the content\n");

        return 1;
    }

    while ((len = getline(&line, &cap, f)) != -1) {

        if (!sep_found) {
            if (strcmp(line, sep) == 0) {
                sep_found = 1;
            }

            continue;
        }

        if (strcmp(line, sep_hmac) == 0) {
            sep_hmac_found = 1;

            break;
        }

        if (*content_len + len > LEN) {
            free(line);
            warnx("Content read exceeds 4KB buffer before separator\n");

            return 1;
        }
    }
}
```



```

    memcpy(*content + *content_len, line, len);
    *content_len += len;
}
free(line);

if (len == -1) {
    warnx("Lines reading with buffering failed\n");

    return 1;
}

if (*content_len > 0 && (*content)[*content_len - 1] == '\\n') {
    *content_len -= 1;
}

if (sep_hmac_found == 0) {
    warnx("Separator not found\n");

    return 1;
}

return 0;
}

int read_hmac_line(FILE *f, char **hmac_b64, int *hmac_b64_len)
{
    size_t cap = 0;

    *hmac_b64_len = getline(hmac_b64, &cap, f);
    if (*hmac_b64_len == -1) {
        warnx("Error reading HMAC\n");

        return 1;
    }

    return 0;
}

int extract_data(FILE *fhandoff, char **content, int *content_len,
                 char **hmac_b64, int *hmac_b64_len)
{
    if (read_until_separator(fhandoff, content, content_len)
        || read_hmac_line(fhandoff, hmac_b64, hmac_b64_len)) {
        return 1;
    }

    return 0;
}

```

Code A.25: Deserialization of the authenticated file content in *User Space*.

Next,

`calculate_hmac`, `encode_base64` and `calc_and_encode_hmac` locally recalculate the authenticator from the extracted content and the same secret key used in the module, which is shared with the administrator running this program, using functionality from the `openssl` library to simplify the cryptographic calculations.

Specifically, the `HMAC` function receives the corresponding cryptographic algorithm, the key and the content together with the lengths of both variables, allowing it to directly calculate the *HMAC* corresponding to the content.

On the other hand, `EVP_ENCODE_LENGTH` and `EVP_EncodeBlock` are mainly used to encode the previously recalculated *HMAC* into *Base64* format.

```
int calculate_hmac(const unsigned char *content, int content_len,
                  unsigned char *hmac_calc, unsigned int *hmac_calc_len)
{
    if (!HMAC(EVP_sha256(), K, K_len, content, content_len, hmac_calc,
              hmac_calc_len)) {
        warnx("HMAC calculation failed\n");

        return 1;
    }

    return 0;
}

int encode_base64(unsigned char *hmac_calc, unsigned int hmac_calc_len,
                  unsigned char **hmac_b64_calc, int *hmac_b64_calc_len)
{
    *hmac_b64_calc_len = EVP_ENCODE_LENGTH(hmac_calc_len);
    *hmac_b64_calc = (unsigned char *)malloc(*hmac_b64_calc_len + 1);
    if (!*hmac_b64_calc) {
        warnx("malloc failed for HMAC encoded in base 64\n");

        return 1;
    }

    *hmac_b64_calc_len = EVP_EncodeBlock(*hmac_b64_calc, hmac_calc,
                                          hmac_calc_len);
    if (*hmac_b64_calc_len <= 0) {
        warnx("EVP_EncodeBlock failed or returned zero length");

        return 1;
    }

    return 0;
}
```

```

int calc_and_encode_hmac(const char *content, int content_len,
    unsigned char **hmac_b64_calc, int *hmac_b64_calc_len)
{
    unsigned char hmac_calc[EVP_MAX_MD_SIZE];
    unsigned int hmac_calc_len = 0;

    if (calculate_hmac((const unsigned char *)content, content_len,
        hmac_calc, &hmac_calc_len)) {
        return 1;
    }

    if (encode_base64(hmac_calc, hmac_calc_len, hmac_b64_calc,
        hmac_b64_calc_len)) {
        return 1;
    }

    return 0;
}

```

Code A.26: Local *HMAC* calculation and *Base64* encoding.

Finally, `verify_hmac` compares the *HMAC* extracted from the file with the one recalculated directly in *Base64* using `CRYPTO_memcmp`. This whole set of functionalities is integrated into a coordinated workflow in the `main` function itself, from opening the file to the final validation of the authenticated content:

```

int verify_hmac(const char *hmac_b64, int hmac_b64_len,
    unsigned char *hmac_b64_calc, int hmac_b64_calc_len)
{
    if (CRYPTO_memcmp(hmac_b64_calc, hmac_b64, hmac_b64_calc_len)) {
        warnx("\nHMAC verification failed: Invalid HMAC\n");

        return 1;
    }

    printf("\nHMAC verification successful: Valid HMAC\n");
    return 0;
}

int main(int argc, char *argv[])
{
    if (argc != 2) {
        errx(EXIT_FAILURE, "Usage: %s <path_file_handoff>\n"
            "e.g.: %s ./file_handoff", argv[0], argv[0]);
    }

    const char *path_fhandoff = argv[1];
}

```

```

FILE *fhandoff = fopen(path_fhandoff, "r");
if (!fhandoff) {
    warnx("fopen failed\n");

    goto err_fhandoff;
}

char *content = NULL;
int content_len = 0;
char *hmac_b64 = NULL;
int hmac_b64_len = 0;

if (extract_data(fhandoff, &content, &content_len,
                &hmac_b64, &hmac_b64_len)) {
    goto free_cont;
}

fclose(fhandoff);
fhandoff = NULL;

printf("Extracted kernel content:\n%s\n", content);
printf("Extracted HMAC(Base64):\n%s\n", hmac_b64);

unsigned char *hmac_b64_calc = NULL;
int hmac_b64_calc_len = 0;

if (calc_and_encode_hmac(content, content_len,
                        &hmac_b64_calc, &hmac_b64_calc_len)) {
    goto free_all;
}

free(content);
content = NULL;

printf("Calculated HMAC(Base64): \n%s\n", hmac_b64_calc);
int ext = verify_hmac(hmac_b64, hmac_b64_len, hmac_b64_calc,
                    hmac_b64_calc_len);

free(hmac_b64);
free(hmac_b64_calc);
exit(ext);

free_all:
if (hmac_b64_calc) {
    free(hmac_b64_calc);
}

if (hmac_b64) {
    free(hmac_b64);
}

```

```

free_cont:
    if (content) {
        free(content);
    }

err_fhandoff:
    if (fhandoff) {
        fclose(fhandoff);
    }

    exit(EXIT_FAILURE);
}

```

Code A.27: Authenticator comparison and main verifier flow.

Once the prototype was completed, its correct operation was validated by checking that the module could correctly generate authenticated content in *Kernel Space*, write it to the file received through the *File Handoff* mechanism and subsequently verify its integrity from *User Space* with the same secret key shared between the administrator and the *kernel*:

```

$ uname -r
6.12.41-current-bcm2711
$ cd LKM
$ chmod +x gen_k_embedded.sh
$ ./gen_k_embedded.sh
OK: generated k_embedded.h with key K (64 bytes).
$ cat k_embedded.h
const unsigned char K[] = {
    0x4a, 0xd3, 0x21, 0xda, 0xc3, 0x6c, 0x38, 0x13, 0xb1, 0x70, 0x48,
    0xac,
    0xb2, 0x6b, 0xf7, 0xe4, 0x1b, 0x7d, 0x56, 0xf2, 0xfe, 0x69, 0xb4,
    0x59,
    0x0e, 0x31, 0xd6, 0xa7, 0xf1, 0xee, 0x40, 0x8a, 0x17, 0x97, 0x4e,
    0xb0,
    0x7a, 0x40, 0x39, 0x40, 0x3f, 0x29, 0x27, 0xd8, 0xa9, 0x84, 0xda,
    0x01,
    0x12, 0x70, 0x23, 0xdd, 0x87, 0x02, 0x1b, 0xb9, 0xaf, 0x8c, 0xee,
    0xe2,
    0x90, 0x7c, 0x00, 0x10
};

const unsigned int K_len = 64;
$ make
make -C /lib/modules/6.12.41-current-bcm2711/build M=/home/LKM modules
make[1]: Entering directory '/usr/src/linux-headers-6.12.41-current-bcm2711'
CC [M] /home/LKM/crypto_fd.o
MODPOST /home/LKM/Module.symvers

```

```

CC [M] /home/LKM/crypto_fd.mod.o
LD [M] /home/LKM/crypto_fd.ko
make[1]: Leaving directory '/usr/src/linux-headers-6.12.41-current-bcm2711'
$ insmod crypto_fd.ko
$ cat /proc/fdev
Waiting for a FD to perform file handoff
$ dmesg -T | tail
...
[Fri Nov 14 18:25:38 2025] /proc/fdev created
...
[Fri Nov 14 18:28:52 2025] /proc/fdev: read handler
[Fri Nov 14 18:28:52 2025] /proc/fdev: 41 bytes sent to userspace
$ cd ..
$ gcc -c -Wshadow -Wvla -Wall mk_fhandoff.c
$ gcc -o mk_fhandoff mk_fhandoff.o
$ gcc -c -Wshadow -Wvla -Wall check_hmac.c
$ gcc -o check_hmac check_hmac.o -lcrypto
$ ./mk_fhandoff ./file_handoff /proc/fdev
File descriptor written in /proc/fdev:
3
$ dmesg -T | tail
...
[Fri Nov 14 18:31:22 2025] /proc/fdev: write handler
[Fri Nov 14 18:31:22 2025] printH: file has been written with HMAC
[Fri Nov 14 18:31:22 2025] mywrite: printH OK for fd 3 (119 bytes written)
$ cat ./file_handoff

--KERNEL-PS--
Kernel-Space: Authentic content protected by HMAC-SHA256

--HMAC--
xUPsVlgNMcm1Uxj2mac0uTD3gdkDTyeNvbXuBZjq6Bw=

$ ./check_hmac ./file_handoff
Extracted kernel content:
Kernel-Space: Authentic content protected by HMAC-SHA256

Extracted HMAC(Base64):
xUPsVlgNMcm1Uxj2mac0uTD3gdkDTyeNvbXuBZjq6Bw=
Calculated HMAC(Base64):
xUPsVlgNMcm1Uxj2mac0uTD3gdkDTyeNvbXuBZjq6Bw=

HMAC verification successful: Valid HMAC
$ cd LKM
$ rmmod crypto_fd
$ dmesg -T | tail
...
[Fri Nov 14 18:31:47 2025] /proc/fdev removed

```

Code A.28: Validation of the authentication mechanism using *HMAC-SHA256*.

This validation confirmed that the module could correctly generate authenticated content in *Kernel Space*, attach its *HMAC-SHA256* tag in *Base64* format and transfer the complete set to the real file received through the *File Handoff* mechanism. Likewise, the `check_hmac.c` program made it possible to verify from *User Space* that the authentication tag received in the file matched the one recalculated locally from the extracted content and the same shared secret key, thereby correctly validating the operation of the content-authenticity mechanism designed in this prototype.

A.1.5 Secure Reporting System for Running System Processes

Finally, once the authentication mechanism for the content to be transferred using *HMAC-SHA256* had been validated, the next step consisted of replacing the fixed test string with real content that is genuinely critical for the system and generated dynamically in *Kernel Space*. For this purpose, a global list of all running system processes was implemented, with a format inspired by the output of `ps`, so that this content could be transferred to the handoff file in an authenticated way and subsequently verified from *User Space* in the same way as in the previous prototype. This module was designed in the `crypto_ps.c` file.

At this point, all the infrastructure developed in the previous prototypes is reused without substantial changes, including the *File Handoff* mechanism, the joint writing of the content and its *HMAC* in *Base64*, as well as the utilities for writing to the `/proc` file and verifying the result in *User Space*.

Initially, new constants and headers had to be incorporated into the module:

```
#include <linux/cred.h>
#include <linux/pid.h>
#include <linux/rcupdate.h>
#include <linux/security.h>
#include <linux/sched.h>
#include <linux/nsproxy.h>
#include <linux/pid_namespace.h>
enum {
    MAX_SIZE = 1024 * 40,
    TIPIC_HMACB64_SIZE = 44,
    UID_SIZE = 12,
```

```

    UIDNS_SIZE = 14,
    PID_SIZE = 12,
    PIDNS_SIZE = 14,
    GID_SIZE = 11,
    CMD_SIZE = 50,
    PS_LINE_SIZE = UID_SIZE + UID_SIZE + UIDNS_SIZE +
        PID_SIZE + PID_SIZE + PIDNS_SIZE + GID_SIZE + CMD_SIZE,
};
static const char header[] =
    "UID_KERNEL UID_LOCAL UID_NS      PID_KERNEL PID_LOCAL PID_NS
      GID      COMMAND\n";
static const int header_len = sizeof(header) - 1;

```

Code A.29: Headers and constants for generating the process list.

In order to build a list of all processes running on the system and in its different namespaces, it was necessary to extend the module headers again. Specifically, `<linux/cred.h>` and `<linux/security.h>` allow the module to access the credentials of each process, `<linux/pid.h>`, `<linux/sched.h>` and `<linux/pid_namespace.h>` provide the definitions required to traverse the global task list and obtain their identifiers in the different namespaces, while `<linux/rcupdate.h>` provides the *RCU* synchronization mechanism required to perform this traversal safely. On the other hand, the auxiliary constants defined bound the maximum size of the generated content, the size reserved for each serialized field and the basic structure of the generated process table with its corresponding metadata.

Once the headers and constants required to structure the content to be transferred had been established, two auxiliary functions were incorporated to format and dynamically and safely build the complete process table inside a single output *buffer*:

```

static int pad_str_right(char *buf, int curr_len, int buf_len, char
    pad_char)
{
    int padding = 0;

    if (curr_len >= buf_len) {
        padding = 0;

        curr_len = buf_len;
    } else {
        padding = buf_len - curr_len;
    }
}

```



```

    if (padding > 0) {
        memset(buf + curr_len, pad_char, padding);

        curr_len += padding;
    }

    return buf_len;
}

static int safe_chunk(u8 *dst, int *current_len, char *src, int src_len)
{
    int max_len = MAX_SIZE - *current_len;

    if (max_len < src_len)
        return -ENOSPC;

    memcpy(dst + *current_len, src, src_len);

    *current_len += src_len;

    return src_len;
}

```

Code A.30: Auxiliary functions for formatting and safely building serialized content.

The `pad_str_right` function pads each textual field with fill characters until a fixed width is reached, thus guaranteeing a homogeneous tabular representation of the metadata of each process in the table.

For its part, `safe_chunk` sequentially incorporates each fragment of information into the global *buffer* that will contain the final table, previously checking that there is enough available space before performing the copy.

Next, the main function responsible for traversing the global system task list and building the complete table in memory was implemented:

```

static int get_ps(u8 **cont, int *cont_len)
{
    struct task_struct *task;

    *cont = (u8 *) kmalloc(MAX_SIZE, GFP_KERNEL);
    if (!*cont) {
        printk(KERN_ERR "get_ps: kmalloc failed\n");

        return -ENOMEM;
    }
}

```

```

if (safe_chunk(*cont, cont_len, (char *)header, header_len) < 0)
    goto out_fail;

rcu_read_lock();

for_each_process(task) {
    if (*cont_len >= MAX_SIZE - PS_LINE_SIZE -
        sep_len - sep_hmac_len - TIPIC_HMACB64_SIZE) {
        printk(KERN_WARNING "get_ps: Buffer full -> Truncating\n");
        goto out_fail;
    }

    if (ps_data(task, *cont, cont_len)) {
        printk(KERN_ERR "get_ps: Extracting process data failed\n");

        goto out_fail;
    }
}

rcu_read_unlock();

printk(KERN_INFO "get_ps: Generated ps-like output of %d bytes.\n",
        *cont_len);

return 0;

out_fail:
rcu_read_unlock();

if (*cont)
    kfree(*cont);

*cont = NULL;
*cont_len = 0;

return -ENOSPC;
}

```

Code A.31: Construction of the global process list in *kernel* memory.

This code acts as the main core for generating the content to be transferred, initially allocating a fixed maximum-size *buffer* where the complete list will be stored and first incorporating the header previously defined in the constants.

It then traverses the global list of active system processes using the `for_each_process` macro under *RCU* read protection, thereby guaranteeing safe access in the presence

of multiple concurrent accesses to the critical `task_struct` structures while their metadata is extracted. During this traversal, the code checks that there is still enough available space before incorporating each new entry, delegating the retrieval and serialization of the fields corresponding to each process to `ps_data`.

Finally, if the traversal finishes correctly, it returns the complete generated content in `cont`, which will be used in `printh`, and its total length in `cont_len`. On the other hand, in case of failure, the allocated memory is released and the corresponding error code is returned.

```
static int ps_data(struct task_struct *task, u8 *cont, int *cont_len)
{
    char kuid_str[UID_SIZE];
    char luid_str[UID_SIZE];
    char uidns_str[UIDNS_SIZE];
    char kpid_str[PID_SIZE];
    char lpid_str[PID_SIZE];
    char pidns_str[PIDNS_SIZE];
    char gid_str[GID_SIZE];
    char comm_str[CMD_SIZE];

    int kuid_len = get_kuid_str(task_uid(task), kuid_str);
    if (kuid_len <= 0)
        goto out_fail;

    if (safe_chunk(cont, cont_len, kuid_str, UID_SIZE) < 0)
        goto out_fail;

    int luid_len = get_luid_str(task, luid_str);
    if (luid_len <= 0)
        goto out_fail;

    if (safe_chunk(cont, cont_len, luid_str, UID_SIZE) < 0)
        goto out_fail;

    int uidns_len = get_uidns_str(task, uidns_str);
    if (uidns_len <= 0)
        goto out_fail;

    if (safe_chunk(cont, cont_len, uidns_str, UIDNS_SIZE) < 0)
        goto out_fail;

    int kpid_len = get_kpid_str(task_pid_nr(task), kpid_str);
    if (kpid_len <= 0)
        goto out_fail;

    if (safe_chunk(cont, cont_len, kpid_str, PID_SIZE) < 0)
        goto out_fail;
```

```

    int lpid_len = get_lpid_str(task, lpid_str);
    if (lpid_len <= 0)
        goto out_fail;

    if (safe_chunk(cont, cont_len, lpid_str, PID_SIZE) < 0)
        goto out_fail;

    int pidns_len = get_pidns_str(task, pidns_str);
    if (pidns_len <= 0)
        goto out_fail;

    if (safe_chunk(cont, cont_len, pidns_str, PIDNS_SIZE) < 0)
        goto out_fail;

    int gid_len = get_gid_str(task, gid_str);
    if (gid_len <= 0)
        goto out_fail;

    if (safe_chunk(cont, cont_len, gid_str, GID_SIZE) < 0)
        goto out_fail;

    int comm_len = get_command_str(task, comm_str);
    if (comm_len <= 0)
        goto out_fail;

    if (safe_chunk(cont, cont_len, comm_str, comm_len) < 0)
        goto out_fail;

    return 0;

out_fail:
    return 1;
}

```

Code A.32: Serialization of the metadata of one process.

The `ps_data` function encapsulates the construction of a single row of the final table corresponding to the task currently traversed in `for_each_process`, sequentially extracting and serializing the main metadata of each specific process.

For this purpose, the extraction of each metadata item was encapsulated in independent functions:

Next, several auxiliary functions were implemented to extract and format the different user identifiers associated with each process, distinguishing between the global identifier

visible from the *kernel*, its relative value inside the process namespace and the identifier of the namespace itself:

```
static int get_kuid_str(kuid_t uid_struct, char *uid_str)
{
    int uid_len = 0;

    int uid = from_kuid(&init_user_ns, uid_struct);

    uid_len = snprintf(uid_str, UID_SIZE, "%d", uid);

    pad_str_right(uid_str, uid_len, UID_SIZE, ' ');

    return uid_len;
}

static int get_luid_str(struct task_struct *task, char *uid_str)
{
    int uid_len = 0;

    const struct cred *cred = get_task_cred(task);
    if (!cred) {
        printk(KERN_ERR "get_luid_str: No credentials found for task\n");

        return -ENOSPC;
    }

    int uid = from_kuid(cred->user_ns, cred->uid);

    put_cred(cred);

    uid_len = snprintf(uid_str, UID_SIZE, "%d", uid);

    pad_str_right(uid_str, uid_len, UID_SIZE, ' ');

    return uid_len;
}

static int get_uidns_str(struct task_struct *task, char *uidns_str)
{
    const struct cred *cred = get_task_cred(task);
    if (!cred) {
        printk(KERN_ERR "get_uidns_str: No credentials found for task\n");

        return -ENOSPC;
    }

    unsigned int uid_ns = cred->user_ns->ns.inum;

    put_cred(cred);
}
```

```

    int uidns_len = snprintf(uidns_str, UIDNS_SIZE, "%u", uid_ns);

    pad_str_right(uidns_str, uidns_len, UIDNS_SIZE, ' ');

    return uidns_len;
}

```

Code A.33: User identifiers of each process.

These functions make it possible to obtain and format the user identifiers (*UIDs*) associated with each process. Specifically, `get_kuid_str` obtains the global UID visible from the *kernel*, `get_luid_str` obtains the UID relative to the process namespace and `get_uidns_str` obtains the user namespace to which those credentials belong.

```

static int get_kpid_str(int pid, char *pid_str)
{
    int pid_len = snprintf(pid_str, PID_SIZE, "%d", pid);

    pad_str_right(pid_str, pid_len, PID_SIZE, ' ');

    return pid_len;
}

static int get_lpid_str(struct task_struct *task, char *pid_str)
{
    struct pid_namespace *ns = task_active_pid_ns(task);
    if (!ns) {
        printk(KERN_ERR "get_lpid_str: No PID namespace found for task\n");
        return -ENOSPC;
    }

    int pid = task_pid_nr_ns(task, ns);

    int pid_len = snprintf(pid_str, PID_SIZE, "%d", pid);

    pad_str_right(pid_str, pid_len, PID_SIZE, ' ');

    return pid_len;
}

static int get_pidns_str(struct task_struct *task, char *pidns_str)
{
    struct pid_namespace *pid_ns = task_active_pid_ns(task);
    if (!pid_ns) {
        printk(KERN_ERR "get_pidns_str: No PID namespace found for task\n");
        return -ENOSPC;
    }
}

```

```

    int pidns_len = snprintf(pidns_str, PIDNS_SIZE, "%u", pid_ns->ns.inum);

    pad_str_right(pidns_str, pidns_len, PIDNS_SIZE, ' ');

    return pidns_len;
}

```

Code A.34: Process identifiers of each task.

In this case, these functions make it possible to obtain and format the process identifiers (*PIDs*) of each task. In particular, `get_kpid_str` obtains the global PID visible from the *kernel*, `get_lpid_str` retrieves the PID relative to the active process namespace and `get_pidns_str` identifies the process namespace to which the analyzed task belongs.

```

static int get_gid_str(struct task_struct *task, char *gid_str)
{
    int gid_len = 0;

    const struct cred *cred = get_task_cred(task);
    if (!cred)
        return -ENOSPC;

    int gid = from_kgid(&init_user_ns, cred->gid);
    put_cred(cred);

    gid_len = snprintf(gid_str, GID_SIZE, "%d", gid);

    pad_str_right(gid_str, gid_len, GID_SIZE, ' ');

    return gid_len;
}

static int get_command_str(struct task_struct *task, char *comm_str)
{
    int comm_len = 0;

    if (task->mm == NULL) {
        comm_len = snprintf(comm_str, CMD_SIZE, "[%s]\n", task->comm);

        goto cmd_finished;
    }

    struct file *exe_file = task->mm->exe_file;

```

```

if (exe_file) {
    char *path = d_path(&exe_file->f_path, comm_str, CMD_SIZE);

    if (!IS_ERR(path)) {
        comm_len = snprintf(comm_str, CMD_SIZE, "%s\n", path);

        goto commd_finished;
    }
}

comm_len = snprintf(comm_str, CMD_SIZE, "%s\n", task->comm);

commd_finished:
if (comm_len >= CMD_SIZE)
    comm_len = CMD_SIZE - 1;

return comm_len;
}

```

Code A.35: Group identifiers and command name associated with each process.

Finally, these functions complete the process information with the group identifier and the command associated with its execution. Specifically, `get_gid_str` obtains and formats the global group identifier associated with the process, and `get_command_str` retrieves the command corresponding to its execution, distinguishing between user processes with an accessible executable, *kernel* threads and cases where only the name stored in `task->comm` can be used.

Once the dynamic generation of metadata for the active system processes had been integrated into the authenticated write flow to the handoff file, the complete behavior of the final prototype was validated by incorporating container functionality to simulate different namespaces in the system:

```

$ uname -r
6.12.41-current-bcm2711
$ cd LKM
$ chmod +x gen_k_embedded.sh
$ ./gen_k_embedded.sh
OK: generated k_embedded.h with key K (64 bytes).
$ make
make -C /lib/modules/6.12.41-current-bcm2711/build M=/home/LKM modules
make[1]: Entering directory '/usr/src/linux-headers-6.12.41-current-bcm2711'
  CC [M] /home/LKM/crypto_ps.o
  MODPOST /home/LKM/Module.symvers
  CC [M] /home/LKM/crypto_ps.mod.o
  LD [M] /home/LKM/crypto_ps.ko
make[1]: Leaving directory '/usr/src/linux-headers-6.12.41-current-bcm2711'
$ insmod crypto_ps.ko
$ cat /proc/fdev
LKM ready to receive file descriptors from user.
$ dmesg -T | tail
...
[Thu Nov 27 18:42:11 2025] New proc file created: /proc/fdev
...
[Thu Nov 27 18:42:11 2025] /proc/fdev: read handler
[Thu Nov 27 18:42:11 2025] /proc/fdev myread: read 48 bytes by userspace
$ cd ..
$ gcc -c -Wshadow -Wvla -Wall mk_fhandoff.c
$ gcc -o mk_fhandoff mk_fhandoff.o

```



```

$ gcc -g -c -Wshadow -Wvla -Wall check_hmac.c
$ gcc -g -o check_hmac check_hmac.o -lcrypto
$ cat /etc/docker/daemon.json
{
    "userns-remap": "default"
}
$ systemctl restart docker
$ docker run -d --name docker1 debian:stable sleep 10000
7fd1c45a07c0...
$ docker run -d --name docker2 debian:stable sleep 10000
77276c1a7026...
$ docker run -d --name docker3 debian:stable sleep 10000
4a4421829e80...
$ docker ps
CONTAINER ID   IMAGE      COMMAND                  CREATED        STATUS        NAMES
4a4421829e80   debian:stable "sleep 10000"           29 seconds ago Up 28 seconds docker3
77276c1a7026   debian:stable "sleep 10000"           35 seconds ago Up 34 seconds docker2
7fd1c45a07c0   debian:stable "sleep 10000"           About a minute ago Up 58 seconds docker1
$ ./mk_fhandoff ./file_handoff /proc/fdev
File descriptor written in /proc/fdev:
3
$ dmesg -T | tail
...
[Thu Nov 27 18:43:49 2025] /proc/fdev: write handler
[Thu Nov 27 18:43:49 2025] /proc/fdev write: written 1 bytes from the user
[Thu Nov 27 18:43:49 2025] get_ps: Generated ps-like output of 21251 bytes.
[Thu Nov 27 18:43:49 2025] printH: file has been written with HMAC
[Thu Nov 27 18:43:49 2025] mywrite: printH OK for fd 3 (21320 bytes written)
$ cat ./file_handoff

--KERNEL-PS--
UID_KERNEL  UID_LOCAL  UID_NS      PID_KERNEL  PID_LOCAL  PID_NS      GID      COMMAND
0           0          4026531837  1           1          4026531836  0        /usr/lib/systemd/systemd
0           0          4026531837  2           2          4026531836  0        [kthreadd]
0           0          4026531837  3           3          4026531836  0        [pool_workqueue_]
...
113        113        4026531837  1440        1440        4026531836  119      /usr/sbin/chronyd
113        113        4026531837  1448        1448        4026531836  119      /usr/sbin/chronyd
0           0          4026531837  1489        1489        4026531836  1000     /usr/libexec/gdm-session-worker
1000       1000       4026531837  1496        1496        4026531836  1000     /usr/lib/systemd/systemd
1000       1000       4026531837  1497        1497        4026531836  1000     /usr/lib/systemd/systemd-executor
...
0           0          4026531837  8211        8211        4026531836  0        /usr/bin/dockerd
0           0          4026531837  8212        8212        4026531836  0        [kworker/1:1]
0           0          4026531837  8239        8239        4026531836  0        [kworker/1:2H]
0           0          4026531837  8500        8500        4026531836  0        /usr/bin/containerd-shim-runc-v2
100000     0          4026532674  8520        1           4026532678  100000   /usr/bin/sleep
0           0          4026531837  8532        8532        4026531836  0        [kworker/2:0]
0           0          4026531837  8596        8596        4026531836  0        /usr/bin/containerd-shim-runc-v2
100000     0          4026532754  8617        1           4026532758  100000   /usr/bin/sleep
0           0          4026531837  8667        8667        4026531836  0        [kworker/2:2H]
0           0          4026531837  8678        8678        4026531836  0        /usr/bin/containerd-shim-runc-v2
100000     0          4026532834  8698        1           4026532838  100000   /usr/bin/sleep
0           0          4026531837  8741        8741        4026531836  0        [kworker/u20:2]
0           0          4026531837  8935        8935        4026531836  0        [kworker/0:0]
0           0          4026531837  8947        8947        4026531836  0        [kworker/u16:3]
0           0          4026531837  8989        8989        4026531836  0        [kworker/3:2]
0           0          4026531837  9033        9033        4026531836  0        /home/noel/Desktop/TFG/Project/mk_fhandoff

--HMAC--
ckvLUckcgm6t6RfFdGzuvgnAEgt9rLPJ5q9FYJoFKAW0=

$ ./check_hmac ./file_handoff
Extracted kernel content:
UID_KERNEL  UID_LOCAL  UID_NS      PID_KERNEL  PID_LOCAL  PID_NS      GID      COMMAND
0           0          4026531837  1           1          4026531836  0        /usr/lib/systemd/systemd
0           0          4026531837  2           2          4026531836  0        [kthreadd]
0           0          4026531837  3           3          4026531836  0        [pool_workqueue_]
...
113        113        4026531837  1440        1440        4026531836  119      /usr/sbin/chronyd
113        113        4026531837  1448        1448        4026531836  119      /usr/sbin/chronyd
0           0          4026531837  1489        1489        4026531836  1000     /usr/libexec/gdm-session-worker
1000       1000       4026531837  1496        1496        4026531836  1000     /usr/lib/systemd/systemd
1000       1000       4026531837  1497        1497        4026531836  1000     /usr/lib/systemd/systemd-executor
...
0           0          4026531837  8211        8211        4026531836  0        /usr/bin/dockerd
0           0          4026531837  8212        8212        4026531836  0        [kworker/1:1]
0           0          4026531837  8239        8239        4026531836  0        [kworker/1:2H]
0           0          4026531837  8500        8500        4026531836  0        /usr/bin/containerd-shim-runc-v2
100000     0          4026532674  8520        1           4026532678  100000   /usr/bin/sleep
0           0          4026531837  8532        8532        4026531836  0        [kworker/2:0]
0           0          4026531837  8596        8596        4026531836  0        /usr/bin/containerd-shim-runc-v2
100000     0          4026532754  8617        1           4026532758  100000   /usr/bin/sleep
0           0          4026531837  8667        8667        4026531836  0        [kworker/2:2H]
0           0          4026531837  8678        8678        4026531836  0        /usr/bin/containerd-shim-runc-v2
100000     0          4026532834  8698        1           4026532838  100000   /usr/bin/sleep
0           0          4026531837  8741        8741        4026531836  0        [kworker/u20:2]
0           0          4026531837  8935        8935        4026531836  0        [kworker/0:0]
0           0          4026531837  8947        8947        4026531836  0        [kworker/u16:3]
0           0          4026531837  8989        8989        4026531836  0        [kworker/3:2]
0           0          4026531837  9033        9033        4026531836  0        /home/noel/Desktop/TFG/Project/mk_fhandoff

Extracted HMAC(Base64):
ckvLUckcgm6t6RfFdGzuvgnAEgt9rLPJ5q9FYJoFKAW0=
Calculated HMAC(Base64):
ckvLUckcgm6t6RfFdGzuvgnAEgt9rLPJ5q9FYJoFKAW0=

HMAC verification successful: Valid HMAC
$ cd LKM
$ rmmod crypto_ps
$ dmesg -T | tail
...
[Thu Nov 27 18:46:06 2025] Proc file deleted: /proc/fdev

```

Code A.36: Validation of the secure metadata reporting system for active processes authenticated using *HMAC-SHA256*.

It should be noted that the complete results of `cat ./file_handoff` and `./check_hmac ./file_handoff` are not shown due to the large number of active processes in the system, which generates a very long table. For this reason, the validation only presents a representative excerpt of the transferred and verified content, with the *HMAC* and the reported sizes corresponding to the complete output generated during the real execution.

This validation confirmed that the module could correctly generate a global list of the active system processes from *Kernel Space*. It was also verified that the list included processes running in other *Namespaces*, as observed by running several containers with *Docker*, showing differences with respect to the metadata of the rest of the processes, mainly in the values related to `UID_LOCAL`, `PID_LOCAL`, `UID_NS` and `PID_NS`.

On the other hand, as in the previous prototype, the `check_hmac.c` program correctly validates from *User Space* that the *HMAC-SHA256* tag in *Base64* matches the one recalculated over the complete process table, thereby guaranteeing secure and authenticated reporting of critical system data between both regions of the system.

Bibliography

- [1] Schneier, Bruce, “A Plea for Simplicity,” Schneier on Security, online essay, November 1999. [Online]. Available: https://www.schneier.com/essays/archives/1999/11/a_plea_for_simplicit.html
- [2] Souppaya, Murugiah and Scarfone, Karen, “Guide to Malware Incident Prevention and Handling for Desktops and Laptops,” National Institute of Standards and Technology (NIST), Special Publication (SP), July 2013. [Online]. Available: <https://csrc.nist.gov/pubs/sp/800/83/r1/final>
- [3] Akamai Technologies, “What is malware?” Web article, accessed on March 10, 2026. [Online]. Available: <https://www.akamai.com/es/glossary/what-is-malware>
- [4] Fonticons, Inc., “Font Awesome Free,” Website, accessed on March 13, 2026. [Online]. Available: <https://fontawesome.com/>
- [5] MITRE ATT&CK, “MITRE ATT&CK,” Official source of information on adversary tactics and techniques, The MITRE Corporation, Tech. Rep., 2026, accessed on March 26, 2026. [Online]. Available: <https://attack.mitre.org/>
- [6] European Union Agency for Cybersecurity (ENISA), “ENISA Threat Landscape 2025,” ENISA, Tech. Rep., October 2025, accessed on March 10, 2026. [Online]. Available: <https://www.enisa.europa.eu/publications/enisa-threat-landscape-2025>
- [7] Joint Task Force, “Security and Privacy Controls for Information Systems and Organizations,” National Institute of Standards and Technology (NIST), Special Publication (SP), September 2020. [Online]. Available: <https://csrc.nist.gov/publications/detail/sp/800-53/rev-5/final>
- [8] Scarfone, Karen and Mell, Peter, “Guide to Intrusion Detection and Prevention Systems (IDPS),” National Institute of Standards and Technology (NIST), Special Publication (SP), February 2007. [Online]. Available: <https://csrc.nist.gov/publications/detail/sp/800-94/final>

- [9] Soriano-Salvador, Enrique and Guardiola Múzquiz, Gorka and González Gómez, Juan, “User-space library rootkits revisited: Are user-space detection mechanisms futile?” Universidad Rey Juan Carlos, Tech. Rep., June 2025. [Online]. Available: <https://arxiv.org/abs/2506.07827>
- [10] National Institute of Standards and Technology (NIST), “Rootkit,” Cybersecurity glossary, accessed on March 14, 2026. [Online]. Available: <https://csrc.nist.gov/glossary/term/rootkit>
- [11] Guardiola Múzquiz, Gorka and Soriano Salvador, Enrique, *Fundamentos de Sistemas Operativos: Una aproximación práctica usando Linux*, 3rd ed. GSyC, Universidad Rey Juan Carlos, November 2025. [Online]. Available: <https://honecomp.github.io/librossoo.html>
- [12] VARGUX, “File:Unix history-simple es.svg,” Vectorial file SVG, June 2023, published on June 26, 2023. CC BY-SA 4.0 license. Accessed on March 13, 2026. [Online]. Available: https://commons.wikimedia.org/wiki/File:Unix_history-simple_es.svg
- [13] Wikipedia contributors, “Protection ring,” Wikipedia, online article, January 2026. [Online]. Available: [https://es.wikipedia.org/wiki/Anillo_\(seguridad_inform%C3%A1tica\)](https://es.wikipedia.org/wiki/Anillo_(seguridad_inform%C3%A1tica))
- [14] Love, Robert, *Linux Kernel Development*, 3rd ed. Developer’s Library, Addison-Wesley, 2010, ch. 1, 13. [Online]. Available: <https://github.com/swadhinsekhar/books/blob/master/Linux%20Kernel%20Development%203rd%20Edition%20-%20Love%20-%202010.pdf>
- [15] Silberschatz, Abraham and Galvin, Peter Baer and Gagne, Greg, *Fundamentos de sistemas operativos*, 9th ed. Madrid, Spain: McGraw-Hill Interamericana, 2013, ch. 9. [Online]. Available: <https://books.google.es/books?id=b2qynQEACAAJ>
- [16] Albritton, William McDaniel, “Module 2, Session 7 - Linux File System Structure,” University of Hawaii, Tech. Rep., 2016, accessed on February 24, 2026. [Online]. Available: https://tldp.org/LDP/intro-linux/html/chap_03.html
- [17] Gmkziz, “Understanding Linux Namespaces,” Hackerstack, online article, 2024, accessed on February 25, 2026. [Online]. Available: <https://www.hackerstack.org/understanding-linux-namespaces/>

- [18] *namespaces(7)* - *Linux Programmer's Manual*, Linux Kernel Organization, January 2024, available by running `man 7 namespaces` in a Linux terminal or in the official Kernel documentation. Last update accessed. [Online]. Available: <https://man7.org/linux/man-pages/man7/namespaces.7.html>
- [19] Docker, Inc., "What is Docker?" Docker Docs, Tech. Rep., 2026, accessed on March 13, 2026. [Online]. Available: <https://docs.docker.com/get-started/docker-overview/>
- [20] —, "Docker Brand Guidelines and Logo Kit," Official brand resources page, 2026, official logo kit. Accessed on March 13, 2026. [Online]. Available: <https://www.docker.com/company/newsroom/media-resources/>
- [21] GeeksforGeeks, *Linux/Unix — Linux Kernel Module Programming (Hello World Program)*, Online article, January 2019, accessed on March 7, 2026. [Online]. Available: <https://www.geeksforgeeks.org/linux-unix/linux-kernel-module-programming-hello-world-program/>
- [22] Deeppadmani, "Understanding the /proc file system in Linux," Online article, February 2025, accessed on March 8, 2026. [Online]. Available: <https://medium.com/@deeppadmani98.2021/understanding-the-proc-file-system-in-linux-90746e3ba76a>
- [23] Raspberry Pi Ltd, "Raspberry Pi 4 Model B specifications," Raspberry Pi, Tech. Rep., 2019, accessed on March 23, 2026. [Online]. Available: <https://www.raspberrypi.com/products/raspberry-pi-4-model-b/specifications/>
- [24] —, "Raspberry Pi 4," Raspberry Pi, Tech. Rep., 2019, accessed on March 23, 2026. [Online]. Available: <https://www.raspberrypi.com/products/raspberry-pi-4-model-b/>
- [25] Armbian Community, "Armbian Documentation," Armbian Documentation, Tech. Rep., 2026, accessed on March 23, 2026. [Online]. Available: <https://docs.armbian.com/>
- [26] Prima produkcija, "Raspberry Pi - Armbian," Armbian Documentation, Tech. Rep., 2026, accessed on March 23, 2026. [Online]. Available: <https://www.armbian.com/rpi4b/>

- [27] Armbian Community, “Armbian Linux branding material,” Official GitHub repository, 2026, accessed on March 23, 2026. [Online]. Available: <https://github.com/armbian/stationery/blob/master/logo/PDF.PDF>
- [28] NIST CSRC, “Rootkit - Glossary,” Computer Security Resource Center based on NIST SP 800-83 Rev. 1 under Rootkit resource, accessed on March 30, 2026. [Online]. Available: <https://csrc.nist.gov/glossary/term/rootkit>
- [29] Marier, Francois and Gouin, Patrick, *unhide-linux(8) — unhide — forensic tool to find hidden processes*, Debian Manpages, January 2017, accessed on March 26, 2026. [Online]. Available: <https://manpages.debian.org/testing/unhide/unhide-linux.8.en.html>
- [30] Debian Manpages, “chkrootkit(8) — Scan the system for signs of rootkits,” 2026, accessed on March 27, 2026. [Online]. Available: <https://manpages.debian.org/testing/chkrootkit/chkrootkit.8.en.html>
- [31] K. Thompson, “Reflections on Trusting Trust,” *Communications of the ACM*, vol. 27, no. 8, pp. 761–763, August 1984. [Online]. Available: <https://doi.org/10.1145/358198.358210>
- [32] Parmar, Kelvin, “Deep Dive into Linux Namespaces: Building Your Own Containers,” Hashnode, online article, August 2024, accessed on March 2, 2026. [Online]. Available: <https://kelvin-parmar.hashnode.dev/deep-dive-into-linux-namespaces-building-your-own-containers>